POLITECNICO MILANO 1863 | SCHOOL OF INDUSTRIAL AND INFORMATION ENGINEERING

Progetto Aerodinamico – A.A. 2024-2025



Formation-flight Optimization for a SubSonic mission with Extended Range

Federico Poleni, 10788059
Giacomo Visconti, 10791073
Marco Zappoli, 10710044

Lecturers:
Dr. Maurizio Boffadossi

Abstract

The FOSSER (Formation-flight Optimization for a SubSonic mission with Extended Range) project aims to evaluate and enhance the aerodynamic performance of fighter aircraft operating in two-ship formation flight, leveraging low-fidelity modeling techniques based on a traditional vortex lattice method (VLM). The study begins with a comparative analysis of multiple fighter jet configurations, representative of different technological generations, to quantify the potential aerodynamic benefits, in terms of a reduction in induced drag, achievable through formation flight.

Using the features of AVL, the aerodynamic interactions between paired aircraft are assessed in a reference spatial configuration, allowing a first comparison between several aircraft types under an approximate trim condition. Selected vehicles, namely the Fiat G.91R, the McDonnell Douglas F/A-18C Hornet and the Lockheed Martin/Boeing F-22A Raptor, are then subjected to aerodynamic shape optimization focused on the wing planform. This optimization is performed using a Particle Swarm Optimization (PSO) algorithm on the grounds of superior performance, targeting improved lift-to-induced-drag ratios.
Following wing shape optimization, the relative positioning of the two aircraft is further refined to maximize aerodynamic synergies. Due to the computational effort of a single evaluation, this task is carried out by extending the results of a Latin hypercube sampling (LHS) to the entire space of variables by means of the Kriging technique.
In the final phase of the project, the effects of the optimized planform are analyzed to assess potential benefits during refueling operations. The flight of a F/A-18 and a F-22 in the wake of a Boeing KC-135R Stratotanker is simulated in AVL to this end. A simple application of the Breguet range equation shows that a two-aircraft formation can significantly extend ferry range, with further gains when optimized geometries are used.

The results demonstrate that significant range extension can be achieved by the optimized aircraft, although the best wing geometry for formation flight, except for the G.91, is likely to be subject to detrimental aeroelastic behavior and might require major modifications to the existing aircraft. Comparisons are satisfactorily drawn between the findings for the F/A-18 and experimental studies concerning airplanes of the same family.

Contents

Abstract **1**

1 Introduction **5**
1.1 Background and state of the art . . . 5
1.2 Aim of this research . . . 6

2 Problem formulation **8**
2.1 The classical lifting-line theory . . . 8
2.2 The lifting-surface theory and the vortex lattice method . . . 8
2.3 Downwash in formation flight . . . 9

3 Modeling and simulation workflow **10**
3.1 Geometry modeling . . . 10
3.2 Aerodynamic analysis . . . 10
3.3 Optimization and post-processing . . . 10

4 Generation and comparison of aircraft geometries **11**
4.1 The investigated aircraft types . . . 11
4.2 Reconstructing the geometries in an AVL-compatible format . . . 11
4.3 Some notes about the coordinate system for formation flight . . . 13
4.4 Reference trimmed-cruise conditions . . . 15
4.5 Downselection of aircraft types to be analyzed . . . 15

5 Wing optimization **17**
5.1 Fitness function and penalty terms . . . 17
5.1.1 Fitness function . . . 17
5.1.2 Penalty functions . . . 18
5.2 Optimization methods . . . 18
5.2.1 Particle swarm optimization (PSO) . . . 18
5.2.2 Genetic algorithm (GA) . . . 19
5.2.3 Computational cost and optimizer selection . . . 19
5.3 Optimization results . . . 20
5.3.1 McDonnell Douglas F/A-18C Hornet . . . 21
5.3.2 Lockheed Martin/Boeing F-22A Raptor . . . 23
5.3.3 Fiat G.91R . . . 23
5.4 Trimmed-cruise condition for optimized configurations . . . 26

6 Influence of the relative position **27**
6.1 Sensitivity of the results with respect to formation-position . . . 28

7 Aerial refueling and range extension **37**
7.1 Fighter refueling trim study . . . 37
7.1.1 Limitations and caveats . . . 38
7.1.2 Refueling results . . . 38
7.2 Range extension via formation flight . . . 39

8 Conclusions **42**

Bibliography **42**

A Aircraft geometries **44**

B MATLAB codes **57**

List of Figures

1 The formation flight of the North American XB-70A Valkyrie (Air Vehicle No. 2) on 8 June 1966, moments before the fatal midair collision. . . . 6
2 The Italian Air Force aerobatic team *Pattuglia Acrobatica Nazionale (PAN) Frecce Tricolori* performing in a V formation. . . . 7
3 Prandtl's lifting-line model: a finite wing replaced by a bound vortex with trailing vortices. 8
4 Illustrations of formation flight: (a) operational use in combat air patrol and (b) experimental investigation of wingtip vortices and wake interactions. . . . 9
5 AVL-rendered geometry of the McDonnell Douglas F/A-18C fighter. . . . 12
6 Airfoils used during the particle-swarm optimization of the Fiat G.91R. . . . 13
7 Coordinate system used in the Autonomous Formation Flight Project [1] to specify the position of the trailing aircraft with respect to the formation leader. . . . 14
8 Evolution of the particle swarm during the optimization of the trim angle of attack for the F/A-18 formation. . . . 16
9 (a) Wall-clock time required to complete a single optimization run (50 PSO iterations or 50 GA generations; one iteration = 120 AVL calls) on the F/A-18 benchmark as a function of allocated MATLAB® cores. (b) Corresponding parallel speedup for PSO and GA obtained from the measured times in (a): $S_n = T_1/T_n$. . . . 20
10 Solution of the particle swarm optimization for the McDonnell Douglas F/A-18C Hornet. 21
11 Comparison of pressure coefficient differences (ΔC_p) across the McDonnell Douglas F/A-18C Hornet wing before and after PSO optimization. . . . 21
12 Solution of the particle swarm optimization for the Lockheed Martin/Boeing F-22A Raptor. . . . 23
13 Comparison of pressure coefficient differences (ΔC_p) across the Lockheed Martin/Boeing F-22A Raptor wing before and after PSO optimization. . . . 24
14 Solution of the particle swarm optimization for the Fiat G.91. . . . 24
15 Comparison of pressure coefficient differences (ΔC_p) across the Fiat G.91R wing before and after PSO optimization. . . . 25
16 Summary of induced drag ($C_{D,i}$) reduction, relative to the non-optimized solo flight, shown as contour plots as a function of Y-Z position for the McDonnell Douglas F/A-18C Hornet. . . . 30
17 Comparison of the effect of relative formation position on the induced drag change $\Delta C_{D,i}$ (%) of the McDonnell Douglas F/A-18C Hornet trailing aircraft. . . . 31
18 Induced-drag reduction trends along X for the McDonnell Douglas F/A-18C Hornet trailing and leader aircraft. . . . 32
19 Comparison of the effect of relative formation position on the induced drag change $\Delta C_{D,i}$ (%) of the Lockheed Martin/Boeing F-22A Raptor trailing aircraft. . . . 33
20 Induced-drag reduction trends along X for the Lockheed Martin/Boeing F-22A Raptor trailing and leader aircraft. . . . 34
21 Comparison of the effect of relative formation position on the induced drag change $\Delta C_{D,i}$ (%) of the Fiat G.91R trailing aircraft. . . . 35
22 Induced-drag reduction trends along X for the Fiat G.91R trailing and leader aircraft. 36
23 A NASA SR-71 refuels from an Edwards Air Force Base KC-135. . . . 37
24 AVL representation of the Lockheed Martin/Boeing F-22A Raptor refueling behind the Boeing KC-135R Stratotanker. . . . 39
25 Ferry range comparison for the Lockheed Martin/Boeing F-22A Raptor. . . . 41
26 Ferry range comparison for the McDonnell Douglas F/A-18C Hornet. . . . 41
27 Aermacchi MB-339A. . . . 44
28 Fiat G.91R. . . . 45
29 North American F-100 Super Sabre. . . . 46
30 McDonnell F-101A Voodoo. . . . 47
31 Lockheed F-104G Starfighter. . . . 48
32 McDonnell Douglas F-4E Phantom II. . . . 49

33 Northrop F-5E Tiger II. 50
34 Convair F-106 Delta Dart. 51
35 McDonnell Douglas F-15A Eagle. 52
36 General Dynamics F-16A Fighting Falcon. 53
37 McDonnell Douglas F/A-18C Hornet. 54
38 Lockheed Martin/Boeing F-22A Raptor. 55
39 Boeing KC-135R Stratotanker. 56

List of Tables

1 Representative aircraft selected across different generations. 11
2 Trimmed AOA of all investigated types. 16
3 Trimmed AOA of all baseline types, flying solo (α_{trim}^{solo}) and in a two-aircraft formation (α_{trim}^{lead} and α_{trim}^{trail}), in the reference configuration. The induced drag coefficient $C_{D,i}$ is reported in counts. 26
4 Trimmed AOA of all optimized types, flying solo (α_{trim}^{solo}) and in a two-aircraft formation (α_{trim}^{lead} and α_{trim}^{trail}), in the reference configuration. The induced drag coefficient $C_{D,i}$ is reported in counts. 26
5 Percentage variation of induced drag coefficients $\Delta C_{D,i}$ for optimized configurations, referenced to the solo-flight values of the corresponding baseline geometries. 26
6 Latin Hypercube Sampling (LHS) grid showing the selected 3×3 combinations of design variables. 27
7 Optimum position for the maximum induced drag reduction of the trailer aircraft with the original (X^{OG}) and the optimized (X^{OPT}) wing geometry. 29
8 Trimmed AOA of all original types, flying solo (α_{trim}^{solo}) and in a two-aircraft formation (α_{trim}^{lead} and α_{trim}^{trail}), in the optimum position. The percentage variation in $C_{D,i}$ is also reported for both the leader and the trailing plane with respect to the individual aircraft with original wing geometry. 29
9 Trimmed AOA of all optimized types, flying solo (α_{trim}^{solo}) and in a two-aircraft formation (α_{trim}^{lead} and α_{trim}^{trail}), in the optimum position. The percentage variation in $C_{D,i}$ is also reported for both the leader and the trailing plane with respect to the individual aircraft with original wing geometry. 29
10 Trimmed angle of attack for the fighters tested: solo flight (α_{trim}^{solo}), refuelling in reference configuration ($\alpha_{trim}^{og,fuel}$), and refuelling in optimized configuration ($\alpha_{trim}^{opt,fuel}$). Percentage variations in induced drag coefficient $\Delta C_{D,i}$ are reported relative to the original solo-flight configuration. 38

1. Introduction

1.1. Background and state of the art

Fighter aircraft consume large amounts of fuel during high-speed cruise, making drag reduction a critical concern for mission range and efficiency. As an example, the F-16 Fighting Falcon, one of the most widely deployed fighter jets worldwide, burns approximately 3950 liters of fuel per hour during typical subsonic flight [2]. By comparison, the more advanced F-22 Raptor, a fifth-generation fighter, consumes around 6000 liters per hour under similar conditions [2]. The higher fuel consumption arises from the F-22 design philosophy. Stealth measures (edge-aligned inlets/nozzles, 2-D nozzles, shaped ducts, engine obscuration) complicate the propulsion system aerodynamic integration, while the added weight and complexity of thrust-vectoring hardware increase the thrust required for trim, raising overall fuel demand [3]. Although actual fuel flow is strongly dependent on throttle setting, altitude, Mach number, and propulsion-specific fuel consumption, the reduction in aerodynamic drag in cruise directly translates into lower required engine thrust and reduced fuel consumption for a given mission profile.

A well-established aerodynamic mechanism to reduce induced drag is formation flight: in the early days of jet engines, docking an escort jet fighter to each wingtip of a larger bomber aircraft, especially a propeller-driven one, was seen as an effective way to increase the range of the former. This notion of a three-ship formation or composite aircraft was notably explored during USAF project Tom Tom beginning in 1955 [4].
In addition, this phenomenon is widely observed in nature, where migratory birds commonly adopt V- or J-shaped formations. An impressive 70% range extension for a flock of 25 birds was studied by Lissaman and Schollenberger in 1970 [5].

Formation flight offers a way to exploit aerodynamic interactions between aircraft: a trailing aircraft flying in the upwash of the leader's wingtip vortex can effectively reduce the lift-induced component of its own drag. By flying with a lateral offset within the leader's wake, the follower encounters an upward-directed flow that lowers its required angle of attack for the same lift, thus reducing the thrust needed to maintain level cruise. The formation benefits are most pronounced in the subsonic cruise regime, where induced drag constitutes a significant fraction of total aerodynamic drag for lifting configurations. For transonic or supersonic regimes, wave drag and viscous/pressure-related components increasingly dominate, reducing the relative importance of induced-drag interactions.

The aerodynamic benefits of formation flight have been demonstrated in many studies. Theoretical analyses of simple wing pairs predict large induced-drag reductions: for example, two elliptically-loaded wings in close formation can achieve up to 44% induced drag savings, and much greater when more aircraft form a V-shaped echelon [6]. Flight test experiments have confirmed substantial gains. In NASA's Autonomous Formation Flight (AFF) project, a highly instrumented F/A-18A flying as the trail aircraft was positioned within the wingtip vortex of a leading F/A-18 [1]. The experiment measured direct performance benefits: depending on the relative spacing, the trailing jet experienced up to about 50% reduction in induced drag, 20% reduction in total aerodynamic drag, and roughly an 18% reduction in total fuel flow when in the optimal separation position.
Previous research on formation flight has mostly focused on specific platforms or general aerodynamics, but the detailed dependence on wing geometry across different fighter aircraft remains less explored. Fighter jets typically have low-aspect-ratio, swept wings optimized for high-speed maneuverability, which affects how their wakes interact. For instance, stubby wings generate stronger tip vortices, while highly swept wings shift the downwash pattern. Due to these differences, not all fighter designs will benefit equally from formation flying. Yet in tactical transit or patrol missions where fuel economy and loiter time matter, identifying which fighter design can save the most fuel in formation has practical value.

Of course, the design implications of formation flight have long been pondered and a surprising diversion of the "three-ship formation" concept did find its way on the drawing boards of Boeing, North

American and the Convair Division of General Dynamics during the Weapon System 110A contest, which resulted in the North American XB-70 Valkyrie [7]. The initial proposals of all three companies ended up being an aircraft with low-aspect-ratio supersonic wings to which sizable tip extensions (echoing escort fighters) were hinged, each carrying a fuel tank. These expendable tip tanks were supposed to separate before the supersonic dash. Boeing is known to have studied similar wingtip extensions for the (subsonic) B-47 and (X)B-52 bombers, under the designations Model 450-36-10 and 464-79-0, respectively.
In light of what design preceded it (i.e. NA-239), it is sadly ironic that the XB-70 A/V-2 would then be lost in the infamous mid-air collision during a photoshoot involving close formation flight of multiple aircraft.
These authors are not aware of other designs that were shaped with formation flight, or any idea stemming from it, in mind.



Figure 1: The formation flight of the North American XB-70A Valkyrie (Air Vehicle No. 2) on 8 June 1966, moments before the fatal midair collision. Flying in close echelon are a Northrop T-38A Talon, a McDonnell F-4B Phantom II, a Lockheed F-104N Starfighter (piloted by NASA's chief research pilot Joseph A. Walker) and a Northrop YF-5A Freedom Fighter. Within seconds, Walker's F-104 would be drawn into the Valkyrie's wingtip vortex, precipitating the catastrophic accident that claimed his life and, soon after, that of XB-70 co-pilot Maj. Carl Cross.

1.2. Aim of this research

To address the existing knowledge gap, the present study systematically compares a representative set of jet fighter geometries spanning multiple generations and design philosophies to determine their relative potential for induced-drag reduction in formation.
Potential-flow solutions are initially used to compare the implications of a two-aircraft formation on some relevant aerodynamic performance metrics of the baseline configurations.
For selected configurations, a geometry optimization is performed to identify planform and relative offset combinations that maximize cruise drag reduction subject to practical geometric and performance

constraints.
The implications of a geometry optimized for formation flight are finally examined in a scenario with the jet fighter trailing behind a large tanker aircraft, in an attempt to establish whether the changes could be beneficial during aerial refueling as well.

Any consequence that formation flight and geometry optimization might have on the lateral and directional characteristics of the aircraft involved, including control surface deflections, is beyond the scope of this study and of the level of accuracy of the low-fidelity geometry models: the focus is solely on the interaction between the formation leader and the trailer in terms of induced velocity components and the associated variations in aerodynamic coefficients. As a first approximation, no attention was devoted to how longitudinal stability and trim would be affected.



Figure 2: The Italian Air Force aerobatic team *Pattuglia Acrobatica Nazionale (PAN) Frecce Tricolori* performing in a V formation while releasing the green, white, and red smoke of the Italian flag. This iconic display illustrates the aerodynamic principle of formation flight.

2. Problem formulation

2.1. The classical lifting-line theory

The earliest theoretical framework for modeling the aerodynamics of finite wings was formulated by Prandtl in the 1910s. Known as the lifting-line theory, this model was the first attempt to extend the two-dimensional aerodynamic concepts of airfoils to wings of finite span, and it marked a decisive step in understanding how spanwise flow and vortex shedding modify the lift and drag characteristics of real aircraft. In Prandtl's formulation, the finite wing is replaced by a bound vortex of circulation Γ positioned along the quarter-chord line. The Kutta–Joukowski theorem provides the connection between circulation and lift, yielding the expression $L = -\rho V_\infty \Gamma$. Because circulation cannot terminate abruptly at the wingtips, the bound vortex continues downstream as two trailing vortices, completing the well-known horseshoe vortex system (Figure 3).

These trailing vortices induce a downward velocity field along the span of the wing, thereby reducing the local effective angle of attack; the consequence is the appearance of induced drag, a penalty that is absent in the two-dimensional case.

Although the lifting-line theory provided remarkable insight with relatively simple mathematics, its assumptions imposed clear limits on its range of validity. The model is most accurate for straight, untapered wings with high aspect ratios, where spanwise variations are relatively mild. For wings of low aspect ratio, or those with significant taper, sweep, or delta-like geometry, the simplifications inherent in the lifting-line concept become inadequate. These planforms are precisely those encountered in modern aviation, from combat aircraft to transport configurations, motivating the development of more general models capable of representing the three-dimensional aerodynamic field in greater detail.

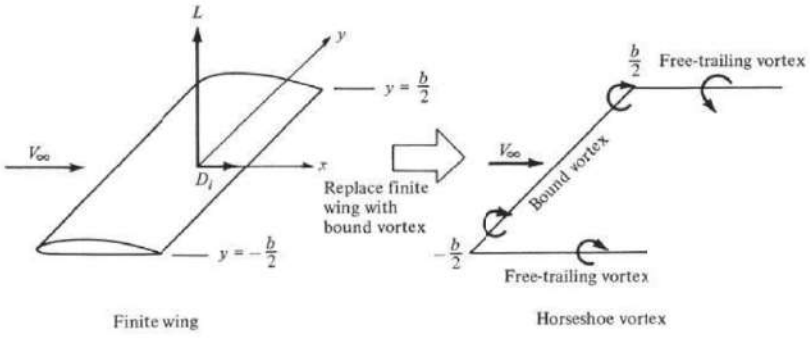


Figure 3: Prandtl's lifting-line model: a finite wing replaced by a bound vortex with trailing vortices.

2.2. The lifting-surface theory and the vortex lattice method

The natural progression beyond the lifting-line concept was the introduction of lifting-surface theory. Instead of reducing the entire wing to a single line of circulation, this formulation represents the wing as a surface over which vorticity is unevenly distributed. The essential idea is that the flow field can be approximated more accurately if the wing is divided into multiple elements, each with its own vortex system, rather than relying on a single bound vortex with trailing legs.

In practice, this theory is most commonly implemented through the *vortex lattice method* (VLM). The wing surface is discretized into a lattice of panels, and each panel is assigned a horseshoe vortex whose bound leg lies at the quarter-chord line and whose trailing legs extend downstream indefinitely. By enforcing the no-penetration condition at designated control points on each panel, a system of linear equations is assembled, the solution of which yields the distribution of circulation across the lifting surface.

Compared to lifting-line theory, the vortex lattice method provides a much richer description of the aerodynamics, since it captures both the spanwise and chordwise variation of circulation. It can be applied to straight, swept, tapered, or even low-aspect-ratio wings, and it remains computationally inexpensive when compared to higher-fidelity computational fluid dynamics. This balance between

physical fidelity and efficiency explains why VLM continues to play a prominent role in preliminary aircraft design and performance analysis.
Modern implementations, such as AVL, extend the classical VLM framework in several directions. They introduce corrections for compressibility effects at subsonic Mach numbers through the Prandtl–Glauert transformation, which allows the incompressible formulation to remain valid with appropriate scaling. They also provide functionality for trim analysis, control surface effectiveness evaluation, and linearized dynamic stability studies. In this way, AVL and similar tools represent not just a direct application of lifting-surface theory, but an integrated environment for analyzing aircraft aerodynamics in a wide range of configurations with reasonable accuracy and minimal computational cost.

2.3. Downwash in formation flight

A central feature of both lifting-line and lifting-surface formulations is their prediction of the downwash field generated by finite wings. The trailing vortex system shed by the wing extends far downstream, shaping the flow environment in which following aircraft operate. The induced velocity field is not uniform: regions of strong downwash occur in the vicinity of the wingtip vortices, while adjacent areas experience upwash. This spatial variation of the flow has deep implications for formation flight, where multiple aircraft fly in close proximity to each other. When a trailing aircraft positions itself in a favorable location within the wake of a leader, typically just outside and slightly behind the wingtip, it enters an upwash region. In this region, the effective angle of attack of the follower's wing increases without the need to generate additional circulation. As a result, the follower achieves the required lift at a lower induced drag penalty, leading to a measurable gain in aerodynamic efficiency.
Understanding and predicting the structure of the downwash field is therefore essential for quantifying the benefits of formation flight. The vortex lattice method provides a convenient framework for this task, since it resolves the distribution of trailing vorticity and thus the induced velocity field with sufficient accuracy to capture the essential physics of wake interactions. By analyzing the mutual influence of aircraft within a formation, it becomes possible to estimate the induced drag savings.
In the context of this work, the combination of lifting-surface theory and the vortex lattice method forms the foundation upon which the subsequent aerodynamic analysis is built. These models provide both the theoretical background and the computational tools required to describe the aerodynamic interactions between wings, making them ideally suited for investigating the downwash effects and potential performance improvements associated with formation flight.



(a) Two F-15 Eagles from the Massachusetts Air National Guard's 102nd Fighter Wing conducting a combat air patrol over New York City in support of Operation Noble Eagle.



(b) Smoke generators show the twisting paths of wingtip vortices behind two NASA Dryden F/A-18's used in the Autonomous Formation Flight (AFF) program.

Figure 4: Illustrations of formation flight: (a) operational use in combat air patrol and (b) experimental investigation of wingtip vortices and wake interactions.

3. Modeling and simulation workflow

3.1. Geometry modeling

The aircraft geometry was modeled using publicly available three-view drawings as the primary reference, along with some basic dimensional data, such as wingspan, overall length, and wing surface area of the aircraft.
Geometric datums were then extracted by importing the reference imagery in Microsoft Excel®, where it was carefully rescaled against known dimensions to ensure geometric fidelity. Once properly scaled, reference lines were superimposed on significant elements of the layout (e.g., a wing chord), and their length could be read in the shape menu featured by Excel® itself.
The surface definitions were finally encoded within an `.avl` input file, which served as the basis for the subsequent aerodynamic analysis in AVL, described next. The process was made easier by the program AERO_Console [8], a user-friendly interface for AVL (version 3.37) developed by Dr. A. Rahimi (University of Windsor).

3.2. Aerodynamic analysis

Aerodynamic calculations were carried out using AVL, a software tool specifically designed to model lifting-surface aerodynamics and to evaluate stability and control characteristics. In addition to its aerodynamic solver, AVL also provides functionalities for linearized dynamic stability analysis, trim calculations, and the assessment of control surface effectiveness. AVL version 3.37 was chosen for consistency with the use of AERO_Console, see above.

AVL is based on the vortex lattice method (VLM), which is most accurate when applied to configurations composed primarily of thin lifting surfaces operating at relatively small angles of attack and sideslip. Within this linearized framework, each lifting surface and its corresponding trailing wake are represented as single-layer vortex sheets, discretized into a lattice of horseshoe vortex filaments. The trailing legs of these filaments are assumed to extend downstream parallel to the body x-axis, simulating the wake advection in an inviscid, irrotational flow field.
To account for compressibility effects at subsonic speeds, AVL implements the Prandtl–Glauert (PG) transformation. This classical correction transforms the compressible potential-flow equation into a form equivalent to the Laplace equation, thereby allowing the incompressible vortex lattice formulation to be applied with appropriate scaling.

3.3. Optimization and post-processing

In addition to AVL, MATLAB® was used as the primary programming environment for implementing the optimization routines and managing the aerodynamic data. The Global Optimization Toolbox proved to be a fundamental add-on for the purposes of optimization tasks, and its features have been extensively used throughout the code.
Custom functions and scripts were developed in MATLAB® to perform the wing planform optimization. These routines automated the modification of geometric parameters and iteratively evaluated candidate configurations using AVL output. By structuring the optimization process within MATLAB®, it was possible to systematically explore the design space and identify planform geometries that best satisfied the desired aerodynamic and performance objectives.
Furthermore, MATLAB® was used as a central repository to store and organize aerodynamic coefficients obtained from AVL simulations. These data were subsequently used for post-processing, including visualization through plots, contour maps, and comparative performance charts. This workflow enabled both quantitative assessment of the drag-reduction potential and qualitative interpretation of the flow mechanisms driving the observed changes.

4. Generation and comparison of aircraft geometries

4.1. The investigated aircraft types

The first stage of the study consisted of the reconstruction of representative aircraft geometries. As outlined in Section 3.1, this process relied extensively on three-view drawings and technical reference tables, which were digitized and parameterized to reproduce the main planform and tail configurations of the chosen designs. The objective was not to capture every detail of the outer mold line, but rather to create aerodynamic models that faithfully reflect the most relevant geometric features that determine lift, drag, and stability.

Military aviation has undergone profound changes since the Second World War, driven by innovations in materials, propulsion systems, and aerodynamic concepts. These developments gave rise to different design philosophies, including delta wings, variable-sweep wings, and unconventional empennage configurations such as V-tails and T-tails. In order to capture this diversity, a set of eleven representative fighter aircraft was selected, covering a range of designs from first to fifth generation. In addition, the Aermacchi MB-339A jet trainer was included for its relevance to formation flight.

An initial attempt was made to include Soviet aircraft such as the Mikoyan-Gurevich (MiG) and Sukhoi (Su) families. However, reliable data concerning their wing sections and planform geometries are scarce and, when available, often incomplete or inconsistent. For this reason, the final selection was restricted to aircraft in service with NATO countries, for which both geometric data and airfoil information could be obtained with reasonable confidence. By focusing on aircraft with known and documented aerodynamic profiles, the generated geometries ensure both consistency in modeling and relevance for subsequent analysis.

The selected aircraft are highlighted in Table 1, where they are divided and grouped according to the generation to which they belong.

Generation	Representative Aircraft
Trainer	Aermacchi MB-339A
1	Fiat G.91R
2	North American F-100 Super Sabre
2	McDonnell F-101A Voodoo
2	Lockheed F-104G Starfighter
3	McDonnell Douglas F-4E Phantom II
3	Northrop F-5E Tiger II
3	Convair F-106 Delta Dart
4	McDonnell Douglas F-15A Eagle
4	General Dynamics F-16A Fighting Falcon
4	McDonnell Douglas F/A-18C Hornet
5	Lockheed Martin/Boeing F-22A Raptor

Table 1: Representative aircraft selected across different generations.

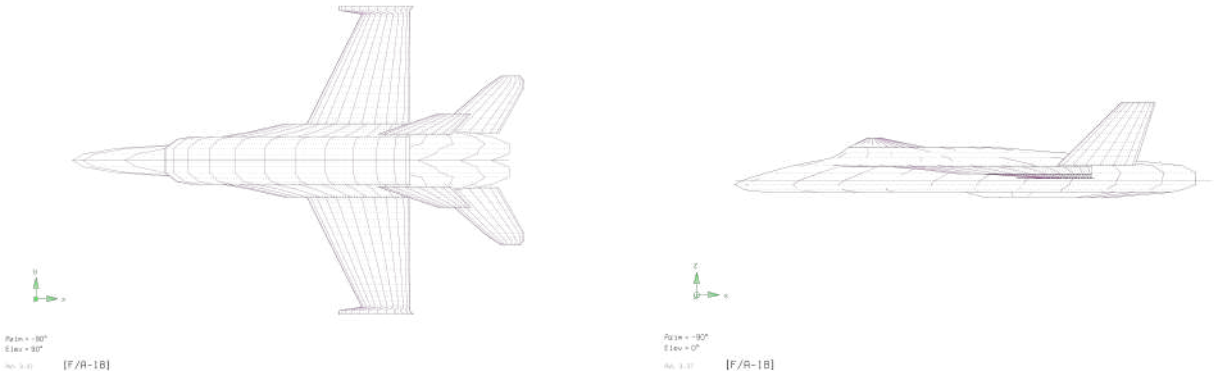
Table 1 does not include additional aircraft whose models were designed for use in AVL within the framework of project FOSSER, as they do not fall under the definition either of a fighter or a trainer. These are the Boeing KC-135R Stratotanker refueling tanker aircraft, and the North American XB-70 Valkyrie (A/V-2) strategic bomber.

4.2. Reconstructing the geometries in an AVL-compatible format

During the construction of the aerodynamic models, particular care was devoted to the representation of the fuselage and its interaction with the lifting surfaces. As the slender-body option available in

AVL was intentionally excluded[1], the fuselage geometry was reconstructed in a simplified manner by intersecting its projections in the longitudinal and vertical planes, and giving it the appearance of a cruciform when viewed from the front. This approach allowed the model to preserve the outline of both the planform and side views of the original aircraft without introducing unnecessary complexity into the vortex-lattice framework.
A critical aspect of this process was the use of the `COMPONENT` option available in AVL. This feature enables multiple input `SURFACE` definitions to be grouped into a single composite virtual surface by assigning them the same `Lcomp` identifier. When surfaces share the same `Lcomp` value, AVL suppresses the finite-core influence of horseshoe vortices and their corresponding control points within that component. Without this precaution, spurious self-interactions could arise, severely compromising the accuracy of the aerodynamic predictions. By systematically grouping surfaces that belong to the same projection plane, the models achieved a robust balance between geometric fidelity and numerical stability. In addition, this prevents the circulation of a wing or tailplane from unrealistically becoming null at the root section, that is, where its `SURFACE` counterpart ends in correspondence of the fuselage.

The resulting set of geometries therefore reflects the essential aerodynamic characteristics of each aircraft while remaining suitable for analysis with the vortex lattice method.
In Figure 5, the geometry of a fighter aircraft selected during the initial screening is shown as a reference. A view of each configuration is reported in Appendix A for the sake of completeness. The images shown here correspond to the visualization output of AVL accessible via the `OPER > G` menu. These graphical representations are not only useful as a means of documenting the modeling process, but also serve as a first qualitative check of the fidelity of the reconstructed geometries with respect to the reference three-view drawings.



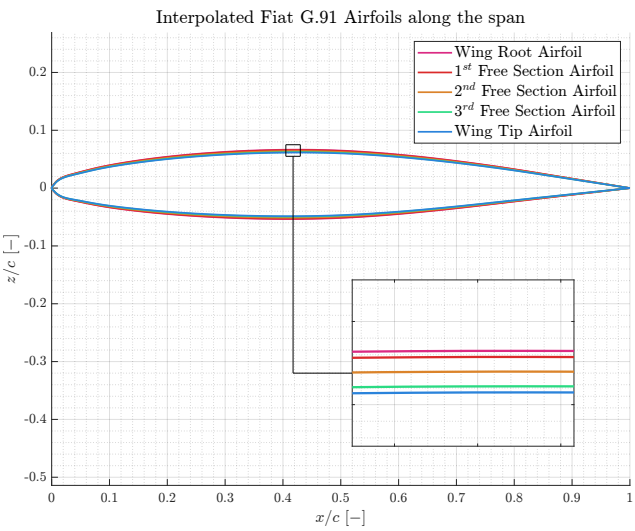
(a) View from above. (b) View from the port side.

Figure 5: AVL-rendered geometry of the McDonnell Douglas F/A-18C fighter.

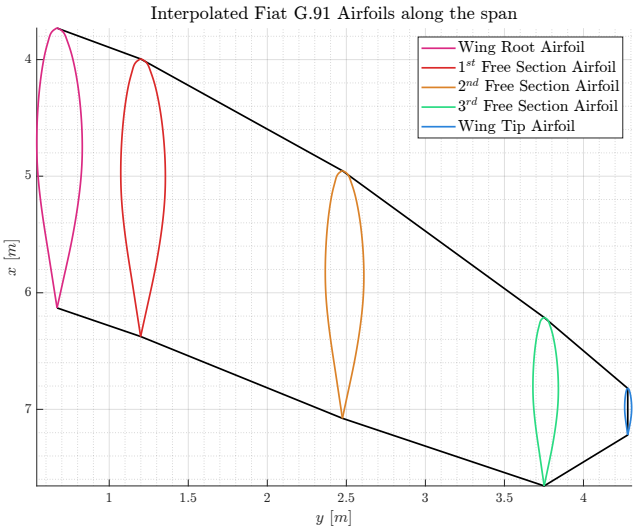
Figure 5 also shows how the spanwise spacing between the legs of the horseshoe vortices was decreased in the presence of kinks or other discontinuities in the chord length (including the portion near the tip), that is, where strong gradients in the spanwise load distribution might occur; an appropriate specification of cosine (`Sspace = ±1`) and sine (`Sspace = ±2`) distributions of vortex lattice panels for each `SURFACE` made this possible.
A similar solution was adopted for the chordwise discretization which was set to a cosine-type distribution (`Cspace = 1`) for all `SURFACE`s, except for the planform and lateral views of the fuselage for which a uniform distribution (`Cspace = 0`) was selected in an attempt to create less stretched and/or skewed elements.

Two AVL keywords were used to further improve the fidelity of the solution:

[1] Although this approximation may be adequate for simple layouts, it is known to yield limited fidelity when applied to more complex configurations.



(a) Fiat G.91R airfoil sections overlaid in normalized coordinates.



(b) Fiat G.91R airfoil sections positioned along the wingspan showing physical coordinates.

Figure 6: Airfoils used during the particle-swarm optimization of the Fiat G.91. Airfoil sections are linearly interpolated along the wingspan between the root and the tip sections.

- `AFILE`: imports an external file containing the airfoil point distribution for a given section. AVL computes the mean line from this file, improving the accuracy of the tangency condition.
- `CLAF`: adjusts the lift-curve slope from thin airfoil theory to account for the effects of thickness. According to the AVL user manual [9], a good approximation for $C_{L\alpha}$ from 2D potential flow theory is:

$$\frac{C_{L\alpha}}{2\pi} = 1 + 0.77\left(\frac{t}{c}\right) \tag{1}$$

  where (t/c) is the thickness-to-chord ratio.

Identification of wing sections was supported by the online resource The Incomplete Guide to Airfoil Usage [10], which provides a consolidated database of airfoil applications across a broad spectrum of aircraft. The airfoil coordinates in the standard Selig's format, instead, were either taken from OpenVSP [11] by exploiting the `File > Export... > Airfoil Points (.dat)` functionality or downloaded from the Airfoil Tools database [12].

When needed, intermediate section airfoils were obtained through linear interpolation between root and tip profiles. Figure 6 shows the interpolated profiles adopted for the optimization of the Fiat G.91R. To generate smooth airfoil distributions suitable for AVL, XFOIL was employed. The `.dat` airfoil files were re-paned with 300 points to ensure that AVL constructed an accurate mean camber line.

One major approximation made during the making of all the models involves wing incidence and geometric twist. These two parameters are seldom reported in aircraft specifications tables and datasheets, and obtaining them from available imagery and drawings is not a trivial task. The authors eventually settled on setting them to zero in the belief that their value should be based on considerations mostly about tailoring the stall behavior, and as such it is acceptable for them not to be considered optimization variables.

4.3. Some notes about the coordinate system for formation flight

The relative position of the trailing aircraft with respect to the leader was described using a Cartesian coordinate system consistent with that employed in NASA's AFF project. The position is expressed in non-dimensional form by normalization with respect to the wingspan of the reference aircraft, denoted as b. This approach allows for the comparison of results across configurations and widely different scales.

The longitudinal separation X is defined such that $X = 0$ corresponds to a nose-to-nose alignment of the two aircraft; positive values of X indicate that the trailing aircraft is positioned aft of the leader. The lateral displacement Y is defined so that $Y = 0$ corresponds to a tip-to-tip alignment of the two aircraft; increasingly negative values of Y indicate wingtip overlap, while positive values correspond to an outward displacement. Vertical displacement Z is defined relative to the plane of the leader's wing. When the aircraft are coplanar, $Z = 0$, while negative values indicate that the trailing aircraft is below the leader and positive values indicate a position above.
A schematic representation of the adopted coordinate system is reported in Figure 7 where the two F/A-18s are shown in what will be called *reference configuration* from now on.

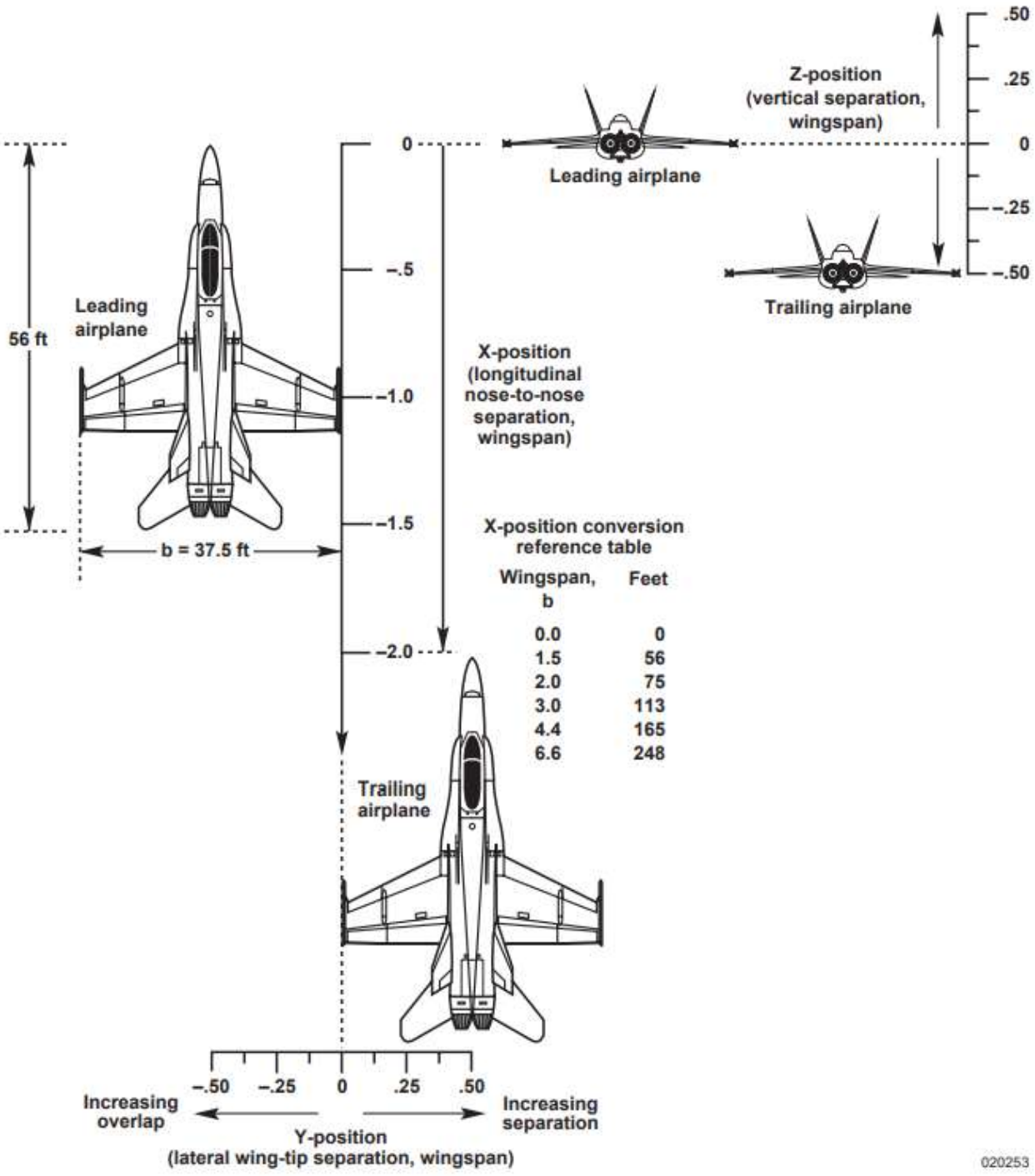


Figure 7: Coordinate system used in the Autonomous Formation Flight Project [1] to specify the position of the trailing aircraft with respect to the formation leader.

4.4. Reference trimmed-cruise conditions

To ensure a consistent basis for comparison among different aircraft and formation configurations, a common flight condition was defined. Specifically, the condition identified in the AFF documentation as *Condition 1* was adopted, corresponding to a Mach number $M = 0.56$ at a flight altitude of 25000 ft. This subsonic flight regime, of interest for formation flight, was selected as the AFF design point to match vortex-prediction data previously produced by The Boeing Company [1]. It also provides a suitable compromise between aerodynamic similarity across aircraft of different generations and the numerical applicability of the vortex-lattice method employed in this study. Moreover, by adopting a condition that has been experimentally validated, the results obtained can be directly compared with reference data available in the literature.
With regard to the relative positioning of the vehicles, the reference configuration mentioned in the previous Section 4.3 was chosen as a benchmark for formation flight in this initial phase of project FOSSER, in which the formation advantages of each aircraft model were analyzed in view of wing optimization.

For each aircraft, the trim condition was established by adjusting the angle of attack to achieve steady, level flight at the specified Mach number and altitude. Publicly referenced figures for gross mass, typically sourced from Wikipedia, were used to determine a lift coefficient consistent with the reference conditions. The AOA of an aircraft flying solo was trivially obtained by setting up a simulation in AVL with C_L constrained to the value computed as described above.
This method cannot be applied in the case of formation flight, since AVL does not natively support the computation of loads of independent subsets of surfaces. Therefore, a particle swarm optimization, an algorithm whose comprehensive discussion is deferred to Section 5.2, was set up to identify the (different) AOAs at which both vehicles in a two-aircraft formation achieved the expected C_L for the trim condition to be satisfied (C_L^{target}): the two distinct AOAs make up the optimization variables and they are individually set within the `.avl` file by leveraging the `ANGLE` field within a `SURFACE` block; accordingly, the simulation in AVL is executed at a null AOA because the geometry file already contains all the information about the actual AOA of each aircraft. The fitness function J of this optimization process is estimated as follows:

$$J = \left| C_L^{lead} - C_L^{target} \right| + \left| C_L^{trail} - C_L^{target} \right| \tag{2}$$

Custom MATLAB® functions take care of replacing the `ANGLE`s in the geometry definition by writing new `.avl` files at each iteration.
Figure 8 shows the evolution of the particle swarm as it progressively converges toward the minimum of the fitness function J. Each panel corresponds to a representative iteration, illustrating how the particles gradually cluster around the optimal solution.
Table 2 shows the results of trimming the two aircraft in a formation, in terms of AOA, compared to an aircraft of the same type flying solo.

The aerodynamic coefficients thus calculated serve as baseline values against which variations due to formation positioning are subsequently evaluated.

4.5. Downselection of aircraft types to be analyzed

A total of three fighters were selected for optimization on the basis of the results reported in Table 2, from which the advantages of formation flight should be clear: with few exceptions, the trailing aircraft greatly benefits from cruising inside the upwash of the leader's wake and often the leader itself shows a mild improvement in aerodynamic performance. The only exceptions consists of the F-100, F-104 and F-106 although these aircraft, along with the F-5 that exhibits a major decrease in $C_{D,i}$ despite an increase in AOA required for trim, might have suffered from numerical issues; in fact, their results look rather abnormal compared to the general trend.

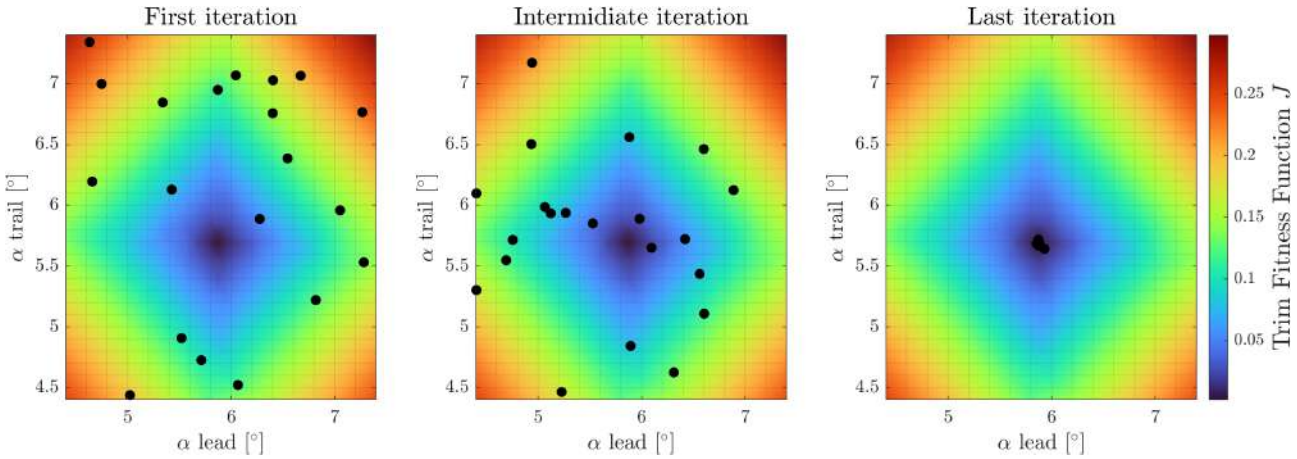


Figure 8: Evolution of the particle swarm during the optimization of the trim angle of attack for the F/A-18 formation. From left to right: the initial iteration, an intermediate iteration halfway through the optimization, and the final iteration. The black dots represent the particles evaluated at the corresponding iteration.

Aircraft type	α_{trim}^{solo}	α_{trim}^{lead}	α_{trim}^{trail}	$\Delta C_{D,i}^{lead}$	$\Delta C_{D,i}^{trail}$
Aermacchi MB-339A	2.3°	2.3°	2.2°	-0.5%	-5.4%
Fiat G.91R	4.1°	4.0°	3.9°	-8.3%	-34.4%
North American F-100 Super Sabre	6.7°	9.0°	8.8°	+11.5%	+7.3%
McDonnell F-101A Voodoo	9.4°	9.1°	8.9°	-1.6%	-8.1%
Lockheed F-104G Starfighter	10.0°	9.9°	9.7°	+32.9%	+27.7%
McDonnell Douglas F-4E Phantom II	8.1°	7.9°	7.7°	-1.6%	-6.5%
Northrop F-5E Tiger II	7.4°	7.6°	7.5°	-24.3%	-27.5%
Convair F-106 Delta Dart	6.9°	6.9°	6.8°	+17.3%	+13.7%
McDonnell Douglas F-15A Eagle	6.7°	6.6°	6.5°	-1.3%	-7.0%
Lockheed Martin F-16A Fighting Falcon	6.3°	6.2°	6.0°	-1.5%	-5.3%
McDonnell Douglas F/A-18C Hornet	5.9°	5.9°	5.7°	-6.3%	-13.0%
Lockheed Martin/Boeing F-22A Raptor	9.1°	8.7°	8.5°	+2.5%	-2.8%

Table 2: Trimmed AOA of all investigated types, flying solo (α_{trim}^{solo}) and in a two-aircraft formation (α_{trim}^{lead} and α_{trim}^{trail}), in the reference configuration. The percentage variation in $C_{D,i}$ is also reported for both the leader and the trailing plane with respect to the individual aircraft.

The Fiat G.91R and Lockheed Martin/Boeing F-22A Raptor have been selected for further studies and optimization in the spirit of comparing old-generation fighters to more advanced ones. Coincidentally, the G.91 appears to be the aircraft that benefits the most from flying formation missions among those featured in the list, while the F-22 is the one showing the smallest improvements, except for the aforementioned extreme circumstances.
The McDonnell Douglas F/A-18C Hornet was, in addition, deemed worthy of mention since airplanes belonging to the same family and really similar in overall appearance, i.e. two F/A-18As, were flown during the NASA AFF reference project. Therefore, it was covered to make a comparison with actual flight test data possible.

5. Wing optimization

As anticipated, the optimization process focused exclusively on the geometry of the wing planform. The guiding principle behind the development of the optimization code was that once the new optimized geometry was obtained, it could be manufactured and retrofitted to the existing aircraft.
For this reason, the section identified as *Section 1* of the numerical model (i.e., the root) was constrained to maintain both the leading-edge position and the chord length fixed. This was imposed as a geometric constraint to ensure structural and integration compatibility with the fuselage.
Another strong constraint imposed was to keep the overall wingspan constant. This choice was motivated by aerodynamic considerations: since the lift-induced drag coefficient $C_{D,i}$ depends inversely on the aspect ratio AR, i.e.:

$$C_{D,i} = \frac{C_L^2}{\pi e\, AR} = \frac{C_L^2}{\pi e\, b^2/S} \tag{3}$$

an increase in span b (while keeping wing surface area and trim conditions constant) would naturally reduce induced drag.
By fixing the span, the optimization had to seek improvements primarily from changes in wing sweep, taper ratio, and wing dihedral, rather than trivially extending the span.

A fully configurable MATLAB® code was developed for this purpose. The code allows the user to:

- define the number of sections into which the wing surface (as defined in the AVL input file) is subdivided;
- select the spacing distribution of the sections (e.g., uniform, cosine, half cosine);
- impose bounds on the sweep angle (both positive and negative) and on the dihedral angle;
- specify allowable percentage variations in chord length;
- apply user-defined penalty coefficients;
- select the objective function to minimize;
- choose the optimization algorithm (e.g., *particle swarm optimization*, *single-objective genetic algorithm*, *multi-objective genetic algorithm*, all described in the following).

The free design variables chosen for each section were:

1. leading-edge position along the X-axis, which is basically a degree of freedom allowing for control on the (local) sweep angle;
2. leading-edge position along the Z-axis, making it possible to vary dihedral;
3. chord length.

Since the wingspan is fixed, the Y-position of each section remains constant.
For a wing discretized into four free sections, the total number of optimization variables is:

$$N_{var} = 4 \times 3 = 12 \tag{4}$$

Wing twist was deliberately excluded from the optimization. Although twist could provide additional aerodynamic benefits (e.g., influencing stall progression near the tip), the lack of detailed data on the original aircraft motivated the decision to fix twist at zero.
During execution, the optimization code calls additional MATLAB® routines developed by the authors. These routines automatically modify the AVL input file to reflect the new wing geometry and generate a valid `.avl` file for each candidate configuration. The optimization code as well as the routines used to modify the geometry and to generate AVL files are presented in Appendix B.

5.1. Fitness function and penalty terms

5.1.1 Fitness function

Regardless of the optimization algorithm chosen, all candidate wing geometries were evaluated using the same fitness function. The evaluation process is as follows:

1. The candidate wing geometry is generated.

2. If formation optimization is enabled, the multi-aircraft geometry is created.
3. AVL is executed to compute the aerodynamic coefficients.
4. Penalty functions are applied if the geometry violates feasibility constraints.

The objective function was defined to maximize the lift-to-induced-drag ratio:

$$\eta = \frac{C_L}{C_{D,i}} \tag{5}$$

where both the lift and induced drag coefficients are obtained from AVL under known flight conditions. The objective function was evaluated in relation to the trailing aircraft only, unless specified, the reason being the greater benefits to be expected compared to those of the leader; in fact, the advantages of formation are shown to be linked mostly to the possibility for an aircraft to fly inside the upwash of another one ahead of it.

5.1.2 Penalty functions

To prevent structurally or aerodynamically unrealistic geometries from being selected, penalty terms were added to the fitness function. The following penalties were introduced:

- many moderate or excessive high-order curvatures in the X and Z directions, or chord length distribution;
- wing area differing from the baseline by more than a specified tolerance;
- excessive differences in chord length between adjacent sections;
- abrupt variations in sweep or dihedral angles;
- geometries with three or more sweep/dihedral changes;
- shapes that significantly deviate from the baseline geometry, which is considered a good starting point;
- reverse-tapered geometries.

The penalized fitness function takes the form:

$$J = -\eta + \sum_i \lambda_i P_i \tag{6}$$

where J is the cost to be minimized, P_i are penalty functions, and λ_i are penalty weights.
This formulation ensures that the optimizer strives towards wings with higher efficiency while maintaining structural and aerodynamic viability.

5.2. Optimization methods

To evaluate the penalized fitness function, two different population-based optimization methods (or *evolutionary algorithms*) available in MATLAB® were initially tested: particle swarm optimization (PSO) and genetic algorithm (GA). Both belong to the class of metaheuristic, stochastic search methods and are widely applied in aeronautical design due to their ability to handle nonlinear and multi-modal cost functions without requiring gradient information. A brief description of both PSO and GA is reported below.

5.2.1 Particle swarm optimization (PSO)

PSO models the collective behavior of a swarm of particles that explore the search space by updating their velocities and positions based on both individual and group experience. The characteristic behavior is the discrete improvement of the global best when a particle discovers a better location, while the swarm mean follows more smoothly toward the optimum as particles are attracted to promising regions.
For the i-th particle at iteration k, the velocity and position updates are defined as:

$$\begin{cases} v_i^{k+1} = W\, v_i^k + y_1 u_1 (p_i - x_i^k) + y_2 u_2 (g - x_i^k) & \text{(7a)} \\ x_i^{k+1} = x_i^k + v_i^{k+1} & \text{(7b)} \end{cases}$$

where W represents the inertia of a particle, y_1 and y_2 represent cognitive/social coefficients, and u_1 and u_2 represent random vectors uniformly distributed between 0 and 1 with a length equal to the number of variables. This update uses a weighted sum of: the previous velocity v^k, the difference between the current position and the best position the particle has seen $(p_i - x_i^k)$, and the difference between the current position and the best position in the current neighborhood $(g - x_i^k)$.
To formalize this behavior of attracting particles toward the optimal value, we denote by $J_i^{(k)}$ the penalized cost for particle i at iteration k. We define the swarm average and the global best (at iteration k):

$$\bar{J}^{(k)} = \frac{1}{N}\sum_{j=1}^{N} J_i^{(k)}, \qquad J_{\text{best}}^{(k)} = \min_{1\le i\le N} J_i^{(k)} \tag{8}$$

Two useful convergence metrics are the gap $(\Delta^{(k)})$ and the swarm fitness standard deviation $(\sigma^{(k)})$:

$$\Delta^{(k)} = \bar{J}^{(k)} - J_{\text{best}}^{(k)}, \qquad \sigma^{(k)} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(J_i^{(k)} - \bar{J}^{(k)})^2} \tag{9}$$

These quantities track the exploitation (reduction of Δ) and exploration (magnitude of σ) dynamics of the swarm.

5.2.2 Genetic algorithm (GA)

Genetic algorithms (GA) are stochastic optimization methods inspired by the principles of Darwinian evolution and natural selection. A GA maintains a population of candidate solutions (individuals) that evolves over generations through the repeated application of three operators: selection, crossover, and mutation. In MATLAB®, the `ga` solver from the Global Optimization Toolbox provides both single-objective and multi-objective (via `gamultiobj`) implementations.
At each generation k, a new population is created from the previous one according to:

1. **Selection:** individuals are chosen with a probability proportional to their fitness value $J_i^{(k)}$. This promotes the survival of the fittest principle by assigning higher reproduction chances to better-performing solutions.
2. **Crossover:** pairs of selected parents exchange portions of their design parameters to create offspring.
3. **Mutation:** with small probability, random alterations are introduced in the offspring's design parameters to maintain genetic diversity and help escape local minima. In particular the mutation function `mutationadaptfeasible` was used, because it generates random changes that adapt based on the success or failure of the previous generation.

In multi-objective optimization, GA uses Pareto dominance to rank solutions based on the evaluation of the two fitness functions, namely, the lift-to-induced-drag ratios of the two aircraft in a formation with a negative sign. The solver `gamultiobj` maintains a Pareto front approximation by combining rank-based selection with crowding distance, thereby balancing convergence toward the true Pareto front and preservation of solution diversity. However, this robustness comes at the cost of increased computational time per generation, since crossover and mutation must be applied to an entire population at each iteration.

5.2.3 Computational cost and optimizer selection

After preliminary benchmarking, the computational cost of performing the entire optimization process was estimated assuming 50 iterations for PSO and 50 generations for GA.
The time required to perform a single iteration (defined as 120 AVL calls) was measured by varying the number of processor cores available to MATLAB®. The PSO solver was employed for single-objective optimization, while the multi-objective GA (`gamultiobj`) was used for formation-flight optimization. The F/A-18C Hornet was chosen as the benchmark aircraft for these timing tests.

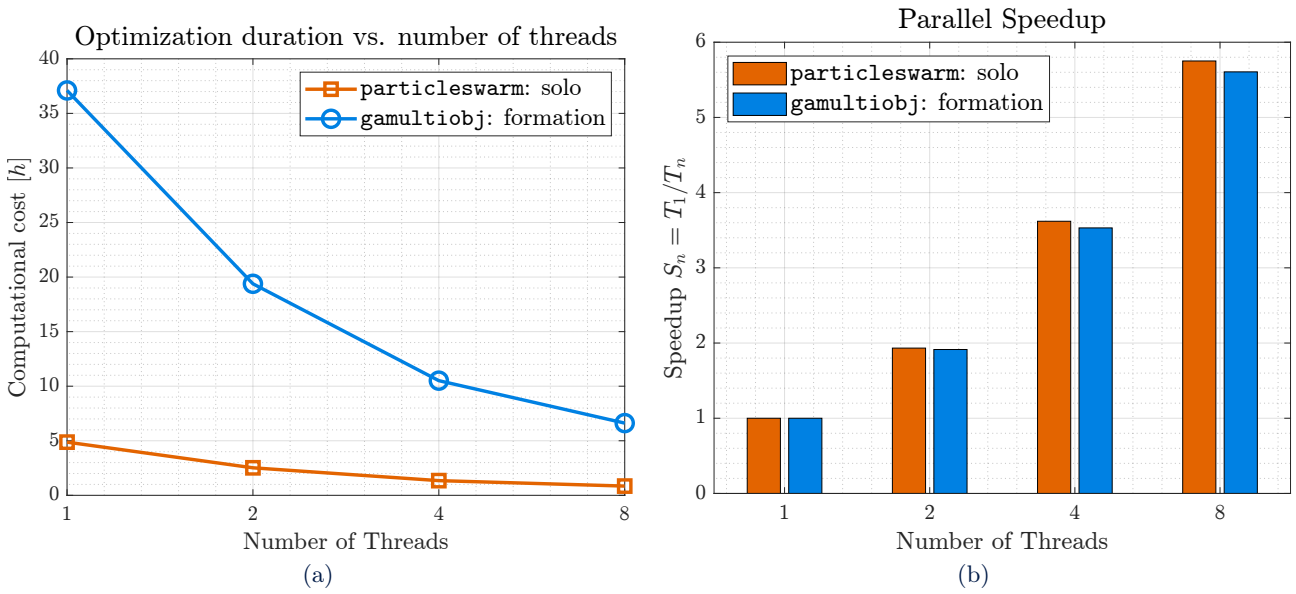


Figure 9: (a) Wall-clock time required to complete a single optimization run (50 PSO iterations or 50 GA generations; one iteration = 120 AVL calls) on the F/A-18 benchmark as a function of allocated MATLAB® cores. (b) Corresponding parallel speedup for PSO and GA obtained from the measured times in (a): $S_n = T_1/T_n$.

As shown in Figure 9a, a consistent downward trend in wall-clock time per iteration is observed as the number of threads increases. The computing time required to complete one formation optimization using the multi-objective method proved prohibitive for achieving a converged Pareto-optimal fitness and for performing the procedure on all three selected aircraft. Furthermore, observing Figure 9b shows how the PSO algorithm speeds up more than a multi-objective algorithm by parallelizing the calculation system, improving its efficiency. For these reasons, the final strategy adopted a compromise: wing optimization was performed in solo trimmed flight using PSO to ensure tractable runtime, while GA results were retained for time-estimate plotting.

5.3. Optimization results

As previously justified, the optimization campaign was carried out independently on three fighter aircraft: the McDonnell Douglas F/A-18C Hornet, the Lockheed Martin/Boeing F-22A Raptor and the Fiat G.91R.

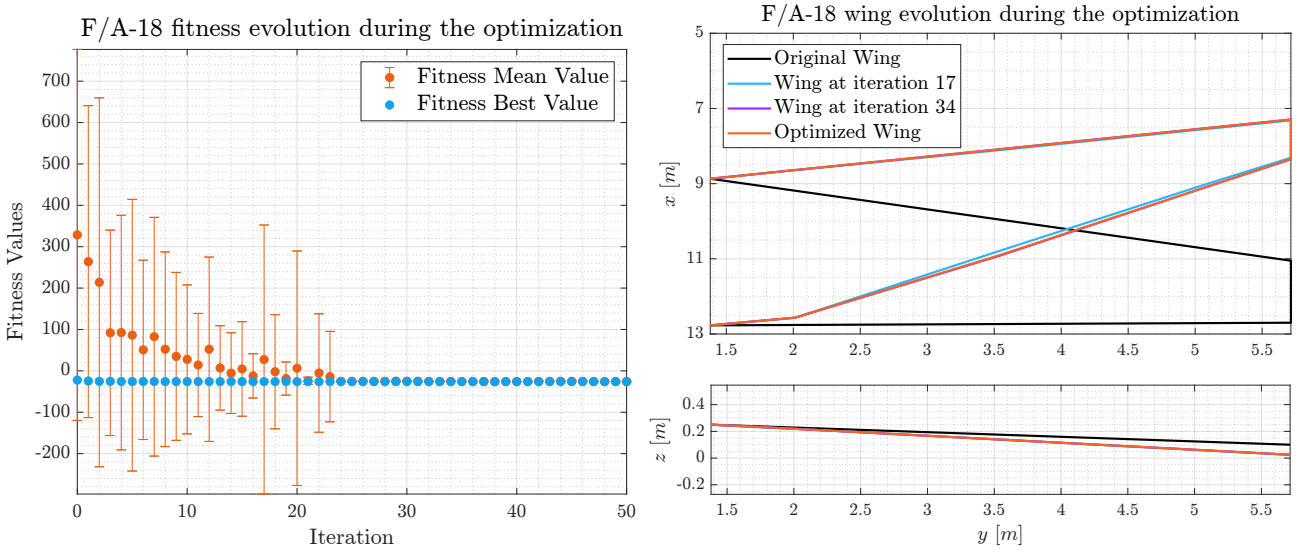
For each aircraft, a population-based particle swarm optimizer was employed to perform single-objective wing design in *solo* trimmed-flight conditions; formation aerodynamics were not included in these runs. The objective function was the penalized aerodynamic efficiency η evaluated at the selected trimmed flight point (see Section 5.1), with constraint violations incorporated through the penalty terms of the fitness.

Each optimization run used a population-based search to explore the continuous design space without gradient information, and convergence was monitored via swarm-based metrics (gap and swarm fitness dispersion) to ensure sufficient reduction of the penalized cost.

Results will be expressed relative to the corresponding baseline geometries and include both convergence histories and final trimmed-point aerodynamic performance.

5.3.1 McDonnell Douglas F/A-18C Hornet

Figure 10 shows a rapid convergence of the swarm mean fitness to the global best fitness. In the present run $\Delta^{(k)}$ decays rapidly to near zero, and the swarm fitness standard deviation $\sigma^{(k)}$ correspondingly decreases. This indicates that the population has mainly concentrated in a small neighborhood of the best solution. This rapid reduction of $\Delta^{(k)}$ demonstrates efficient exploitation: the swarm quickly consolidates toward an improved planform and the optimization is effective at the chosen flight condition.



(a) PSO convergence plot showing swarm mean fitness values with relative standard deviation and global best fitness per iteration.

(b) Planform wing evolution overlay showing the original shape, two intermediate iterations solution and the final planform.

Figure 10: Solution of the particle swarm optimization for the McDonnell Douglas F/A-18C Hornet. Both the evolution of fitness and the evolution of wing shape are graphically represented to show the convergence behavior of the optimization.

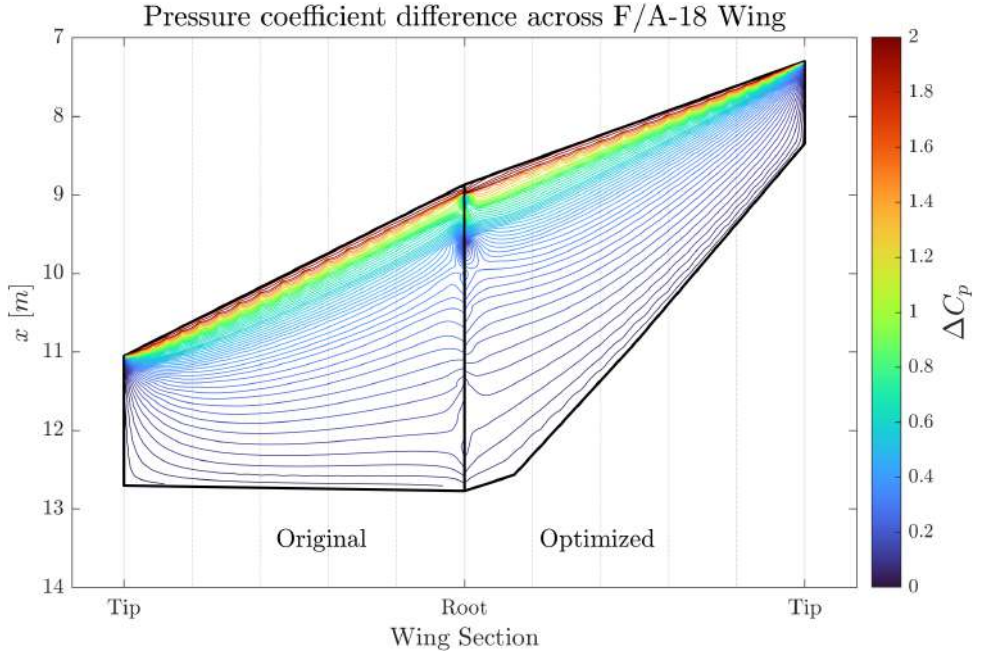


Figure 11: Comparison of pressure coefficient differences (ΔC_p) across the McDonnell Douglas F/A-18C Hornet wing before and after PSO optimization. The left part of the plot shows the original wing, while the right plot shows the optimized configuration.

However, it should be noted that such rapid consolidation may also signal a loss of diversity and the risk of premature convergence towards a local optimum, particularly in highly multimodal contexts or when penalties create highly infeasible regions.

The optimized F/A-18 wing clearly features a forward sweep compared to the original aft-swept planform shown in Figure 10b. This forward sweep reverses the usual spanwise flow due to tridimensional effects: air moves inward toward the root rather than out to the tips.
In practice, this shifts lift inboard, delaying tip stall relative to a rearward-swept wing and keeping ailerons effective at high angles of attack.
In fact, the optimized shape exhibits a pressure coefficient distribution with a maximum absolute value at the wing root, which is higher than in the original configuration. Figure 11 also shows that the ΔC_p isolines tend to load the leading edge (LE) at the tip and the trailing edge (TE) at the root, deviating from the idealized two-dimensional behavior.
In Figure 11, 13, and 15 the ΔC_p isolines show a wavy pattern along the leading edge due to interpolation between adjacent strips. Because AVL employs VLM, the ΔC_p values on the LE panels can reach very high peaks, which distorts the overall pressure distribution display.

Particular emphasis must be placed on the negative aspects of this configuration. At the aeroelastic level there are both benefits and drawbacks. The main benefit and drawback of forward-swept wings is their torsional behavior. Bending loads coupled with torsion tend to twist the wing tips upward (increasing the local angle of attack). While this can increase lift relative to a perfectly rigid wing, it also lowers the divergence dynamic pressure. If the structure does not provide sufficient torsional stiffness, this "wash-in" under load can lead to aeroelastic divergence.

Stability implications must also be considered. For lateral–directional stability the wing contributes via both sweep and dihedral effects. Considering the contribution from sweep alone, a sideslip produces different effective velocities on the two wing panels.
The dynamic pressures on the starboard and port wing can be written as:

$$\begin{cases} q_{\text{Right}} = \frac{1}{2}\rho V^2 \Big[\sin^2\alpha + \cos^2\Lambda \big(\cos^2\alpha + \tan^2\Lambda \sin^2\beta + 2\cos\alpha \sin\beta \tan\Lambda \big) \Big] & \text{(10a)} \\ q_{\text{Left}} = \frac{1}{2}\rho V^2 \Big[\sin^2\alpha + \cos^2\Lambda \big(\cos^2\alpha - \tan^2\Lambda \sin^2\beta - 2\cos\alpha \sin\beta \tan\Lambda \big) \Big] & \text{(10b)} \end{cases}$$

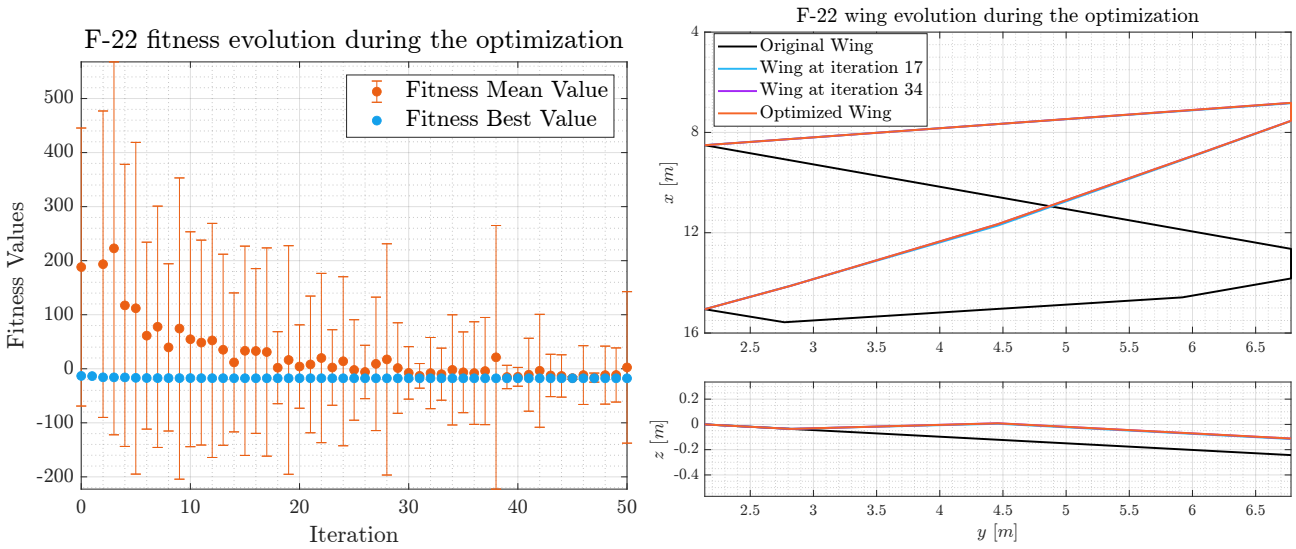
Truncating to first order for $\alpha, \beta \ll 1$ gives:

$$\begin{cases} q_{\text{Right}} = \frac{1}{2}\rho V^2 \Big[\cos^2\Lambda \big(1 + 2\beta \tan\Lambda \big) \Big] & \text{(11a)} \\ q_{\text{Left}} = \frac{1}{2}\rho V^2 \Big[\cos^2\Lambda \big(1 - 2\beta \tan\Lambda \big) \Big] & \text{(11b)} \end{cases}$$

For forward sweep with positive sideslip (i.e., $\Lambda < 0$ and $\beta > 0$), the right wing produces less lift than the left wing (and similarly less drag). Consequently, a forward-swept wing tends to be destabilizing both directionally ($C_{\mathcal{N},\beta}\big|_\Lambda < 0$) and laterally ($C_{\mathcal{L},\beta}\big|_\Lambda > 0$). If not counteracted by appropriate control authority or damping, such effects can excite unstable modes such as Dutch roll, as observed during flight tests of the Grumman X-29, an experimental forward-swept wing aircraft [13].
Longitudinal stability is also affected: forward sweep shifts the aerodynamic center forward, reducing the static margin ($x_G - x_A$) and making the aircraft harder to trim. In practice, this is commonly mitigated by shifting the centre of gravity forward, increasing tail moment arm/area (tail volume coefficient), or employing active control laws to restore acceptable handling characteristics.

5.3.2 Lockheed Martin/Boeing F-22A Raptor

Exactly the same aeroelastic and stability analysis can also be performed for the F-22. The new geometry shows a reversal of the wing sweep, producing a forward-swept planform. However, the optimization behavior differs: as shown in Figure 12a, the swarm mean fitness approaches the global minimum more slowly, and the fitness standard deviation exhibits a continuous up-and-down pattern rather than monotonically decreasing.



(a) PSO convergence plot showing swarm mean fitness values with relative standard deviation and global best fitness per iteration.

(b) Planform wing evolution overlay showing the original shape, two intermediate iterations solution and the final planform.

Figure 12: Solution of the particle swarm optimization for the Lockheed Martin/Boeing F-22A Raptor. Both the evolution of fitness and the evolution of wing shape are graphically represented to show the convergence behavior of the optimization.

This optimization signature has practical consequences. The slow approach of the mean toward the best value and the persistent oscillation of $\sigma^{(k)}$ indicate a more rugged, multimodal design landscape for the F-22 case. Possible causes include competing aerodynamic effects between wing parameters, the presence of tight feasibility constraints imposed by the penalty formulation, or the existence of broad parameter regions containing several near-equivalent local minima. Such characteristics are typical of design problems where multiple mechanisms interact in non-additive ways, making it difficult for the optimizer to consistently drive the population toward a single dominant solution.
The direct consequences are:

- the obtained solution is more likely to depend on initialization and on the random seed of the algorithm, reducing reproducibility across runs.
- additional sensitivity analysis and local refinement are required to confirm that the reported planform provides a genuine aerodynamic improvement rather than being an artifact of stochastic convergence.
- convergence metrics such as $\Delta^{(k)}$ and $\sigma^{(k)}$ should be carefully monitored to distinguish between premature stagnation and true global convergence.
- computational cost may increase significantly, since more iterations or multiple independent runs are necessary to obtain statistically reliable results.

5.3.3 Fiat G.91R

Figure 14 shows that the G.91 case behaves differently. Similar to the F/A-18, the fitness evolution is smooth: the mean fitness improves steadily and the fitness standard deviation decreases almost monotonically, indicating that the swarm converges in a stable and well-conditioned manner without

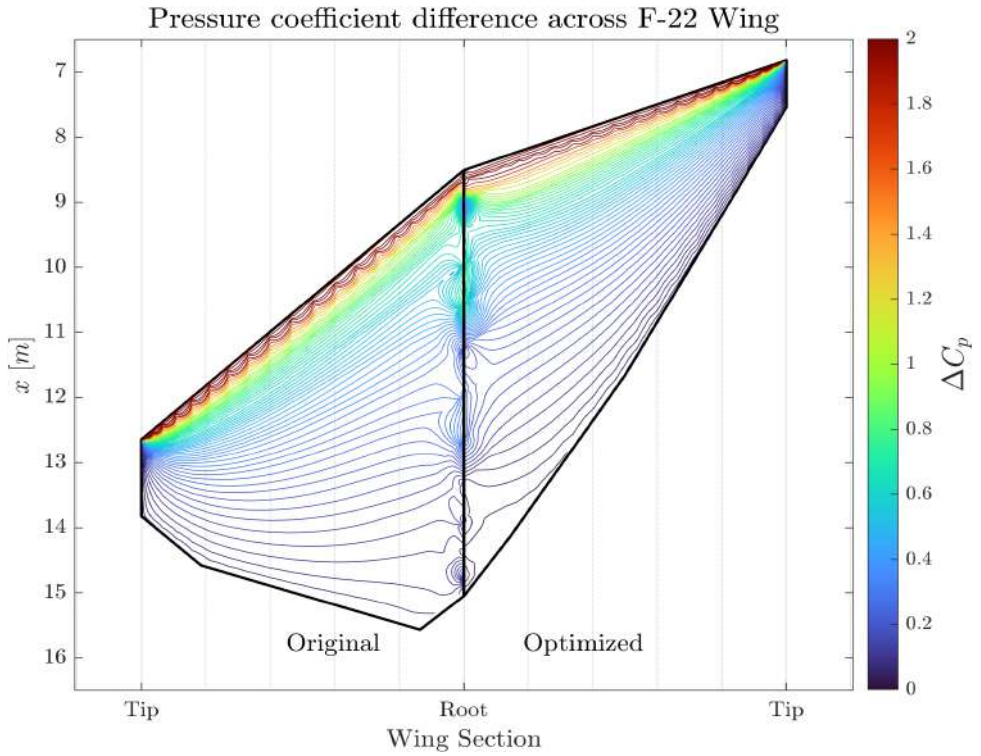
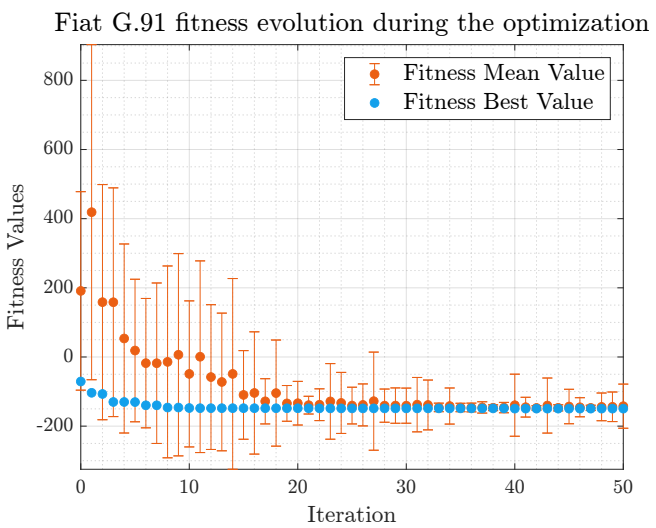
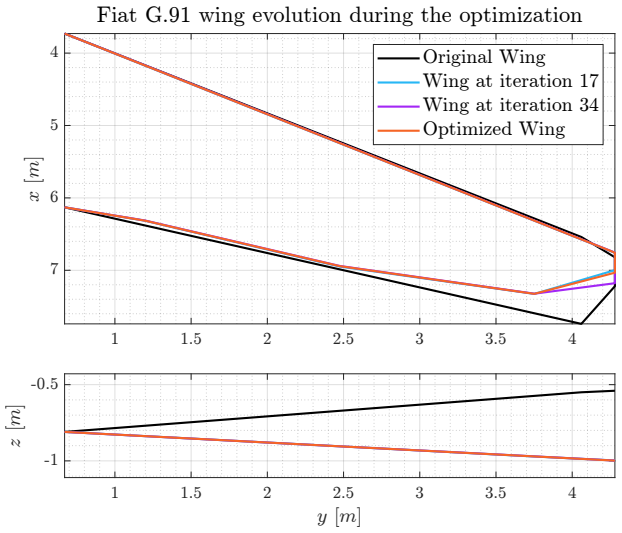


Figure 13: Comparison of pressure coefficient differences (ΔC_p) across the Lockheed Martin/Boeing F-22A Raptor wing before and after PSO optimization. The left part of the plot shows the original wing, while the right plot shows the optimized configuration.

the oscillations observed for the F-22. This behavior suggests a simpler design space with fewer pronounced local minima and a more direct gradient of improvement toward the optimum.



(a) PSO convergence plot showing swarm mean fitness values with relative standard deviation and global best fitness per iteration.



(b) Planform wing evolution overlay showing the original shape, two intermediate iterations solution and the final planform.

Figure 14: Solution of the particle swarm optimization for the Fiat G.91. Both the evolution of fitness and the evolution of wing shape are graphically represented to show the convergence behavior of the optimization.

In contrast, the geometric modifications follow a distinct pattern. The optimized planform remains broadly consistent with the baseline, with no radical departure in overall shape. The most notable

modification is in the chord distribution, which becomes more tapered, effectively reducing area in the outboard sections. This change tends to shift lift inboard, which can decrease induced drag and reduce bending moments at the wing root, thereby enhancing aerodynamic efficiency while also improving structural loading characteristics. Conversely, the dihedral of the baseline configuration is reversed into an anhedral.This adjustment aligns the G.91 with the dihedral trend observed in the F/A-18 and F-22 optimizations, suggesting that the optimization process consistently favors anhedral for fighter-type configurations.

It is therefore necessary to examine the stability implications of introducing wing dihedral. The principal influence appears in the lateral–directional dynamics: dihedral produces a differential in vertical velocity components between the two wings under sideslip. For a sideslip angle β and wing dihedral angle Γ, the vertical velocity components experienced by the right and left wings can be expressed as:

$$\begin{cases} V_{z,\text{Right}} = V \sin\beta \sin\Gamma & \text{(12a)} \\ V_{z,\text{Left}} = -V \sin\beta \sin\Gamma & \text{(12b)} \end{cases}$$

In the case of negative dihedral (anhedral, $\Gamma < 0$) combined with a positive sideslip ($\beta > 0$), the right wing experiences a downward vertical velocity component ($V_{z,\text{Right}} < 0$), while the left wing experiences an upward component ($V_{z,\text{Left}} > 0$). The consequence is a reduction in the effective angle of attack on the right wing and an increase on the left wing, producing a rolling moment that is positive for positive sideslip. Instead of damping the motion, this response amplifies the roll, such that the lateral stability derivative becomes destabilizing: $C_{\mathcal{L},\beta}\big|_{\Gamma} > 0$.
The effect of dihedral on directional stability is more subtle and highly dependent on the detailed geometry. A more refined assessment, for example using strip-theory analysis or higher-fidelity aerodynamic models, would be required to fully quantify its contribution to the yawing stability derivatives.

The results can be interpreted as a compromise between aerodynamic efficiency and lateral–directional stability requirements, since anhedral tends to reduce excessive roll stability and enhance maneuverability—features typically advantageous in combat aircraft. The emergence of this trend across all three aircraft indicates that the optimizer is capturing a recurrent aerodynamic preference rather than an isolated geometric artifact.

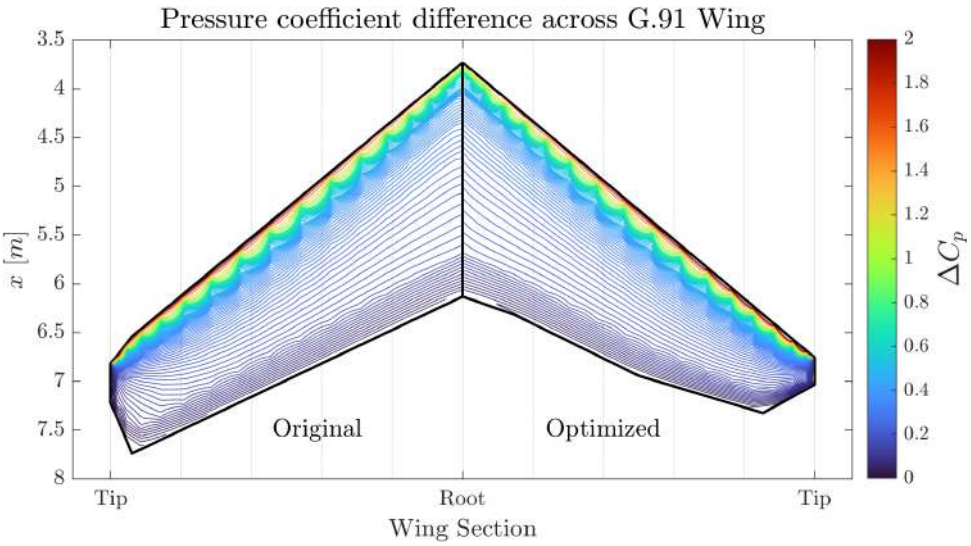


Figure 15: Comparison of pressure coefficient differences (ΔC_p) across the Fiat G.91R wing before and after PSO optimization. The left part of the plot shows the original wing, while the right plot shows the optimized configuration.

5.4. Trimmed-cruise condition for optimized configurations

The trim analysis described in subsection 4.4 was repeated for the optimized geometries. By conducting the comparison in trim, it is possible to determine whether the improvements introduced by the optimization process translate into practical performance gains in steady formation flight, rather than being limited to isolated aerodynamic coefficients.

Table 3 and Table 4 present the results for the original and optimized geometries, respectively. The comparison focuses on the trimmed angle of attack in solo flight as well as the corresponding angles of attack in formation for both the leader and the trailer.
In addition, the induced drag coefficients are reported in counts, where one count corresponds to 10^{-4}. This representation makes it easier to quantify small variations in induced drag between the different configurations.

Aircraft type (original)	$\boldsymbol{\alpha_{trim}^{solo}}$	$\boldsymbol{\alpha_{trim}^{lead}}$	$\boldsymbol{\alpha_{trim}^{trail}}$	$\boldsymbol{C_{D,i}^{solo}}$	$\boldsymbol{C_{D,i}^{lead}}$	$\boldsymbol{C_{D,i}^{trail}}$
Fiat G.91R	4.1°	4.0°	3.9°	40	36	26
McDonnell Douglas F/A-18C Hornet	5.9°	5.9°	5.7°	241	226	210
Lockheed Martin/Boeing F-22A Raptor	9.1°	8.7°	8.5°	317	325	308

Table 3: Trimmed AOA of all baseline types, flying solo (α_{trim}^{solo}) and in a two-aircraft formation (α_{trim}^{lead} and α_{trim}^{trail}), in the reference configuration. The induced drag coefficient $C_{D,i}$ is reported in counts.

Aircraft type (optimized)	$\boldsymbol{\alpha_{trim}^{solo}}$	$\boldsymbol{\alpha_{trim}^{lead}}$	$\boldsymbol{\alpha_{trim}^{trail}}$	$\boldsymbol{C_{D,i}^{solo}}$	$\boldsymbol{C_{D,i}^{lead}}$	$\boldsymbol{C_{D,i}^{trail}}$
Fiat G.91R	4.2°	4.1°	4.0°	27	24	13
McDonnell Douglas F/A-18C Hornet	6.2°	6.1°	6.0°	206	185	172
Lockheed Martin/Boeing F-22A Raptor	10.1°	9.3°	9.1°	284	292	278

Table 4: Trimmed AOA of all optimized types, flying solo (α_{trim}^{solo}) and in a two-aircraft formation (α_{trim}^{lead} and α_{trim}^{trail}), in the reference configuration. The induced drag coefficient $C_{D,i}$ is reported in counts.

All three optimized configurations show clear improvements in aerodynamic performance, both in solo flight and in formation. However, the trimmed angle of attack did not decrease with optimization; instead, it increased. This indicates that an aerodynamically optimized geometry does not necessarily operate at a lower angle of attack in trimmed cruise. Rather, the improvement is achieved through a more favorable distribution of lift and drag, which enhances efficiency even at higher angles of attack.
When comparing induced drag, the general trend shows reductions between solo, leader, and trailer positions, both in the original and optimized configurations. The only exception is the F-22: in both the original and optimized configurations, the leader position consistently experiences higher induced drag than solo flight. This result underlines that the benefits of optimization are not uniformly distributed across all roles in a formation.
Table 5 reports the percentage variation of induced drag relative to the solo-flight value of the baseline configuration, highlighting the induced drag reduction achieved with optimization.

Aircraft type	$\boldsymbol{\Delta C_{D,i}^{solo}}$	$\boldsymbol{\Delta C_{D,i}^{lead}}$	$\boldsymbol{\Delta C_{D,i}^{trail}}$
Fiat G.91R	-33.2%	-39.2%	-68.6%
McDonnell Douglas F/A-18C Hornet	-14.7%	-23.3%	-28.9%
Lockheed Martin/Boeing F-22A Raptor	-10.4%	-7.9%	-12.5%

Table 5: Percentage variation of induced drag coefficients $\Delta C_{D,i}$ for optimized configurations, referenced to the solo-flight values of the corresponding baseline geometries.

6. Influence of the relative position

At the end of the wing planform optimization phase, an additional analysis was performed to evaluate the influence of the optimized geometrical configurations on the aerodynamic coefficients as a function of the relative position between a leading and a trailing aircraft in formation flight. The relative position was parameterized by three independent displacement variables, ΔX, ΔY and ΔZ, each discretized at three levels. The design space was defined as:

$$\Delta X = \{2.0,\, 3.0,\, 4.0\}, \qquad \Delta Y = \{-0.5,\, 0.0,\, 0.5\}, \qquad \Delta Z = \{-0.5,\, 0.0,\, 0.5\}. \tag{13}$$

An initial attempt used $\Delta Y = \{-1.0,\, 0.0,\, 1.0\}$, consistent with the reference system chosen, in which $\Delta Y = -1.0$ corresponds to the trailing aircraft being located directly behind the leader. That configuration, however, led to persistent non-convergence of the aerodynamic solver within acceptable runtime. Therefore, the ΔY range was adjusted to the levels reported above to ensure numerical stability and feasible computation.

To limit the total number of configurations while preserving a representative coverage of the input space, a design of experiments (DoE) approach based on latin hypercube sampling (LHS) was adopted. LHS ensures stratified sampling of each variable range and reduces clustering of points, thus maintaining statistical representativeness with fewer samples. Using LHS, the full-factorial set of $3^3 = 27$ combinations was reduced to nine sampled configurations, which offered a tractable trade-off between exploratory breadth and computational cost. The selected configurations are summarized in Table 6. The formation-position sensitivity analysis was performed on both the optimized and the baseline planforms for each aircraft. This enabled direct pre-/post-optimization comparisons and quantification of the relative aerodynamic advantages introduced by the wing modifications.

$X = 2.0$ $Y = 0.0$ $Z = -0.5$	$X = 3.0$ $Y = -0.5$ $Z = 0.0$	$X = 4.0$ $Y = 0.5$ $Z = 0.5$
$X = 3.0$ $Y = 0.5$ $Z = 0.0$	$X = 4.0$ $Y = 0.0$ $Z = 0.5$	$X = 2.0$ $Y = -0.5$ $Z = -0.5$
$X = 4.0$ $Y = -0.5$ $Z = 0.5$	$X = 2.0$ $Y = 0.5$ $Z = -0.5$	$X = 3.0$ $Y = 0.0$ $Z = 0.0$

Table 6: Latin Hypercube Sampling (LHS) grid showing the selected 3×3 combinations of design variables.

For each sampled formation configuration, the same trimming procedure and aerodynamic evaluations described in Section 4.4 were executed.

The discrete set of resulting response values (hereafter denoted as the change in induced drag, $\Delta C_{D,i}$, associated to each $(\Delta X, \Delta Y, \Delta Z)$ sample) was then extended to the full physical space by means of Gaussian process regression, also known as Kriging. Kriging predicts the value of a function at a given point by computing a weighted average of nearby sampled values, using a covariance (kernel) to encode spatial correlation. The Gaussian-process surrogate was implemented with the MATLAB® function *fitrgp* from the Statistics and Machine Learning Toolbox. The function takes as input the sample data (in this case, the positions used during the simulations and the corresponding $\Delta C_{D,i}$) together with several settings.

A Matérn 5/2 covariance was selected as the kernel (`KernelFunction='matern52'`). The Matérn 5/2 kernel is a stationary (since it only depends on distances between points) isotropic (because in

MATLAB® Euclidean distances are used) covariance function that provides smooth, physically plausible interpolants for aerodynamic responses. The option `FitMethod='none'` was used to skip hyperparameter optimization and use fixed hyperparameter values, while `PredictMethod='exact'` was selected to obtain exact Gaussian process predictions using the complete training dataset. Finally, the `Sigma` parameter was set to 0.5, corresponding to 1% of the characteristic data scale, to account for measurement noise and experimental variability in the AVL results.

6.1. Sensitivity of the results with respect to formation-position

To compare the results of this study with those obtained during the AFF project, Figure 16 presents the induced drag contour plots as a function of Y-Z position at predefined X values for the F/A-18C. The contour sections here do not exactly match those from the AFF project because the initial LHS sampling domain used in this study differed. Despite that, valid comparisons can be made by inspecting the shape, location and intensity of the favorable regions.
A notable feature is the apparent clockwise rotation of the highly unfavorable region relative to the highly favorable region as X increases. This is physically consistent: the leader's right-wing tip vortex rotates counterclockwise, but as the wake convects downstream, it is pushed downward by the downwash, producing the observed clockwise shift of the unfavorable zone upward and outward with increasing X.
Quantitatively, the AVL results for the original wing (Panels 16a, 16c, and 16e give a maximum induced-drag reduction of 46%, compared to 40% reported by AFF under the same flight conditions. The main disagreement is positional: this study finds the peak at $Z \approx 0$, $Y \geq 0$, whereas AFF placed it at $Z < 0$, $Y < 0$. The most likely cause is the sampling density: for computational-cost reasons, the Gaussian-process training set used here is much sparser than the AFF sampling, which can shift the mapped peak location even if the overall trend and magnitude remain comparable.
When comparing the optimized case with the non-optimized one, the region where $\Delta C_{D,i} < 0$ is expanded, particularly within the leader's wake. This is because the optimized forward-swept wing drives the flow velocity inward, reducing the lateral extent of the wake. In addition, the maximum reduction at the most favorable point has increased by approximately 20%, reflecting the effect of the optimized geometry on overall aerodynamic performance.

Figure 17 through Figure 22 present a comparison between flight formation with and without wing optimization for each of the three aircraft analyzed.
The aircraft in Figure 17, Figure 19, and Figure 21 are positioned at the optimal point for minimizing the induced drag of the trailing aircraft. In order to accurately interpret these figures, it is essential to acknowledge that the values displayed represent the percentage variation in the induced drag of the trailer in comparison to the drag that would be experienced when flying solo with the original wing geometry.
The contour slices positioned at fixed X values between these figures (i.e., $X = 1.5, 2.25, 3, 3.75, 4.5$) identify regions in relative position space where the trailer aircraft experiences an induced drag advantage, $\Delta C_{D,i} < 0$. Furthermore, the distribution of the interpolated function $\Delta C_{D,i}$ along the X and Y directions, with Z fixed at the optimal point, is projected onto the floor of each image.
These maps make it possible to identify the favorable flight corridors in which induced drag is minimized and mutual aerodynamic benefits are achieved. Consistent with expectations and with findings reported in the literature, the advantage of formation flying is obtained when there is sufficient longitudinal and lateral separation between the aircraft. In particular, the optimal condition occurs when the wingtips are approximately aligned, that is when the two wingtip vortices interact.
As shown in Figure 17, Figure 19, and Figure 21, the optimized case exhibits an enlargement of the favorable region. This implies that the optimized wing not only improves performance in solo flight but also enhances the robustness of formation benefits, making them less sensitive to small deviations in relative positioning.
Figure 18, Figure 20, and Figure 22 illustrate the variation of induced-drag reduction ($\Delta C_{D,i}$) along the streamwise position X. For each aircraft, both the original and optimized configurations are

compared, considering the trailer and the leader separately. The spanwise offset is fixed at the best lateral separation of the original wing (i.e., Y_{best}^{OG} of Table 7), while different vertical separations are investigated at $Z = 0, \pm 0.5, \pm 1$.
For the trailer, the optimized configurations consistently yield deeper minima than the original designs, indicating greater potential benefits; however, the effect remains strongly dependent on Z. For the leader, the optimized cases show a modest yet systematic improvement over the original, with the added advantage that the sensitivity to vertical displacement is less pronounced than for the trailer.

Tables 7-9 summarize the optimum trailer positions in formation flight and the resulting aerodynamic effects on trimmed angle of attack and induced drag.
For the original wing geometries (Tables 7 and 8), optimum positions cluster near $X \approx 3$, $Y \approx 0$, $Z \approx 0$. In this regime, trailers achieve significant induced drag reductions and lower trimmed angles of attack. Leaders generally experience small benefits (G.91R, F/A-18C) or a slight penalty (F-22A). With optimized wing geometries (Tables 7, 9), the optimum formation remains in the same vicinity, but drag reduction is consistently enhanced. Notably, the F-22A penalty for the leader is alleviated, while the G.91R and F/A-18C see further improvements.
AVL predicts induced-drag reductions of -136.8% (baseline) and -152.9% (optimized) for the G.91R trailer. The trailer aircraft in the formation would have an induced thrust that is 36% and 52% of the induced drag of the solo aircraft. Formation-flight studies consistently report maximum trailing benefits for two-aircraft formations in the range of 30–60% [14] [15]. Interestingly, Veldhuis predicted a 127% drag reduction for a third A330 aircraft in an inverted-V formation [15]. In any case, the AVL results should be regarded as numerical artifacts rather than feasible aerodynamic outcomes, even if other VLM models predicted similar outcomes.

Aircraft type	$\boldsymbol{X_{best}^{OG}}$	$\boldsymbol{Y_{best}^{OG}}$	$\boldsymbol{Z_{best}^{OG}}$	$\boldsymbol{X_{best}^{OPT}}$	$\boldsymbol{Y_{best}^{OPT}}$	$\boldsymbol{Z_{best}^{OPT}}$
Fiat G.91R	3	-0.3	0	3.1	0	0
McDonnell Douglas F/A-18C Hornet	3.1	0	0	3.1	0	0
Lockheed Martin/Boeing F-22A Raptor	3.2	0	0	2.8	0.1	0

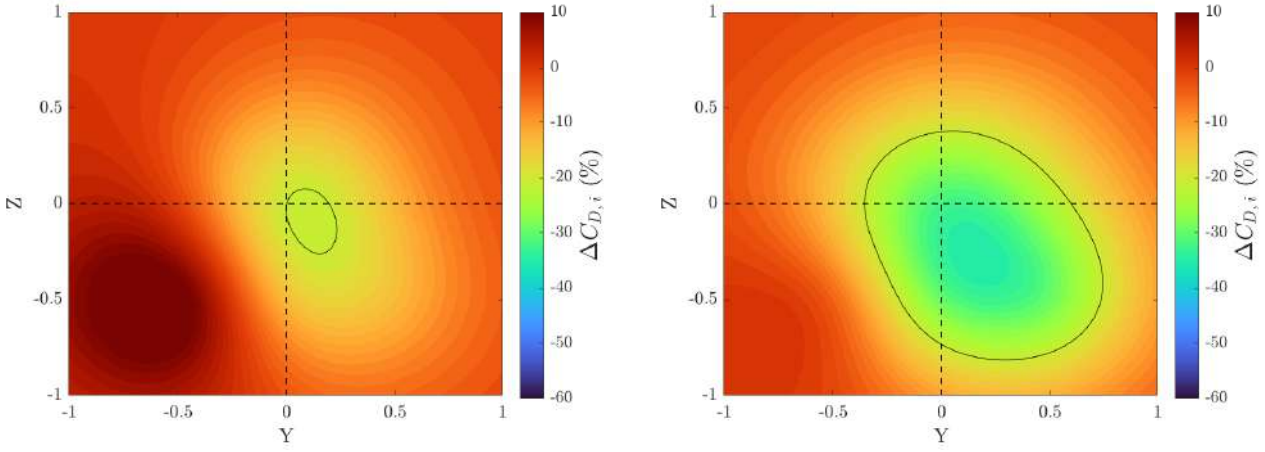
Table 7: Optimum position for the maximum induced drag reduction of the trailer aircraft with the original (X^{OG}) and the optimized (X^{OPT}) wing geometry.

Aircraft type (original)	$\boldsymbol{\alpha_{trim}^{solo}}$	$\boldsymbol{\alpha_{trim}^{lead}}$	$\boldsymbol{\alpha_{trim}^{trail}}$	$\boldsymbol{\Delta C_{D,i}^{lead}}$	$\boldsymbol{\Delta C_{D,i}^{trail}}$
Fiat G.91R	4.1°	4.1°	3.7°	-6.1%	-136.8%
McDonnell Douglas F/A-18C Hornet	5.9°	5.9°	4.9°	-3.1%	-46.5%
Lockheed Martin/Boeing F-22A Raptor	9.1°	8.8°	7.5°	1.6%	-34.1%

Table 8: Trimmed AOA of all original types, flying solo (α_{trim}^{solo}) and in a two-aircraft formation (α_{trim}^{lead} and α_{trim}^{trail}), in the optimum position. The percentage variation in $C_{D,i}$ is also reported for both the leader and the trailing plane with respect to the individual aircraft with original wing geometry.

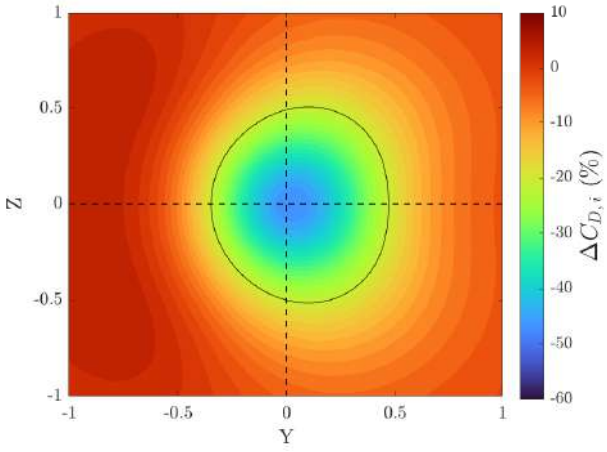
Aircraft type (optimized)	$\boldsymbol{\alpha_{trim}^{solo}}$	$\boldsymbol{\alpha_{trim}^{lead}}$	$\boldsymbol{\alpha_{trim}^{trail}}$	$\boldsymbol{\Delta C_{D,i}^{lead}}$	$\boldsymbol{\Delta C_{D,i}^{trail}}$
Fiat G.91R	4.2°	4.2°	3.5°	-32.9%	-152.9%
McDonnell Douglas F/A-18C Hornet	6.2°	6.2°	5.2°	-9.8%	-60.5%
Lockheed Martin/Boeing F-22A Raptor	10.1°	9.3°	8.1°	-3.8%	-41.3%

Table 9: Trimmed AOA of all optimized types, flying solo (α_{trim}^{solo}) and in a two-aircraft formation (α_{trim}^{lead} and α_{trim}^{trail}), in the optimum position. The percentage variation in $C_{D,i}$ is also reported for both the leader and the trailing plane with respect to the individual aircraft with original wing geometry.

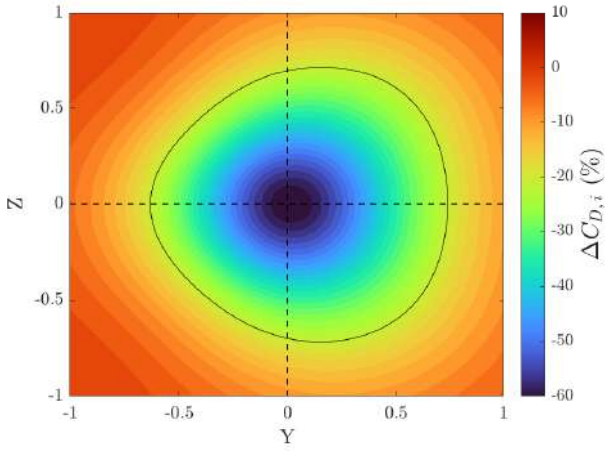


(a) Original wing: $X = 2$

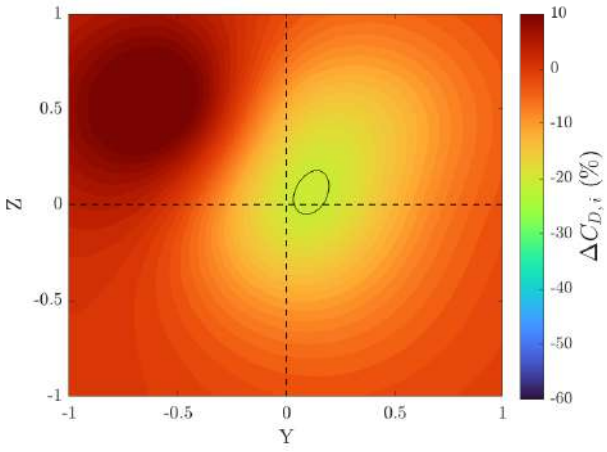
(b) Optimized wing: $X = 2$



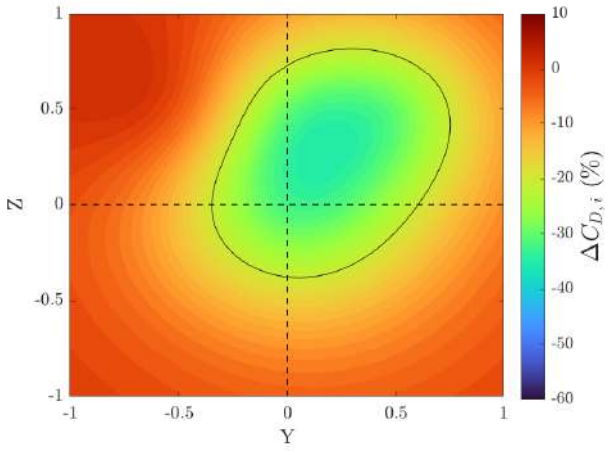
(c) Original wing: $X = 3$



(d) Optimized wing: $X = 3$

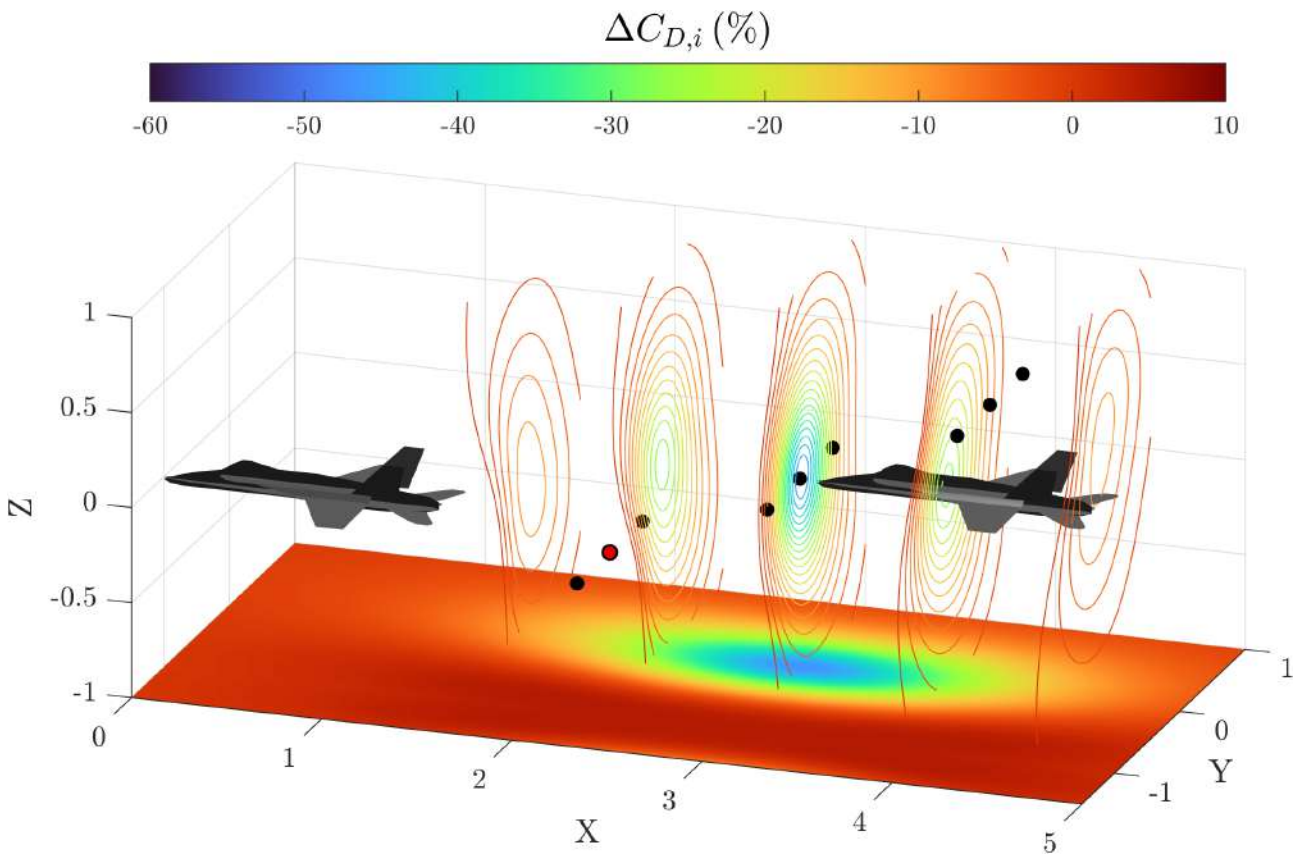


(e) Original wing: $X = 4$

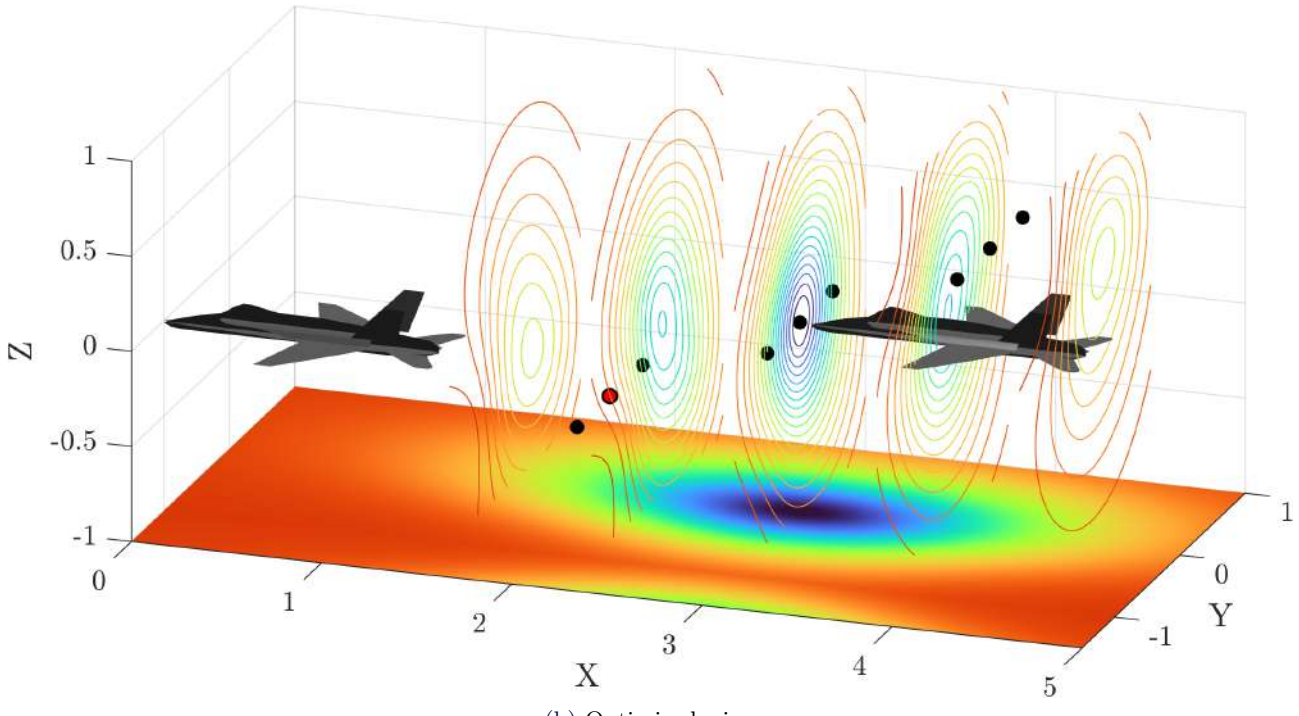


(f) Optimized wing: $X = 4$

Figure 16: Summary of induced drag ($C_{D,i}$) reduction, relative to the non-optimized solo flight, shown as contour plots as a function of Y-Z position for the McDonnell Douglas F/A-18C Hornet. Panels (a), (c), and (e) correspond to the original wing configuration, while panels (b), (d), and (f) correspond to the optimized wing. Solid line contour indicates region of 20% or more induced drag reduction.

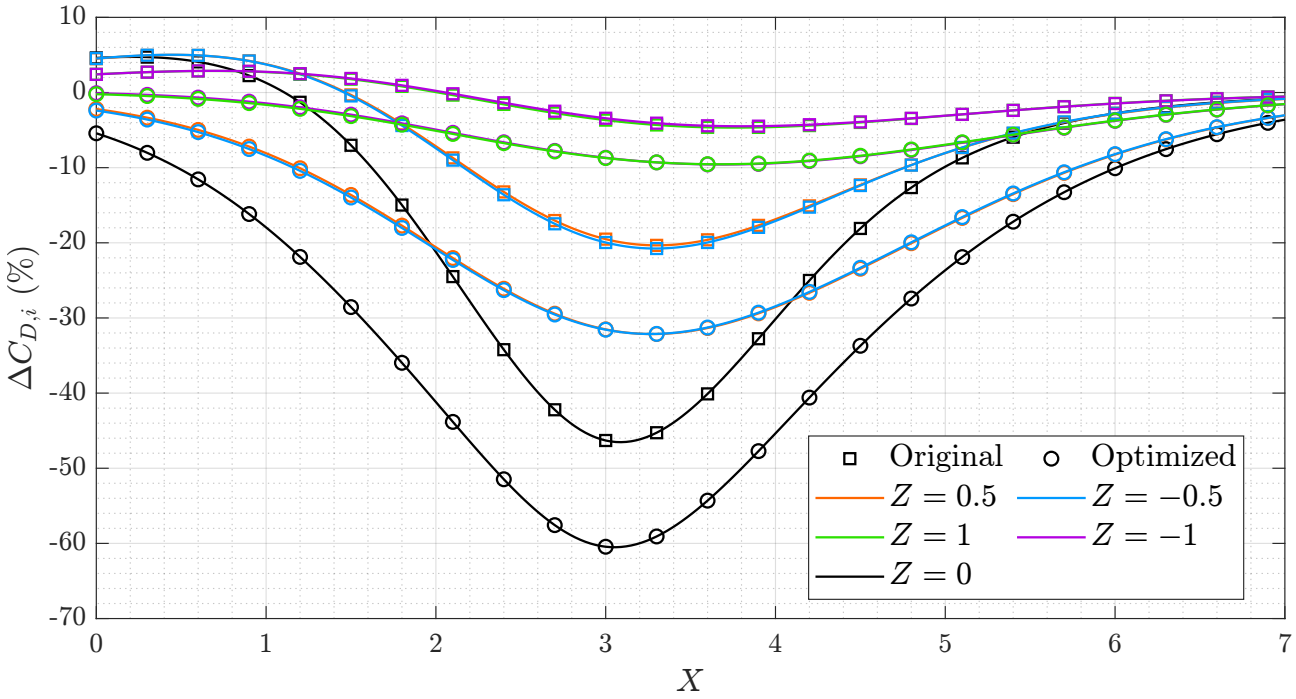


(a) Original wing

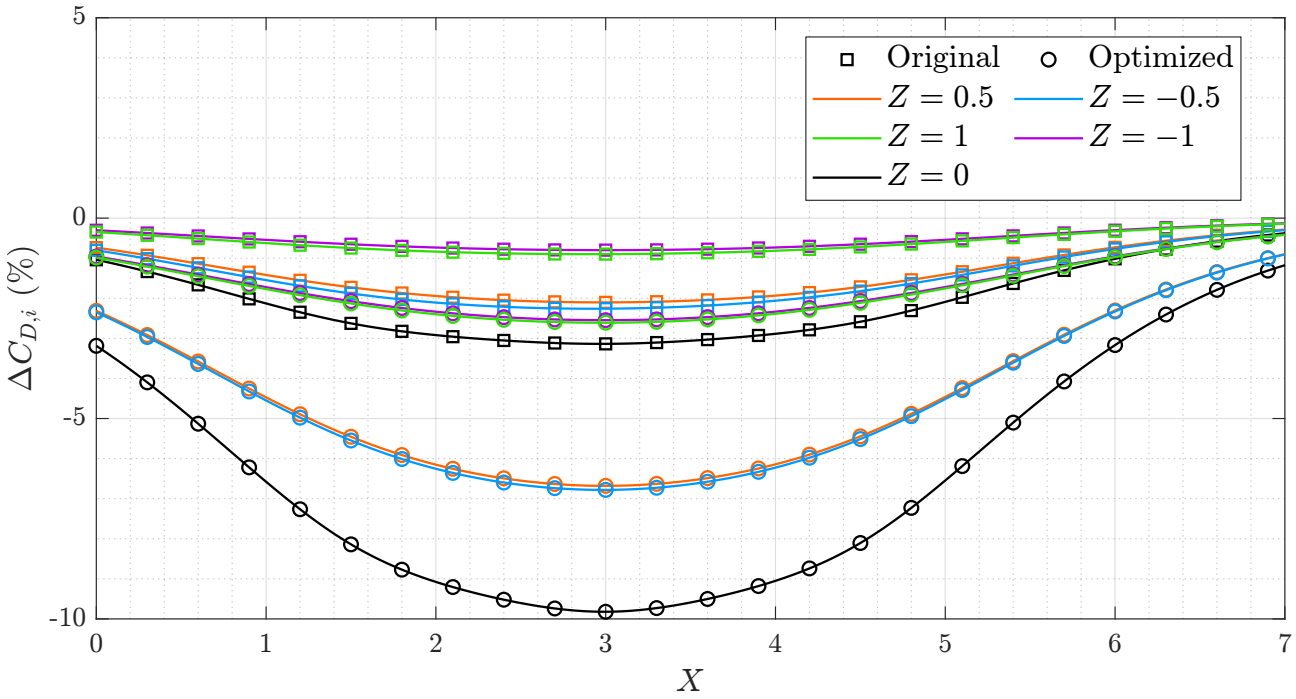


(b) Optimized wing

Figure 17: Comparison of the effect of relative formation position on the induced drag change $\Delta C_{D,i}$ (%) of the McDonnell Douglas F/A-18C Hornet trailing aircraft: original wing shape (a) and optimized wing design (b). $\Delta C_{D,i}$ is defined as the difference between the induced drag coefficient of the trailer in formation and the induced drag coefficient of the same aircraft in solo flight with the original wing geometry. Contour slices indicate regions where $\Delta C_{D,i} < 0$, i.e. where the trailer experiences a drag reduction. The floor projection shows the filled contour in the X-Y plane at the optimal Z-position. The black dots represent the LHS studied point, while the red one the reference configuration.

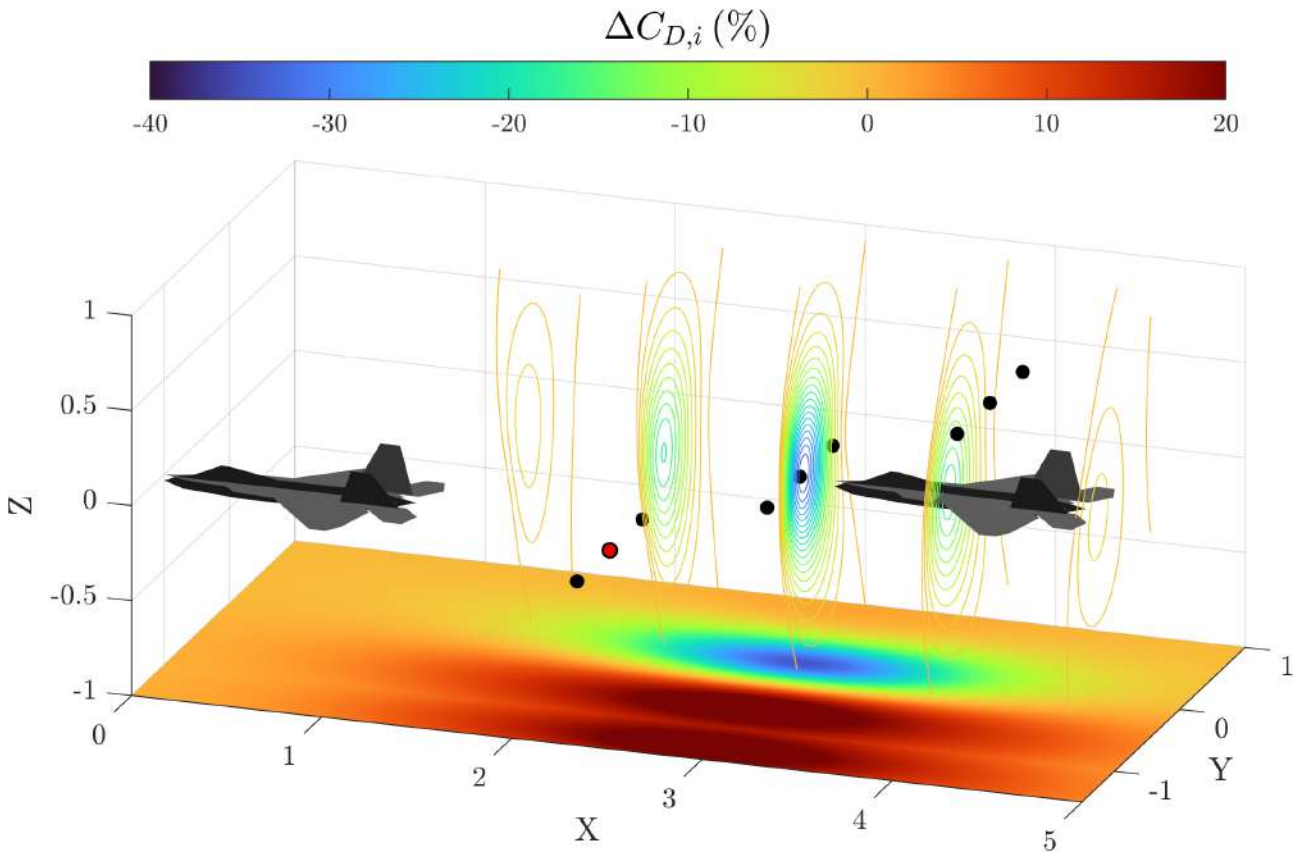


(a) Trailer aircraft

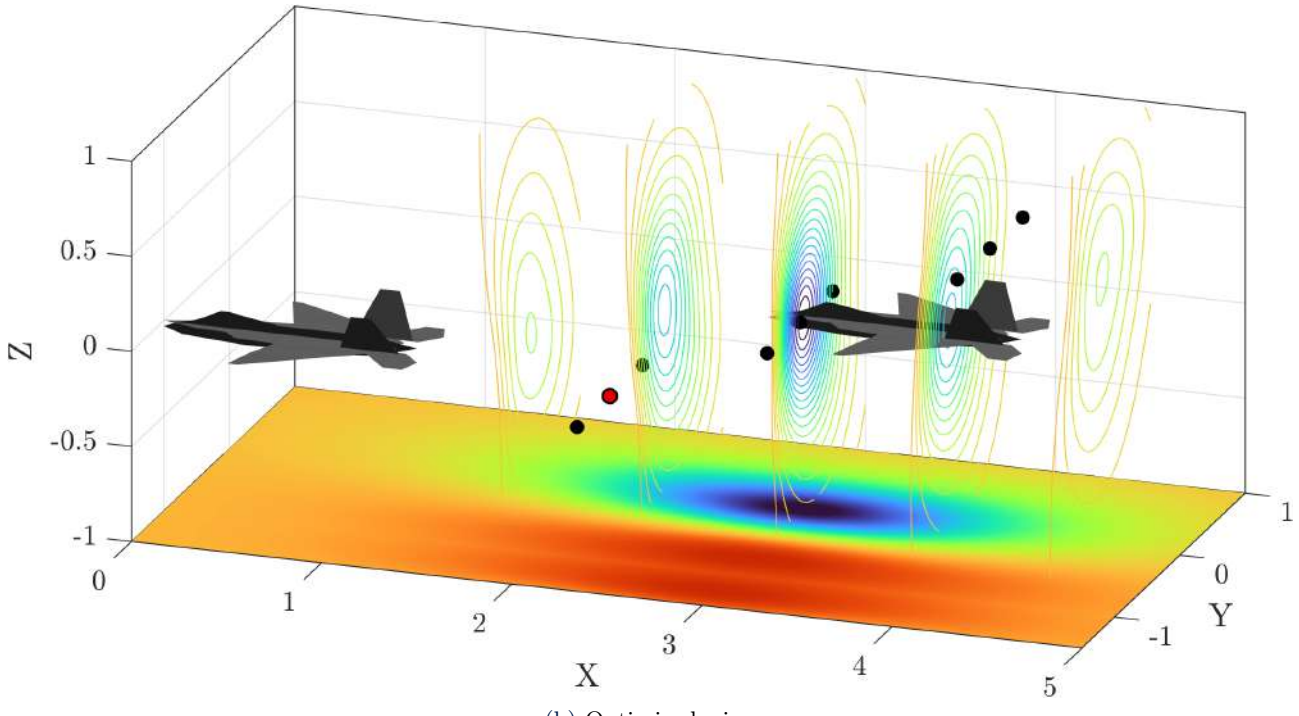


(b) Leader aircraft

Figure 18: Induced-drag reduction trends along X for the McDonnell Douglas F/A-18C Hornet trailing (a) and leader (b) aircraft. Original configurations are shown with square markers, and optimized configurations with circles. The spanwise offset is fixed at the best lateral separation of the original wing (Y_{best}^{OG}), while vertical positions are varied at $Z = 0, \pm 0.5, \pm 1$ (color-coded in the legend).

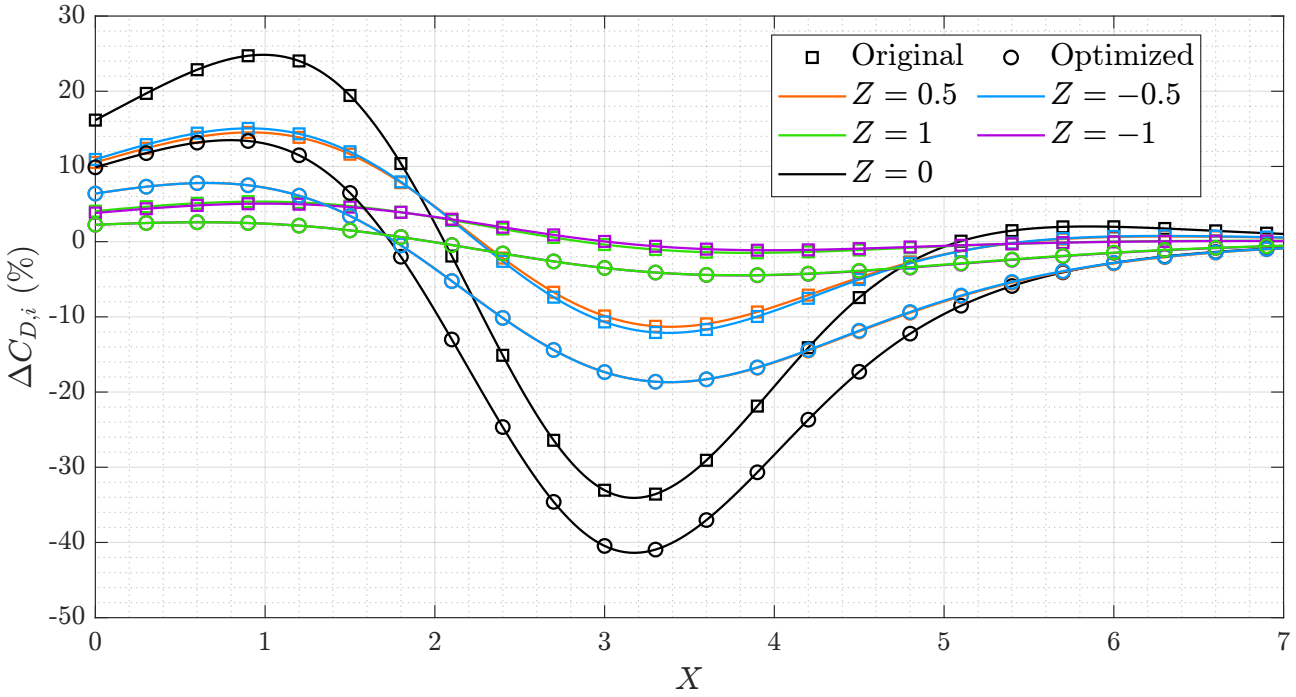


(a) Original wing

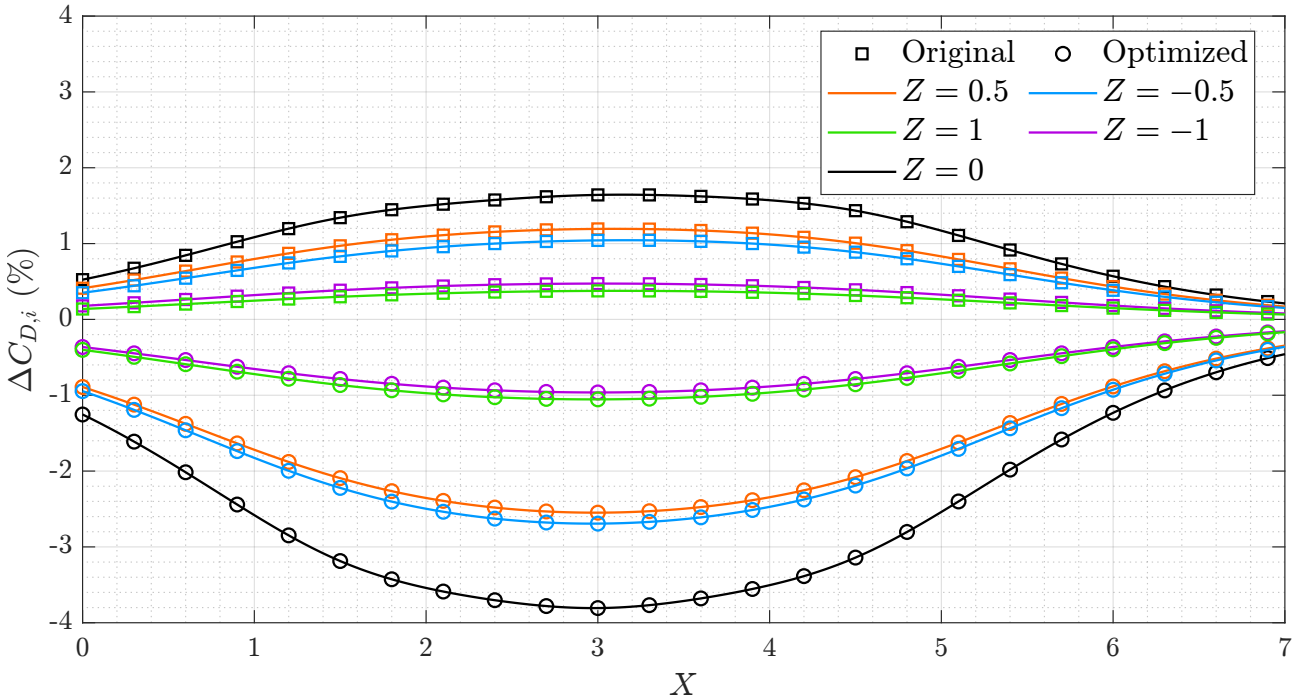


(b) Optimized wing

Figure 19: Comparison of the effect of relative formation position on the induced drag change $\Delta C_{D,i}$ (%) of the Lockheed Martin/Boeing F-22A Raptor trailing aircraft: original wing shape (a) and optimized wing design (b). $\Delta C_{D,i}$ is defined as the difference between the induced drag coefficient of the trailer in formation and the induced drag coefficient of the same aircraft in solo flight with the original wing geometry. Contour slices indicate regions where $\Delta C_{D,i} < 0$, i.e. where the trailer experiences a drag reduction. The floor projection shows the filled contour in the X-Y plane at the optimal Z-position. The black dots represent the LHS studied point, while the red one the reference configuration.

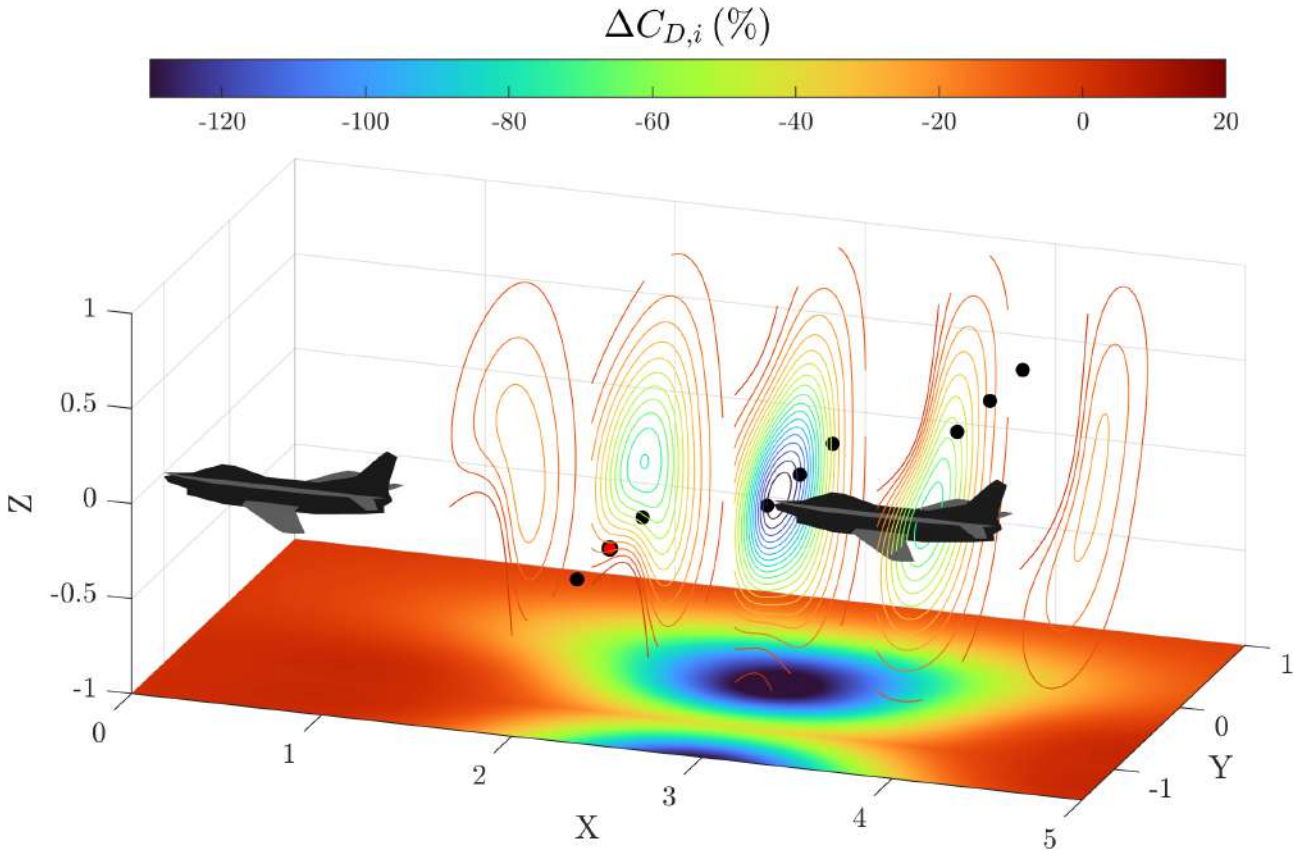


(a) Trailer aircraft

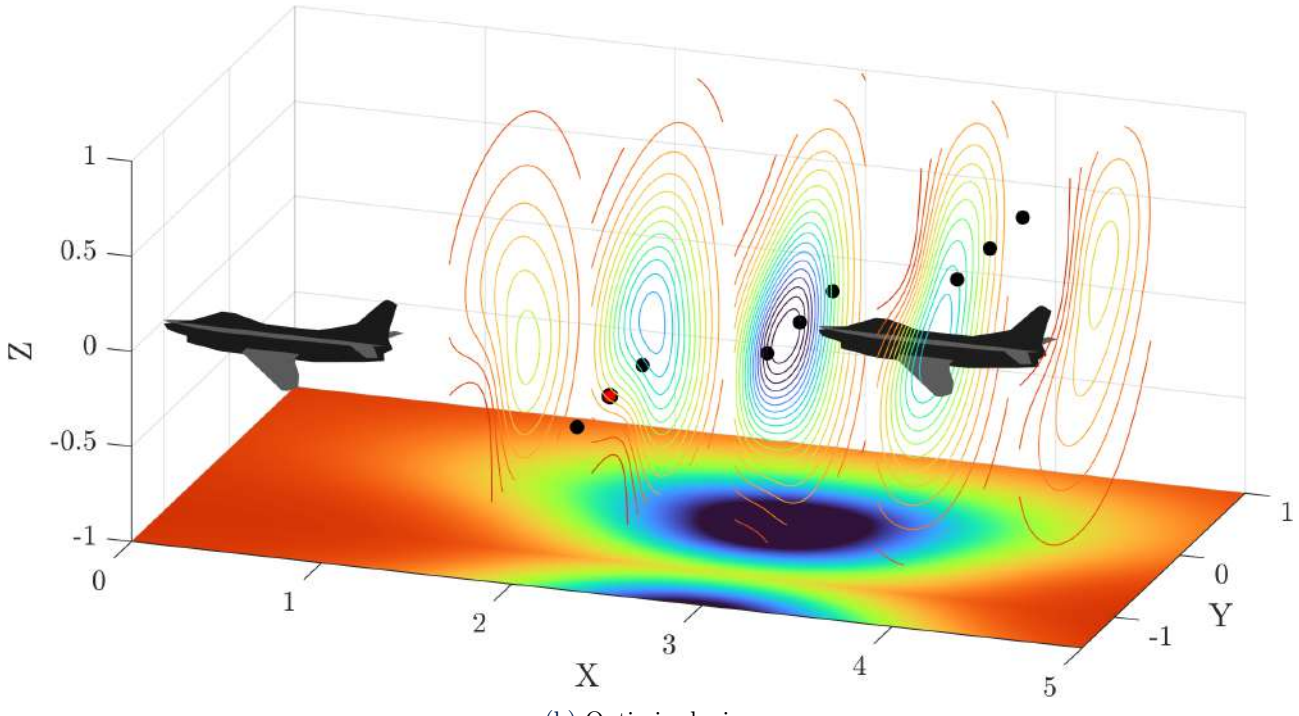


(b) Leader aircraft

Figure 20: Induced-drag reduction trends along X for the Lockheed Martin/Boeing F-22A Raptor trailing (a) and leader (b) aircraft. Original configurations are shown with square markers, and optimized configurations with circles. The spanwise offset is fixed at the best lateral separation of the original wing (Y_{best}^{OG}), while vertical positions are varied at $Z = 0, \pm 0.5, \pm 1$ (color-coded in the legend).

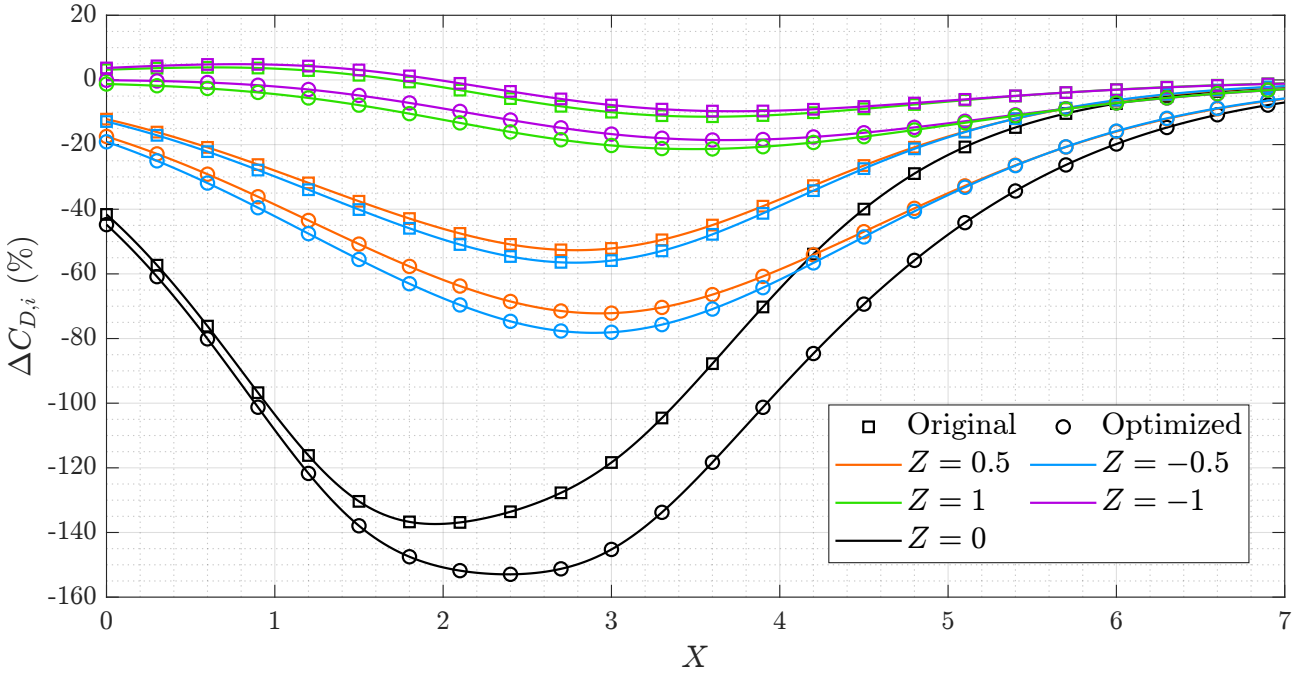


(a) Original wing

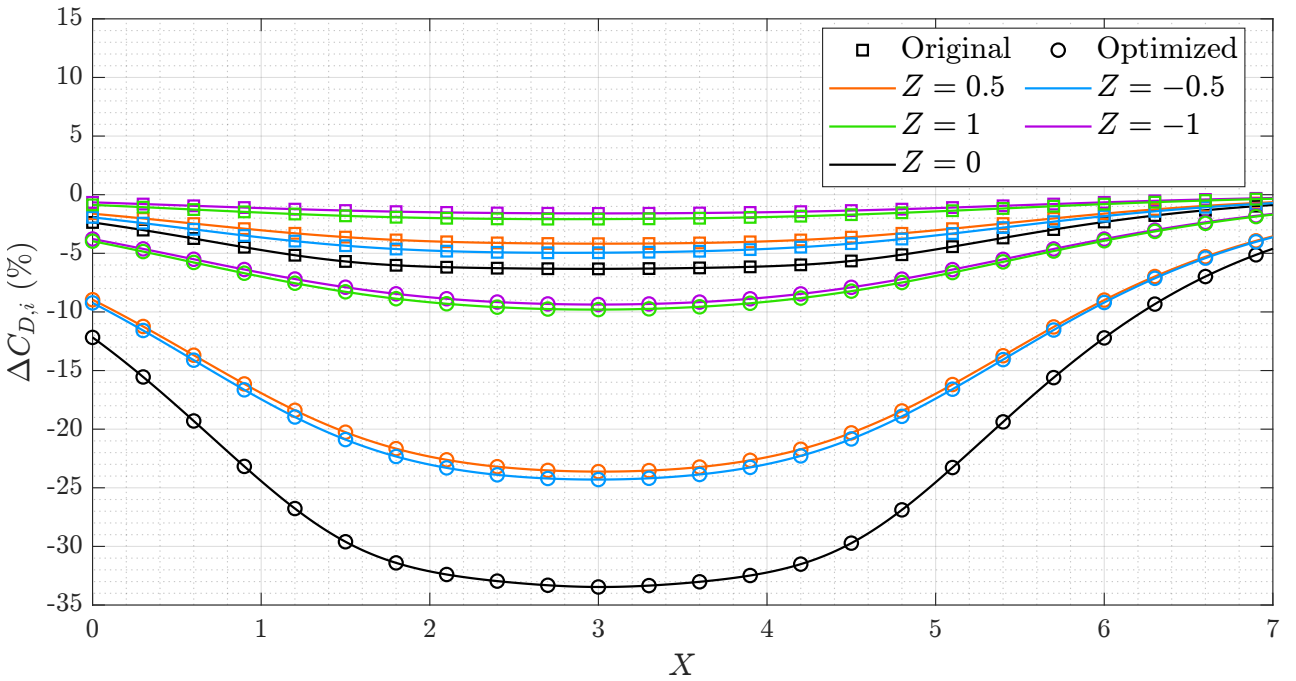


(b) Optimized wing

Figure 21: Comparison of the effect of relative formation position on the induced drag change $\Delta C_{D,i}$ (%) of the Fiat G.91R trailing aircraft: original wing shape (a) and optimized wing design (b). $\Delta C_{D,i}$ is defined as the difference between the induced drag coefficient of the trailer in formation and the induced drag coefficient of the same aircraft in solo flight with the original wing geometry. Contour slices indicate regions where $\Delta C_{D,i} < 0$, i.e. where the trailer experiences a drag reduction. The floor projection shows the filled contour in the X-Y plane at the optimal Z-position. The black dots represent the LHS studied point, while the red one the reference configuration.



(a) Trailer aircraft



(b) Leader aircraft

Figure 22: Induced-drag reduction trends along X for the Fiat G.91R trailing (a) and leader (b) aircraft. Original configurations are shown with square markers, and optimized configurations with circles. The spanwise offset is fixed at the best lateral separation of the original wing (Y_{best}^{OG}), while vertical positions are varied at $Z = 0, \pm 0.5, \pm 1$ (color-coded in the legend).

7. Aerial refueling and range extension

Aerial refueling (also referred to as air refueling, in-flight refueling, air-to-air refueling, and tanking) allows a tanker aircraft to transfer fuel to a fighter (or other combat aircraft) while both aircraft are in flight. This practice dramatically extends a fighter's operational range and loiter time, since modern fighters often cannot take off both fully fueled and with a full weapon loadout. By refueling en route, a fighter can carry heavier payloads or stay on station longer without landing. In strategic terms, this is known as force extension (nearly unlimited range) and force multiplication (longer loiter, more weapons).

On July 6, 1951, the first combat air refueling of fighter-type aircraft took place over Korea. Three USAF RF-80A reconnaissance planes departed from Taegu Air Base with modified tip-tanks and rendezvoused with a KB-29 tanker off the coast of Wonsan, North Korea. Through in-flight refueling, the RF-80s effectively doubled their range, allowing the reconnaissance aircraft to capture crucial photographic intelligence of strategic targets deep within North Korean territory [16]. The USAF never bought another fighter aircraft without in-flight refueling capability.

For context, a tanker like the Boeing KC-135R Stratotanker has roughly 18× the fuel capacity of the McDonnell Douglas F/A-18C Hornet. Due to safety and cost, this technique is used only in the military domain.



Figure 23: A NASA SR-71 refuels from an Edwards Air Force Base KC-135 during the first flight of the NASA/Rocketdyne/Lockheed Martin Linear Aerospike SR-71 Experiment (LASRE). The flight took place October 31 at NASA's Dryden Flight Research Center. Image Credit: NASA/Lori Losey.

7.1. Fighter refueling trim study

Similar to the procedure used to identify trim conditions for formation flying (see Section 4.4), an equivalent study was carried out to obtain the trim conditions while refuelling. The cruise flight condition was kept identical to the rest of the work: $M = 0.56$[2] at a flight altitude of 25000 ft. The trim condition of the KC-135 tanker aircraft was calculated prior to the start of the study to obtain an initial guess of its trim attitude during refueling. For a realistic assessment, the procedure was applied

[2] According to several sources, this is probably faster than the speed at which in-flight refueling procedures are normally carried out, but the results are not expected to be sensitive to modest variations in Mach.

only to the baseline and aerodynamically optimized wings of the F/A-18 and F-22. The Fiat G.91R was not included in the study because it could not be refueled in flight.

Relative positions between tanker and fighter were selected using representative photographs and standard refueling geometry as references (i.e., nose position relative to the drogue/basket and to the tanker longitudinal axis).

7.1.1 Limitations and caveats

The results reported here are intentionally qualitative rather than quantitative.
AVL is a vortex lattice tool that neglects viscous effects, boundary-layer separation and detailed thickness effects, and it assumes quasi-steady flow and thin lifting surfaces. These assumptions limit the validity of the AVL results at large angles of attack and in strong wake-interaction conditions produced by the tanker. Consequently, some trim solutions produced by AVL fall outside the strict validity range of the model and should not be interpreted as definitive numeric values.
Practical implications of these limitations include:

- The tanker wake is a complex, viscous flow, and its effects on the fighter's boundary layer at near-stall attitudes might be significant; the local pressure distribution is only approximated by the VLM approach.
- The relative-position reference taken from images introduces geometric uncertainty; small positional shifts can substantially alter the local upwash/downwash experienced by the fighter.

In summary, AVL provided a useful first indication that optimized wings alter trimmed conditions during refueling and that wake position significantly affects the required angle of attack. However, because of modeling assumptions, these results should be treated as qualitative guidance and followed up with higher-fidelity analyses before drawing firm operational conclusions.

7.1.2 Refueling results

Table 10 reports the trimmed angle of attack in solo cruise (α_{trim}^{solo}), in refueling configuration with the original wing geometry ($\alpha_{trim}^{og,fuel}$), and with the optimized wing geometry ($\alpha_{trim}^{opt,fuel}$). For completeness, the relative variation in induced drag coefficient ($\Delta C_{D,i}$) with respect to the solo-flight baseline is also shown. These values are intended to illustrate the qualitative differences introduced by the optimized wing during refuelling. The Figure 24 shows the AVL representation of the Lockheed Martin/Boeing F-22A Raptor refueling behind the Boeing KC-135R Stratotanker.

Aircraft type	$\boldsymbol{\alpha_{trim}^{solo}}$	$\boldsymbol{\alpha_{trim}^{og,fuel}}$	$\boldsymbol{\alpha_{trim}^{opt,fuel}}$	$\boldsymbol{\Delta C_{D,i}^{og,fuel}}$	$\boldsymbol{\Delta C_{D,i}^{opt,fuel}}$
McDonnell Douglas F/A-18C Hornet	5.9°	32.8°	33.3°	+364%	+268.5%
Lockheed Martin/Boeing F-22A Raptor	9.1°	25.9°	26.2°	+262.9%	+222.3%

Table 10: Trimmed angle of attack for the fighters tested: solo flight (α_{trim}^{solo}), refuelling in reference configuration ($\alpha_{trim}^{og,fuel}$), and refuelling in optimized configuration ($\alpha_{trim}^{opt,fuel}$). Percentage variations in induced drag coefficient $\Delta C_{D,i}$ are reported relative to the original solo-flight configuration.

From Table 10 it is evident that all fighters require a substantially higher trimmed angle of attack during refueling compared with solo cruise. This behavior is a direct consequence of operating in the downwash field of the tanker wake, which forces the aircraft to pitch up in order to maintain lift equilibrium. When comparing original and optimized wings, the difference in required α_{trim} is modest, confirming that the optimization primarily affects overall efficiency rather than the basic trim requirement in the disturbed flow. On the other hand, the induced drag penalty ($\Delta C_{D,i}$) shows high variation: optimization reduces the relative increase in $C_{D,i}$ during refueling, indicating that the optimized wing mitigates part of the aerodynamic cost of flying in the tanker wake.

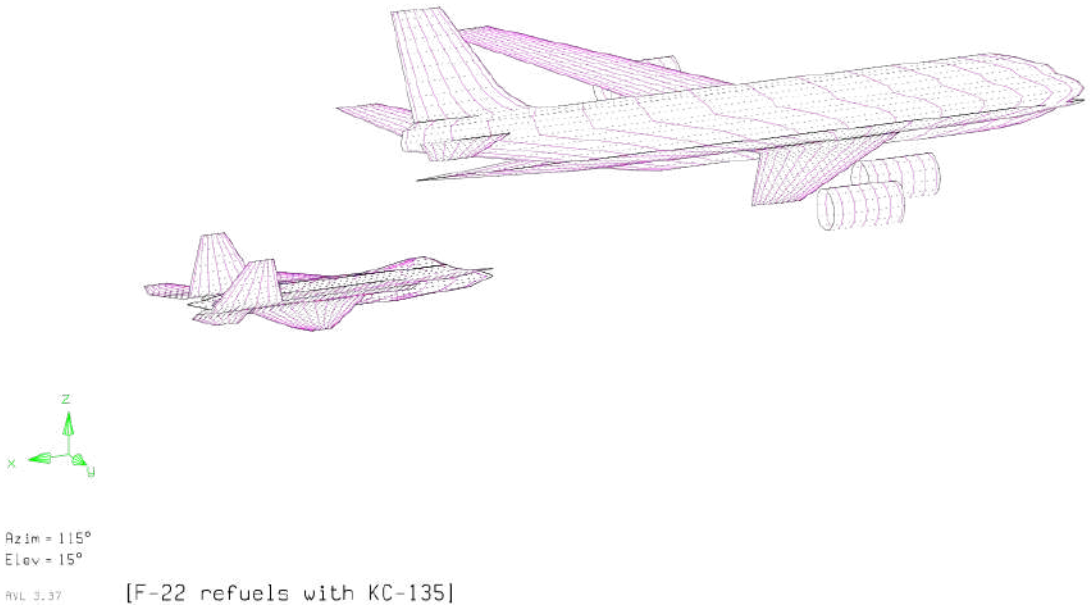


Figure 24: AVL representation of the Lockheed Martin/Boeing F-22A Raptor refueling behind the Boeing KC-135R Stratotanker. The relative position between tanker and fighter was selected using representative photographs.

7.2. Range extension via formation flight

To quantify the range extension enabled by formation flight (and aerial refueling), the well-known Breguet range equation was applied, suitably adapted to highlight aerodynamic efficiency gains. Considering steady, level flight, the aircraft weight change is governed by the fuel mass flow:

$$-\frac{dW}{dt} = g\dot{m}_f = \frac{T}{I_{sp}} \tag{14}$$

with thrust T equal to drag D, and lift L balancing weight W. Thus:

$$\frac{dW}{dt} = -\frac{W}{(L/D)\, I_{sp}}, \quad \frac{dW}{W} = -\frac{dt}{(L/D)\, I_{sp}}. \tag{15}$$

Integrating over the flight duration t_f and multiplying by cruise velocity V gives the Breguet range equation:

$$R = V\, t_f = V\, \frac{L}{D}\, I_{sp}\, \ln\left(\frac{W_i}{W_f}\right) \tag{16}$$

An alternative form, using thrust-specific fuel consumption (TSFC), is:

$$R = \frac{V}{g}\, \frac{1}{\mathrm{TSFC}}\, \left(\frac{L}{D}\right)\, \ln\left(\frac{W_i}{W_f}\right) \tag{17}$$

These expressions demonstrate that range increases with better aerodynamic efficiency (L/D) and greater fuel fraction (W_i/W_f).

To estimate the range using the Breguet equation, it was necessary to determine the parasitic drag term C_{D0}, since AVL only outputs the induced drag contribution. To do this, two approximations were made: the drag polar is assumed to be parabolic (valid for the symmetric profiles used by both the McDonnell Douglas F/A-18C Hornet and Lockheed Martin/Boeing F-22A Raptor), and the calculated trim point is assumed to correspond to the condition of maximum aerodynamic efficiency. Of course, the latter is a tentative guess, in that the best range (or design) Mach and altitude are likely higher

than Mach 0.56 at 25000 ft, especially for a supercruise-capable aircraft like the F-22. However, this approximation is consistent with the low-fidelity nature of the present study.
Under these assumptions, C_{D0} can be derived from the value of C_L obtained from AVL by the following relations:

$$\begin{cases} C_D = C_{D0} + C_{D,i} = C_{D0} + k\,C_L^2 & \text{(18a)} \\ E_{max} = \dfrac{dE}{dC_L} = \dfrac{C_{D0} + k\,C_L^2 - 2\,k\,C_L^2}{\left(C_{D0} + k\,C_L^2\right)^2} = 0 & \text{(18b)} \\ C_L\big|_{E_{max}} = \sqrt{\dfrac{C_{D0}}{k}} & \text{(18c)} \end{cases}$$

From Equation 18c an explicit expression for C_{D0} can be written as:

$$C_{D0} = k\,C_L^2\big|_{E_{max}} \tag{19}$$

At maximum aerodynamic efficiency, the induced drag coefficient is equal to $k\,C_L^2\big|_{E_{max}}$, hence:

$$C_{D0} = k\,C_L^2\big|_{E_{max}} = C_{D,i} \tag{20}$$

Thus, under the stated assumptions, the parasitic drag coefficient C_{D0} can be identified numerically with the induced drag contribution returned by AVL at the assumed maximum efficiency trim point. Since AVL provides the Oswald efficiency factor e, a reverse check was performed on the solo-flight case to verify the consistency of the two assumptions. The lift coefficients computed from the maximum-efficiency hypothesis differ from the AVL-trimmed values as follows: for the F/A-18C, +1.6% for the original wing and +10.8% for the optimized wing; for the F-22, −1% for the original wing and −2.5% for the optimized wing. Small deviations (1–3%) are consistent with the linearized assumptions inherent to AVL, while the larger 10% discrepancy for the optimized F/A-18C suggests a violation of one or more assumptions. Despite this, the AVL results are reported here for completeness. To establish definitive values, the cases with large discrepancies should be re-evaluated with higher-fidelity tools.

Finally, to calculate range, the thrust-specific fuel consumption (TSFC) must be specified. The analysis excludes any use of the afterburner, therefore the dry TSFC values are adopted.
The McDonnell Douglas F/A-18C Hornet is a twin-engine aircraft powered by two General Electric F404-GE-400 engines: the dry TSFC used is $\text{TSFC}_{\text{F/A-18}} = 24.1$ g/(kN · s) [2].
The Lockheed Martin/Boeing F-22A Raptor is also twin-engine, equipped with two Pratt & Whitney F119-PW-100 engines: the dry TSFC used is $\text{TSFC}_{\text{F-22}} = 17.3$ g/(kN · s) [2].
For a realistic range estimate a safety fuel reserve of 20% of the fuel capacity is assumed. This reserve supports several critical contingencies: approach and landing attempts, go-around capability, diversion to an alternate airfield and unforeseen emergency situations.

Figure 25 and 26 present a comparison of the ferry ranges achievable by the two aircraft, starting from their respective deployment bases, across all the studied configurations. For the F-22, Andersen Air Force Base (Guam) was selected as the starting point. The Andersen Air Force Base regularly operates F-22A aircraft among its fleet. For the F/A-18, a European base was chosen, specifically the Zaragoza Air Base in Spain. This choice was made because all U.S. F/A-18s belong to the Navy rather than the Air Force, while the Spanish Air Force employs the type in its land-based operations. Zaragoza Air Base is home to EF-18s operated by the *Ala 15* of the *Ejército del Aire y del Espacio*.
The images also indicate, with red stars, the Main Operating Bases (MOBs) within reach of the aircraft. In US military terminology, a Main Operating Base refers to a permanently manned and well-protected installation designed to support permanently deployed forces, with secure and sustained access via sea and/or air.
As shown, formation flying significantly increases the range[3] of both aircraft. Moreover, the optimized wing geometry further extends the distance covered, enabling the aircraft to reach MOBs that would otherwise be inaccessible without aerial refueling. This highlights the combined benefits of aerodynamic optimization and formation flight in enhancing operational reach and mission flexibility.

[3]The formation ranges presented in the figures correspond to the distance achievable by the trailing aircraft.

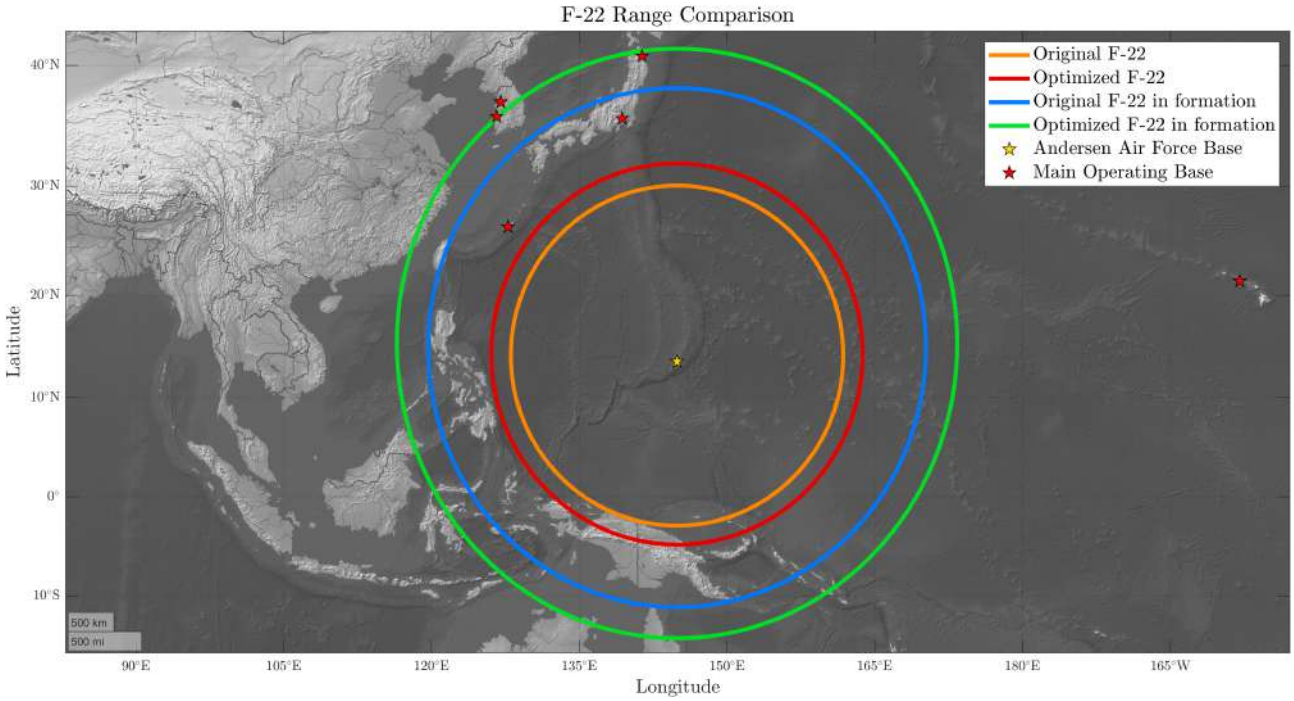


Figure 25: Ferry range comparison for the Lockheed Martin/Boeing F-22A Raptor. The F-22s are assumed deployed from Andersen Air Force Base (Guam); the map inset shows Andersen Air Force Base and the Main Operating Base in the area used for mission planning. Range contours are computed with the Breguet equation at cruise conditions ($M = 0.56$ at 25000 ft), using dry TSFC values and a 20% fuel reserve for safe landing. Formation denotes the range when the trailer benefits from wake-induced drag reduction.

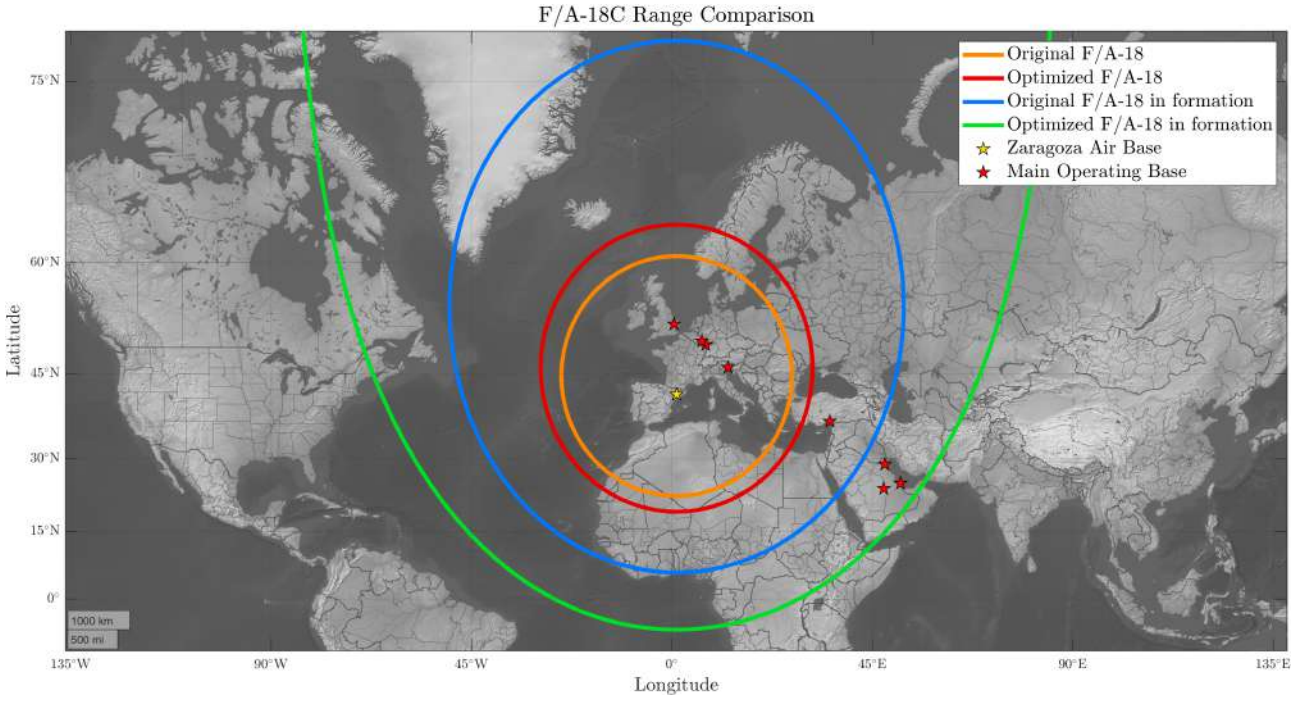


Figure 26: Ferry range comparison for the McDonnell Douglas F/A-18C Hornet. The F/A-18Cs are assumed deployed from Zaragoza Air Base (Spain); the map inset shows Zaragoza Air Base and the Main Operating Base in the area used for mission planning. Range contours are computed with the Breguet equation at cruise conditions ($M = 0.56$ at 25000 ft), using dry TSFC values and a 20% fuel reserve for safe landing. Formation denotes the range when the trailer benefits from wake-induced drag reduction.

8. Conclusions

This project investigated the aerodynamic benefits of formation flight and wing planform optimization for fighter aircraft across different generations, using the vortex lattice method in AVL. Starting from reconstructed geometries of representative aircraft, a comparative analysis confirmed that formation flight generally reduces induced drag for trailing aircraft, with the magnitude of improvement depending strongly on wing design. Using AVL simulations and optimization algorithms, wing planforms for the F/A-18C Hornet, F-22A Raptor, and Fiat G.91R were refined to maximize the lift-to-induced-drag ratio under both cruise and formation conditions.

Wing optimization via particle swarm algorithms produced non-intuitive configurations, often characterized by forward sweep and dihedral adjustments. These modifications significantly improved efficiency at trimmed cruise, although in some cases they introduced potential aeroelastic or stability issues that would need to be resolved before operational implementation. The analysis suggests that purely aerodynamic optimization, while effective in reducing induced drag, must be coupled with structural and control considerations before being applied to operational aircraft.

The influence of relative positioning in formation was systematically examined using Latin hypercube sampling and Kriging interpolation. Results demonstrated that precise tuning of aircraft separation is essential to fully exploit aerodynamic synergies.

Refueling simulations showed that disturbed flow in the wake of the KC-135R Stratotanker imposes substantial trim penalties. Nevertheless, optimized planforms helped mitigate the induced drag increase in these scenarios. Although AVL vortex-lattice approach limits the fidelity of this type of interactions, the results provide valuable first-order insights into how wing optimization can support both formation efficiency and aerial refueling operations. Application of the Breguet range equation confirmed that two-aircraft formations can substantially extend ferry range, with further gains achievable through optimized geometries. This underscores the potential of formation strategies to reduce reliance on tanker support and enhance operational flexibility.

In conclusion, this study demonstrates that coupling formation flight with aerodynamic optimization represents a promising pathway to improve endurance and mission effectiveness in fighter aircraft. The results establish a foundation for more detailed studies employing higher-fidelity CFD or experimental testing, particularly to quantify viscous and unsteady wake effects beyond the capabilities of vortex-lattice models. Further investigations should address structural constraints, flight-control requirements for optimized geometries, and the potential benefits of larger multi-aircraft formations.

References

[1] M. Jake Vachon, Ronald Ray, Kevin Walsh, and Kimberly Ennix. *F/A-18 Aircraft Performance Benefits Measured During the Autonomous Formation Flight Project*. doi: 10.2514/6.2002-4491. URL `https://arc.aiaa.org/doi/abs/10.2514/6.2002-4491`.

[2] Angelos T. Kottas, Michail N. Bozoudis, and Michael A. Madas. Turbofan aero-engine efficiency evaluation: An integrated approach using VSBM two-stage network DEA. *Omega*, 92:102167, 2020. ISSN 0305-0483. doi: https://doi.org/10.1016/j.omega.2019.102167. URL `https://www.sciencedirect.com/science/article/pii/S0305048319306486`.

[3] Jeffrey W. Hamstra and Brent N. McCallum. *Tactical Aircraft Aerodynamic Integration*. John Wiley & Sons, Ltd, 2010. ISBN 9780470686652. doi: https://doi.org/10.1002/9780470686652.eae490. URL `https://onlinelibrary.wiley.com/doi/abs/10.1002/9780470686652.eae490`.

[4] Brian Lockett. Flying Aircraft Carriers of the USAF: Project Tom-Tom. https://www.air-and-space.com/tomtom.htm. Accessed: 2025-09-03.

[5] P. B. S. Lissaman and Carl A. Shollenberger. Formation Flight of Birds. *Science*, 168(3934):1003–1005, 1970. doi: 10.1126/science.168.3934.1003. URL https://www.science.org/doi/abs/10.1126/science.168.3934.1003.

[6] Ashok Gopalarathnam. *Aerodynamic Benefit of Aircraft Formation Flight*. John Wiley & Sons, Ltd, 2010. ISBN 9780470686652. doi: https://doi.org/10.1002/9780470686652.eae023. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/9780470686652.eae023.

[7] Scott Lowther. *US Supersonic Bomber Projects*, volume 1. Mortons Media Group Ltd., 2022. ISBN 191170303X.

[8] Afshin Rahimi. AERO_Console repository on GitHub. https://github.com/drarahimi/AERO_Console, 2022. Accessed: 2025-09-05.

[9] Mark Drela and Harold Youngren. MIT AVL User Primer - AVL 3.36. https://web.mit.edu/drela/Public/web/avl/AVL_User_Primer.pdf, 2017. Accessed: 2025-09-03.

[10] Michael S. Selig. The Incomplete Guide to Airfoil Usage (UIUC Airfoil Data Site). https://m-selig.ae.illinois.edu/ads/aircraft.html, 2025. Last updated July 6, 2025. Accessed: 2025-09-01.

[11] Robert A. McDonald. Wing Airfoil Selection from the OpenVSP Ground School series of training videos. https://vspu.larc.nasa.gov/training-content/chapter-1-vspfundamentals/wings/wing-airfoil-selection/, 2022. Accessed: 2025-09-08.

[12] Airfoil Tools website. http://airfoiltools.com/index. Accessed: 2025-09-08.

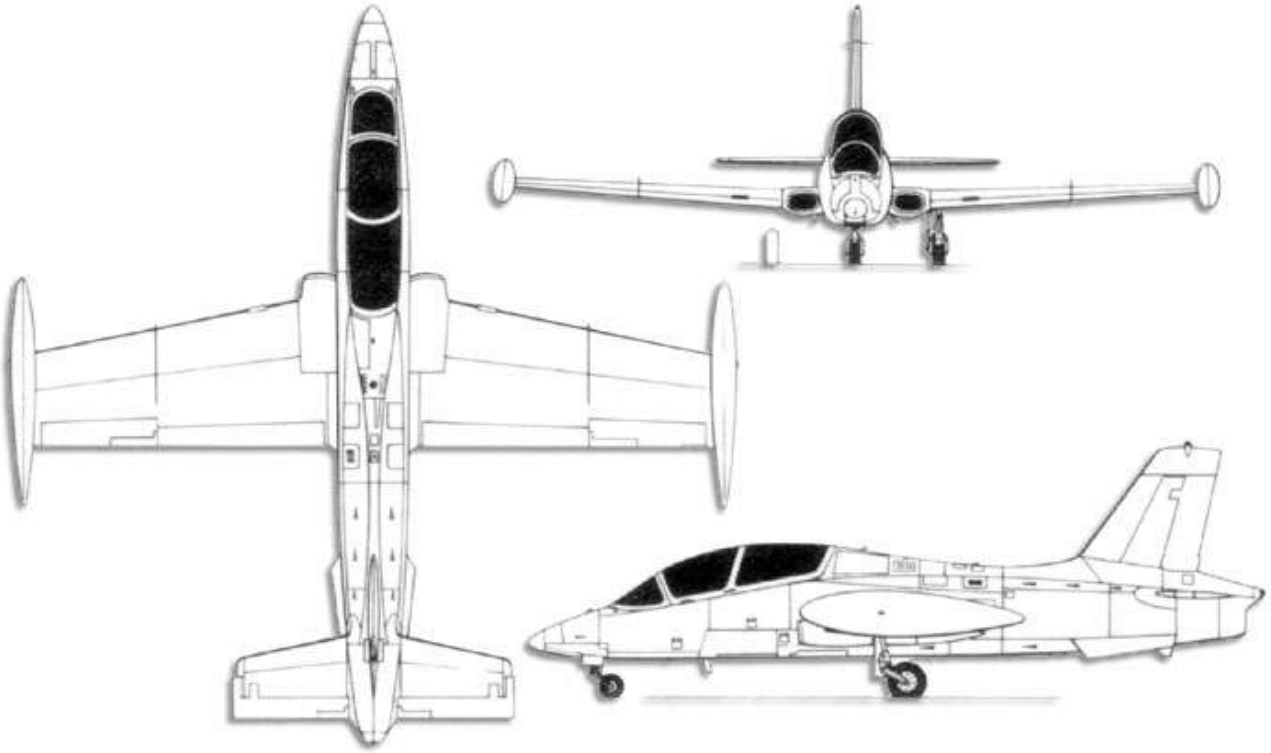
[13] Jeffrey E. Bauer, Robert Clarke, and John J. Burken. Flight Test of the X-29A at High Angle of Attack: Flight Dynamics and Controls. NASA Technical Paper NASA TP-3537, NASA Dryden Flight Research Center, National Aeronautics and Space Administration, Edwards, California, February 1995. Report dated February 1995. Available from the NASA Center for Aerospace Information.

[14] S. Andrew Ning, Tristan C. Flanzer, and Ilan M. Kroo. Aerodynamic Performance of Extended Formation Flight. *Journal of Aircraft*, 48(3):855–865, 2011. doi: 10.2514/1.C031046. URL https://doi.org/10.2514/1.C031046.

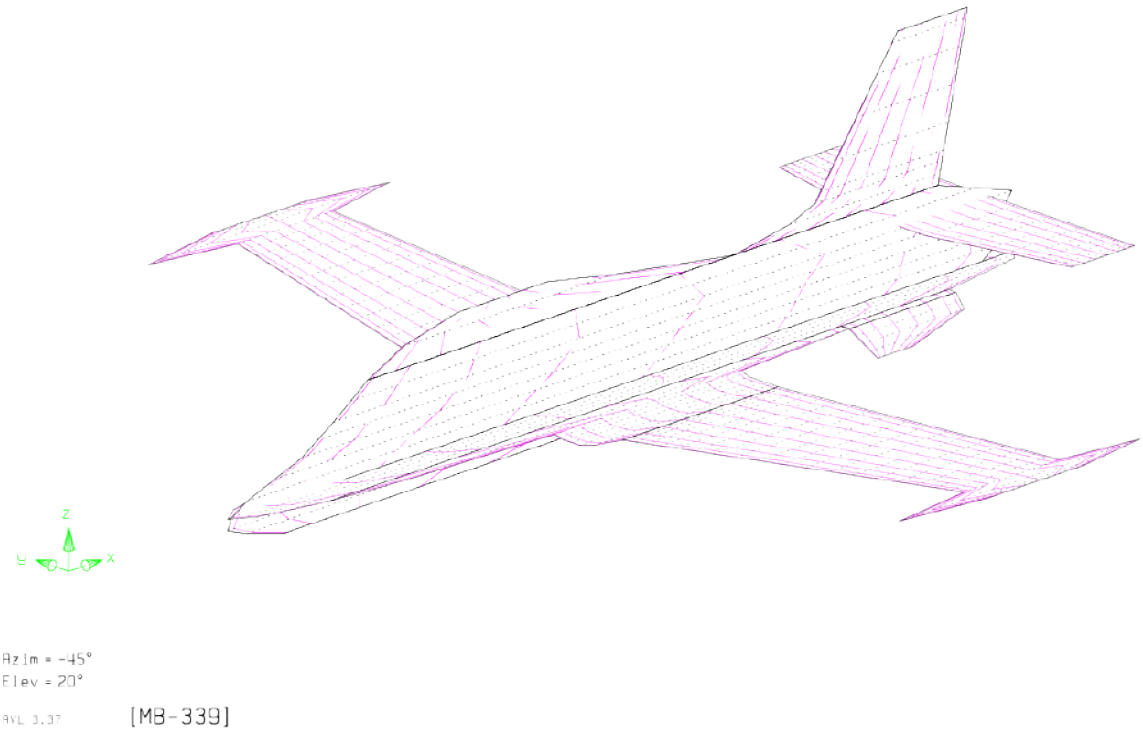
[15] L. L. M. Veldhuis, M. Voskuijl, and B. Fransen. Formation Flight — Fine-tuning of Theoretical Performance Prediction. In *51st AIAA Aerospace Sciences Meeting, including the New Horizons Forum and Aerospace Exposition*, number AIAA 2013-0961, pages 1–17, Grapevine, Texas, USA, 2013. American Institute of Aeronautics and Astronautics (AIAA).

[16] Mark L. Morgan. History of Aerial Refueling: Fueling the Fighters. Air Mobility Command History Office, U.S. Air Force, April 15 2009. URL https://www.amc.af.mil/News/Article-Display/Article/147244/history-of-aerial-refueling-fueling-the-fighters.

A. Aircraft geometries

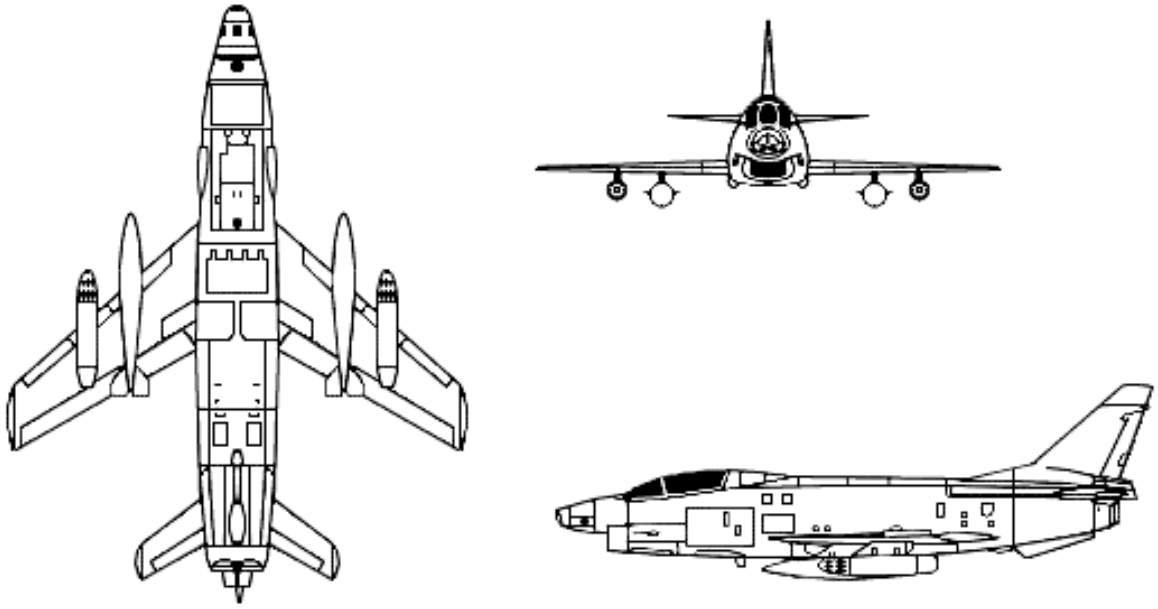


(a) Reference three-view drawing.

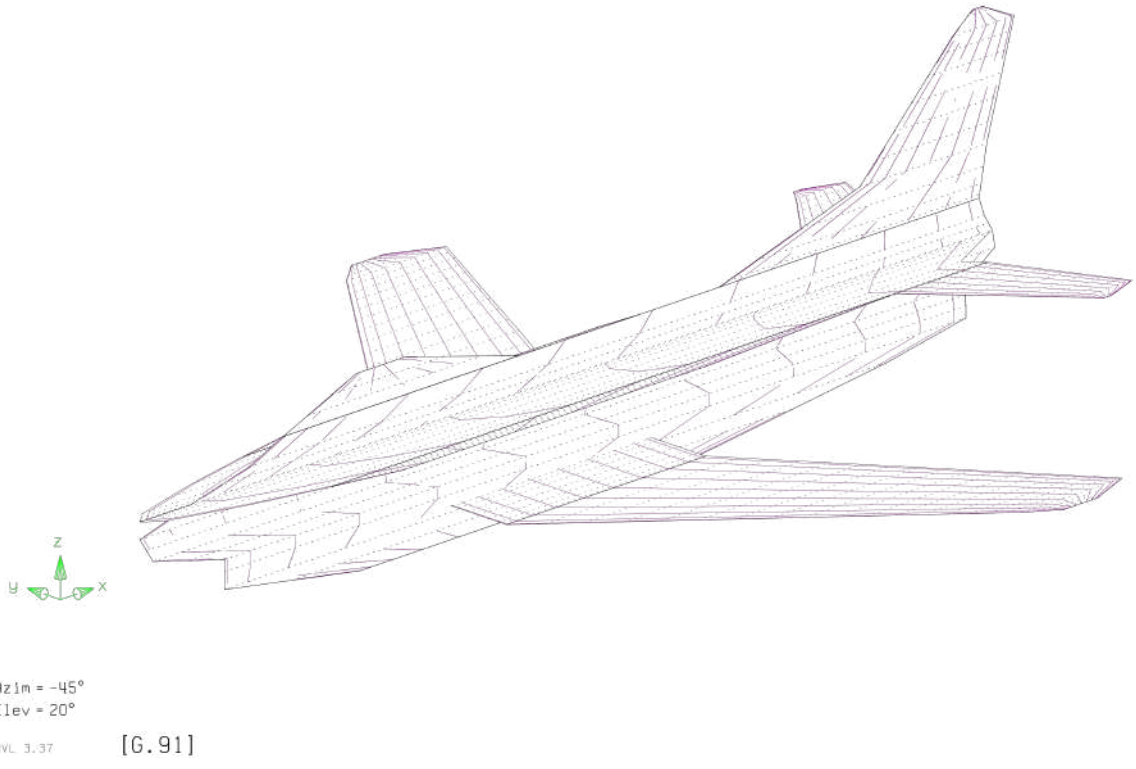


(b) Visualization of the geometry in AVL.

Figure 27: Aermacchi MB-339A.

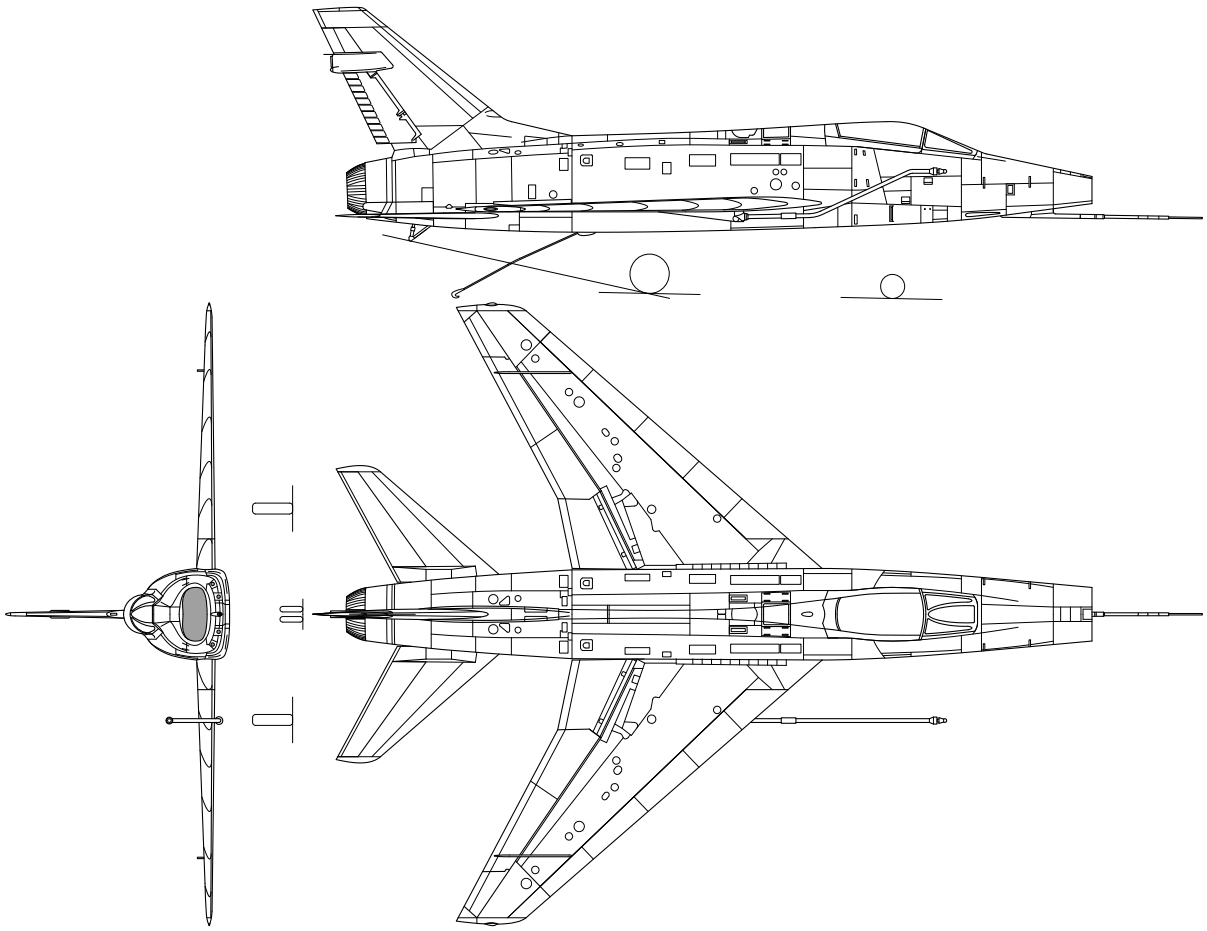


(a) Reference three-view drawing.

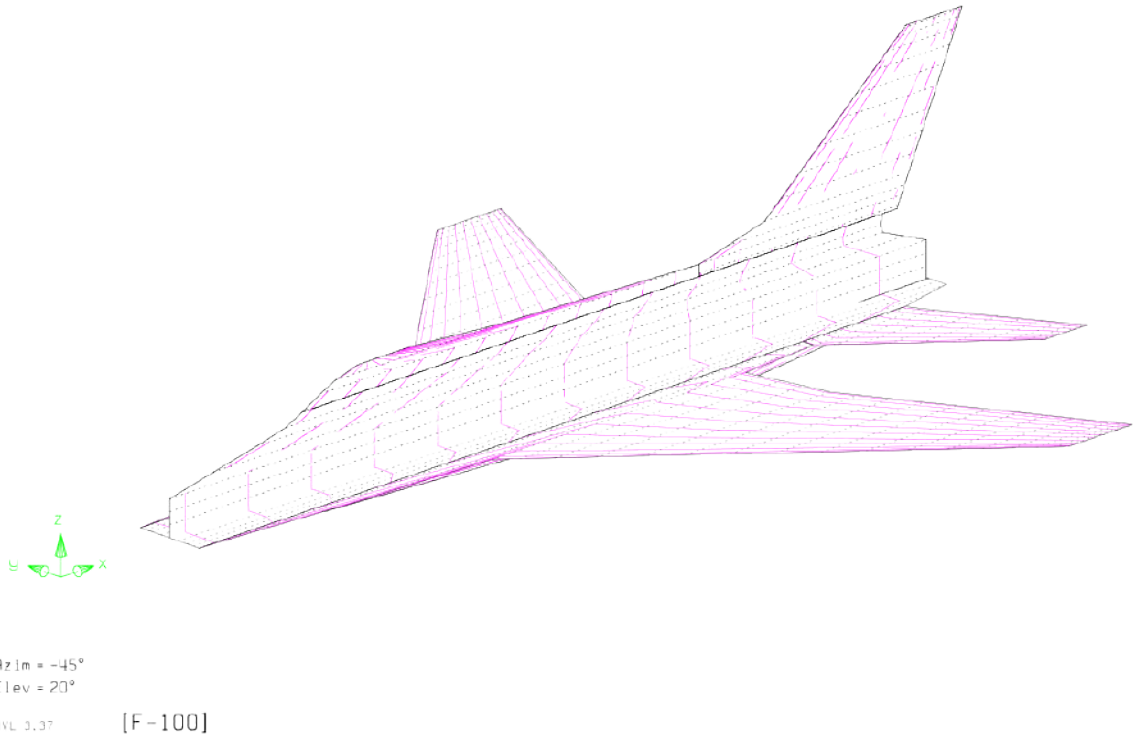


(b) Visualization of the geometry in AVL.

Figure 28: Fiat G.91R.

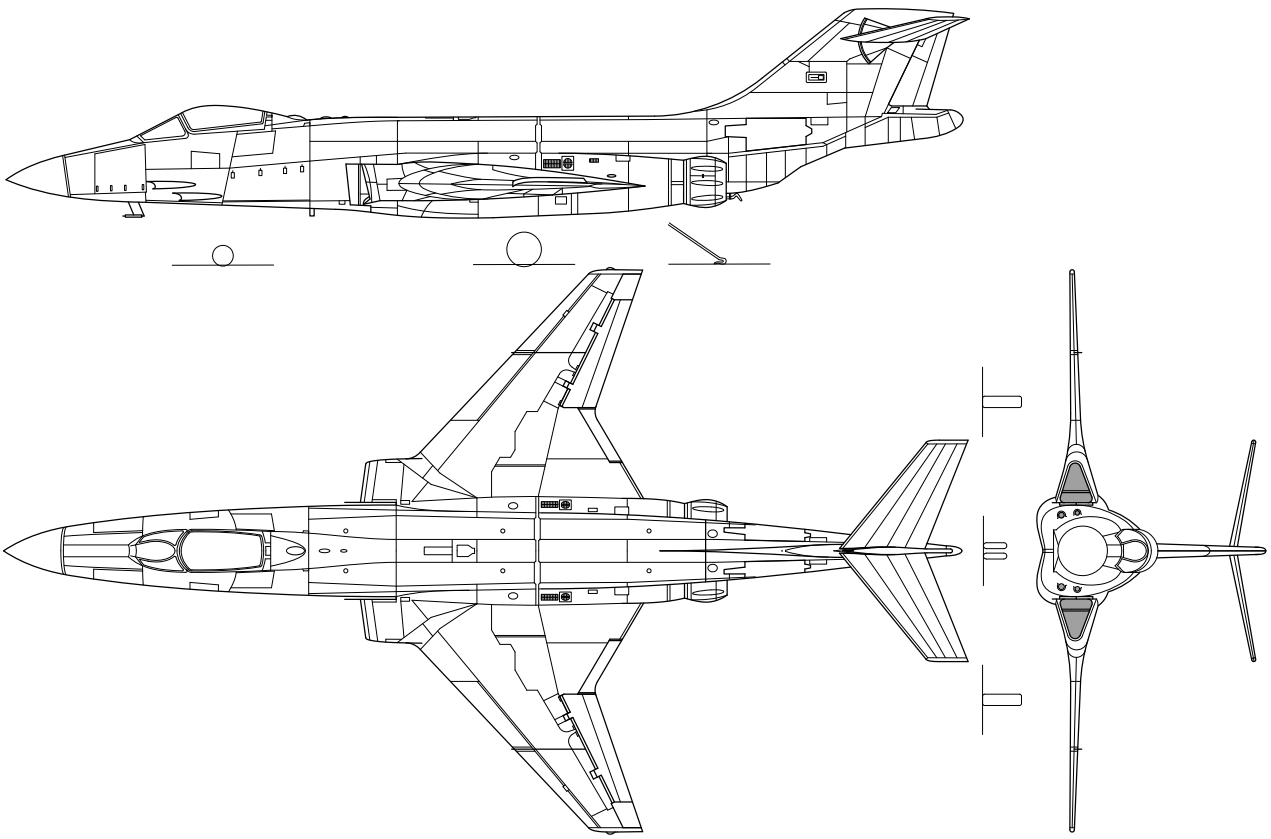


(a) Reference three-view drawing. Image by Kaboldy, CC BY-SA 3.0, via Wikimedia Commons.

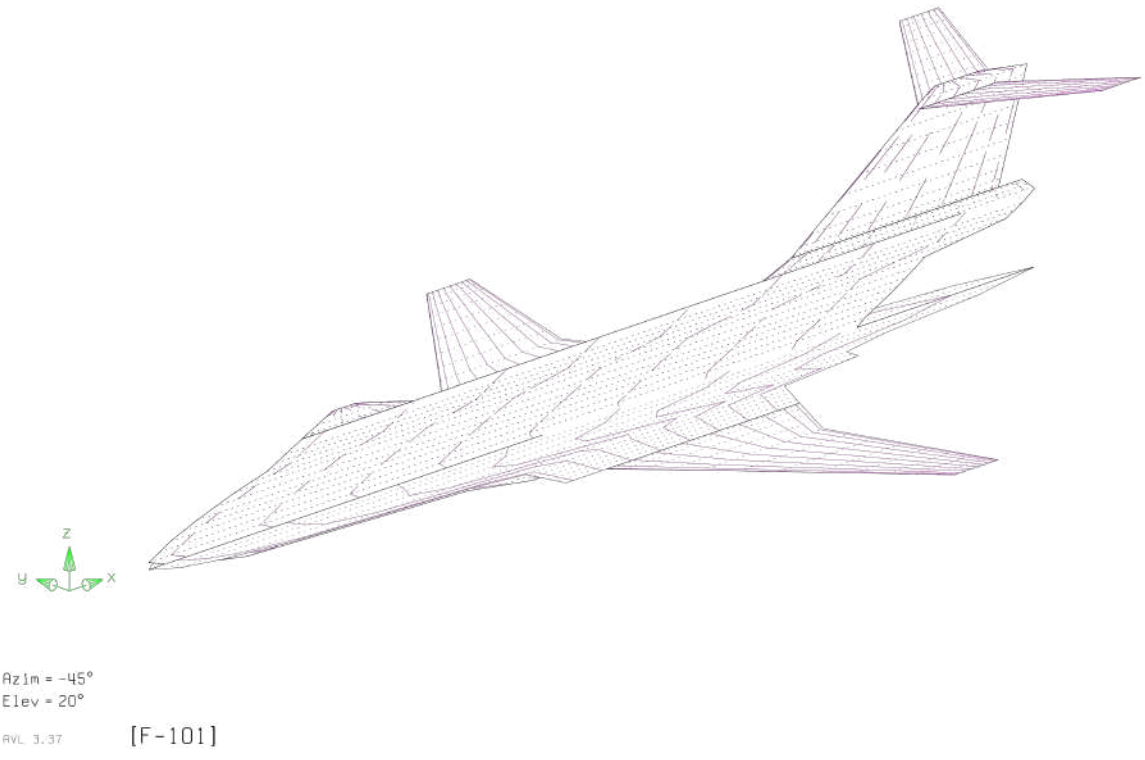


(b) Visualization of the geometry in AVL.

Figure 29: North American F-100 Super Sabre.

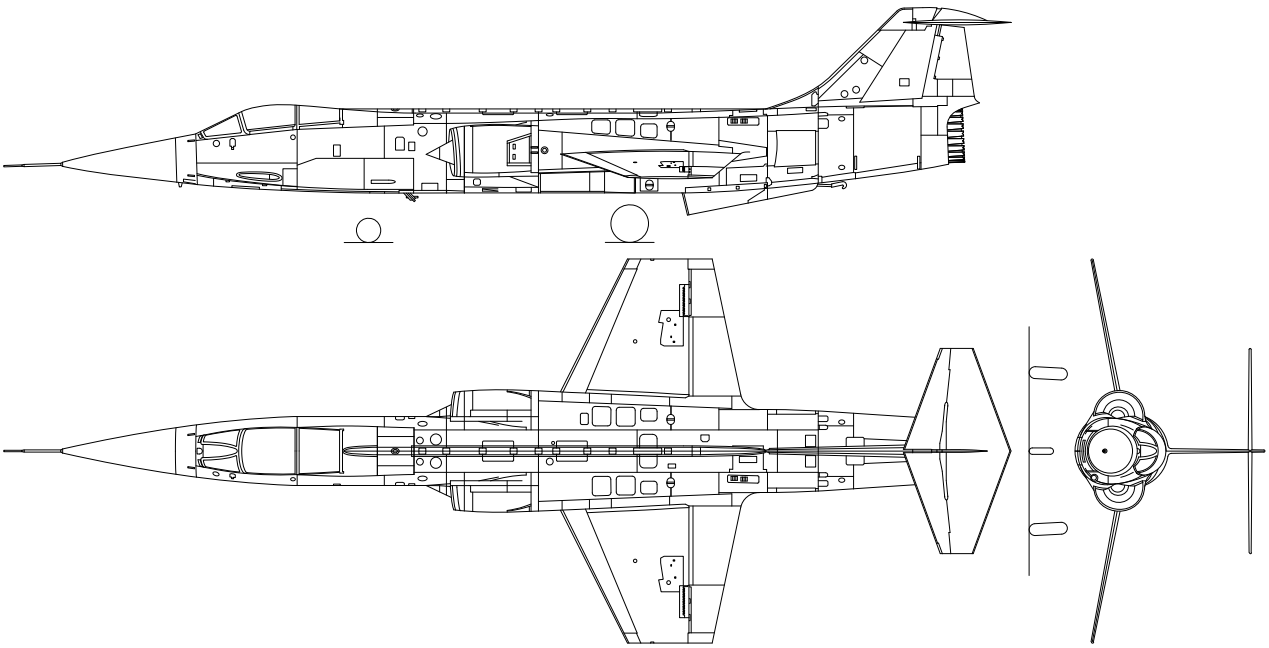


(a) Reference three-view drawing. Image by Kaboldy, CC BY-SA 3.0, via Wikimedia Commons.

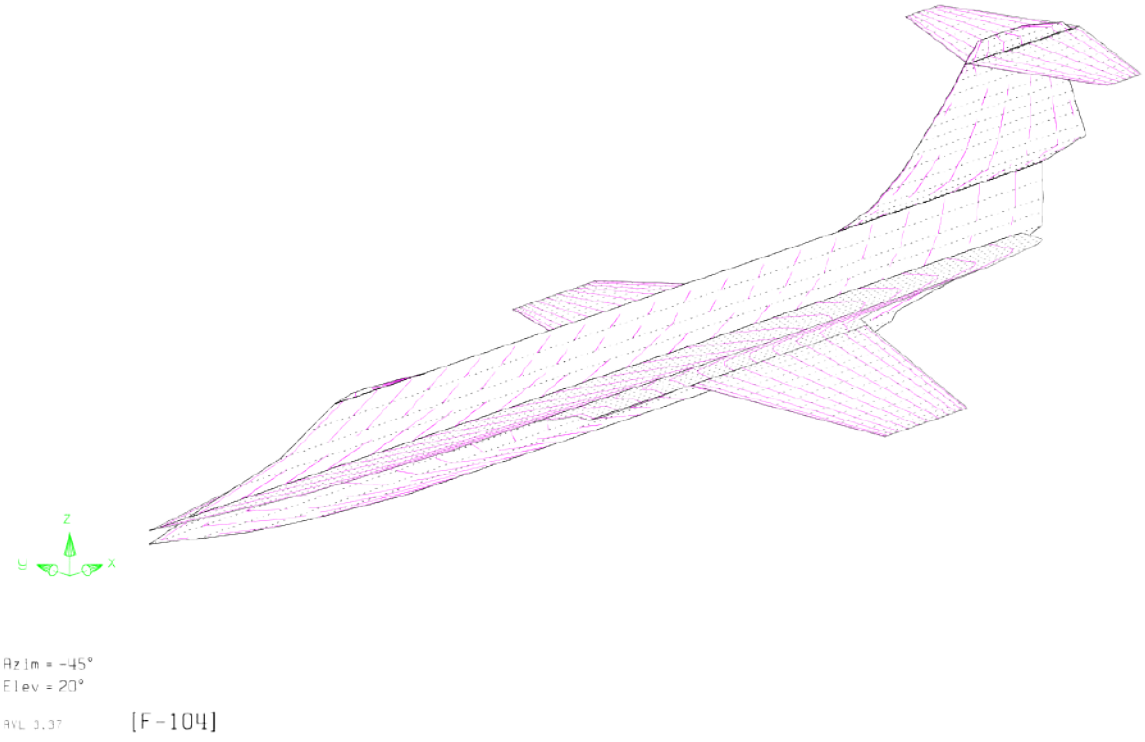


(b) Visualization of the geometry in AVL.

Figure 30: McDonnell F-101A Voodoo.

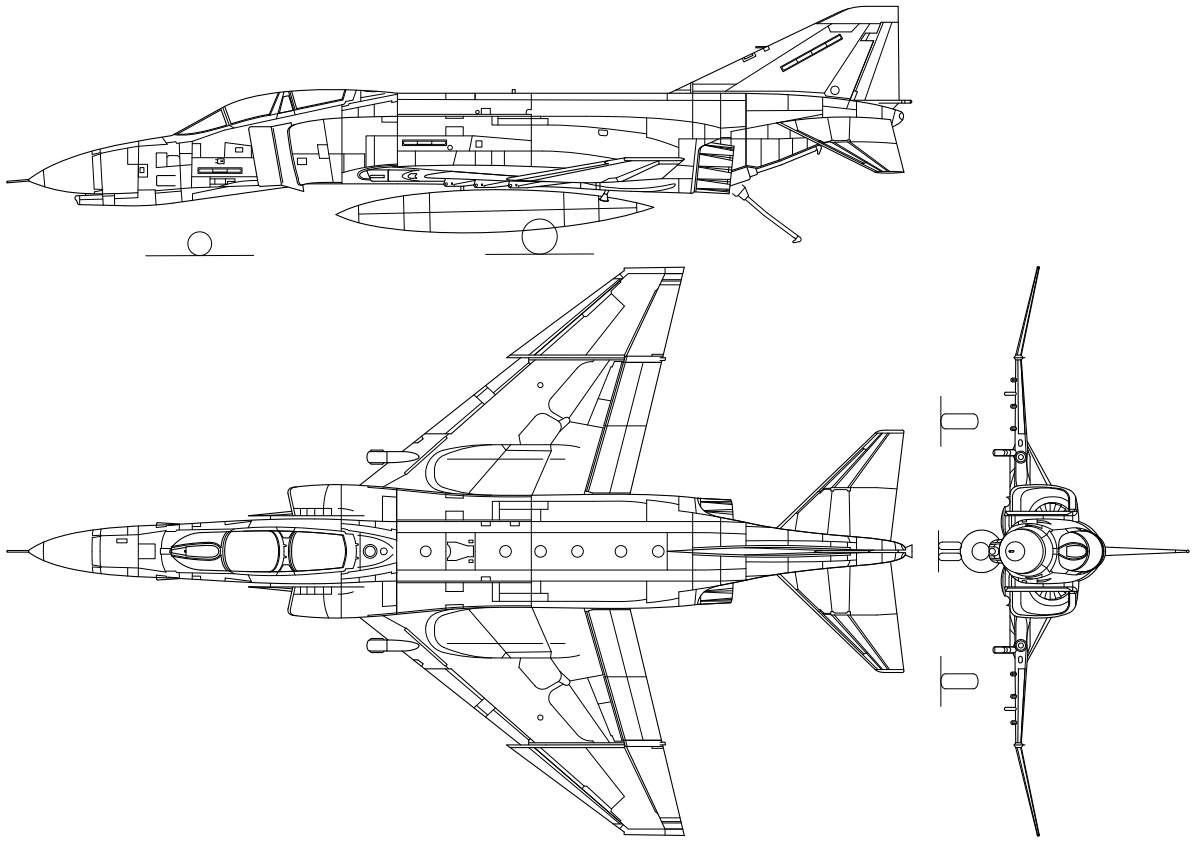


(a) Reference three-view drawing. Image by Kaboldy, CC BY-SA 3.0, via Wikimedia Commons.

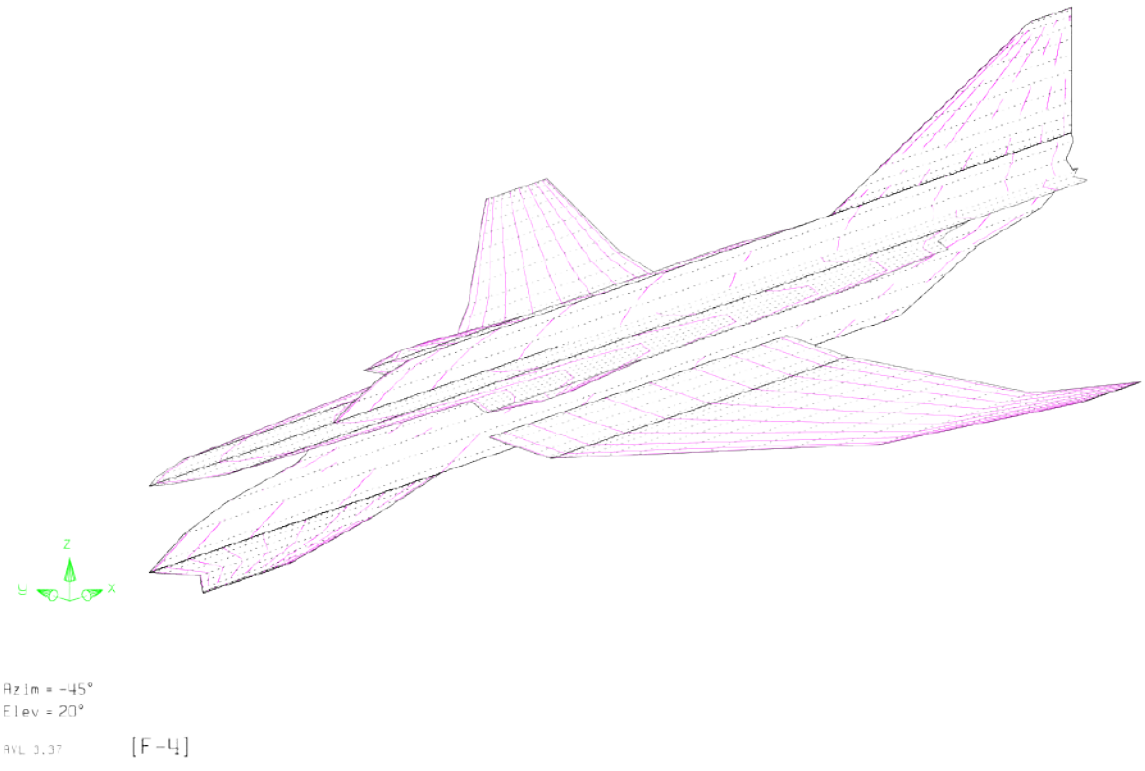


(b) Visualization of the geometry in AVL.

Figure 31: Lockheed F-104G Starfighter.

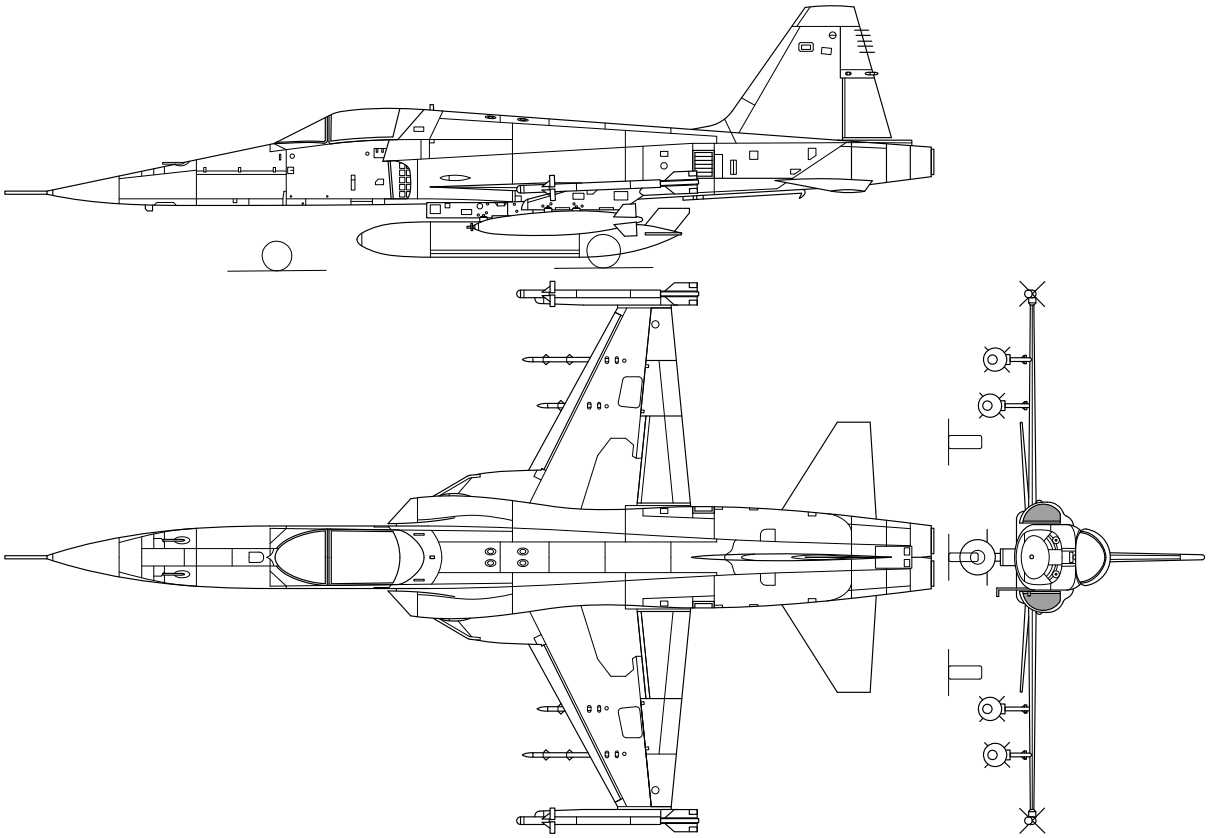


(a) Reference three-view drawing. Image by Kaboldy, CC BY-SA 3.0, via Wikimedia Commons.

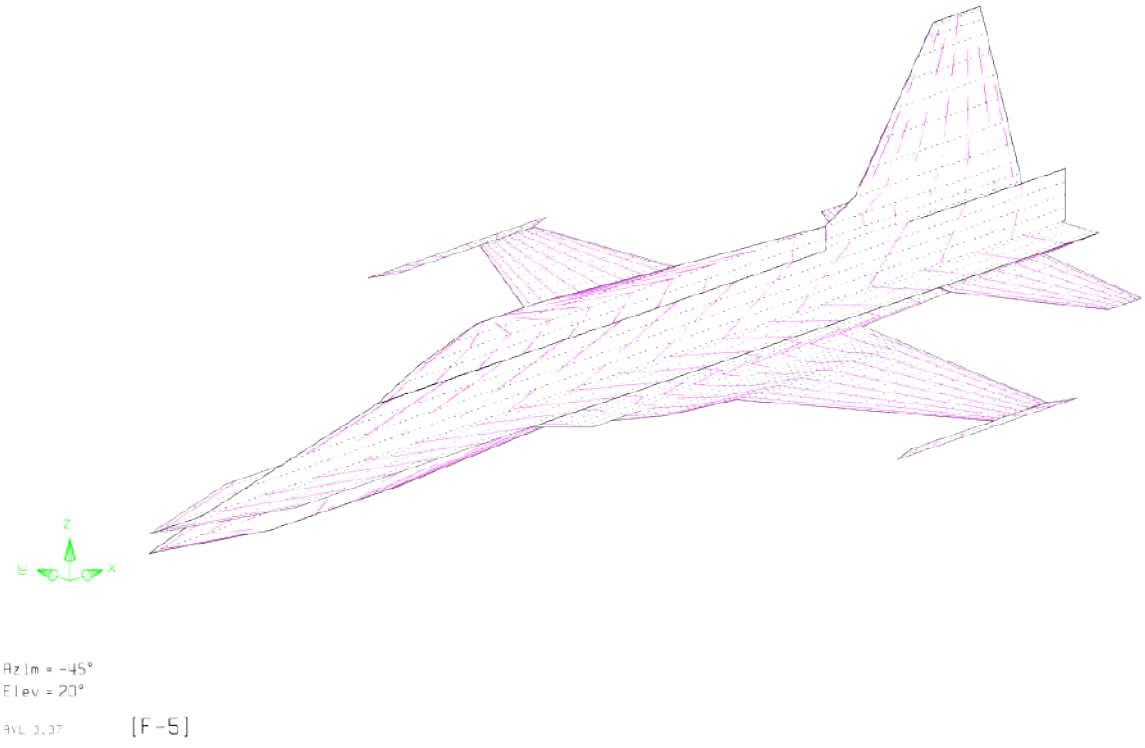


(b) Visualization of the geometry in AVL.

Figure 32: McDonnell Douglas F-4E Phantom II.

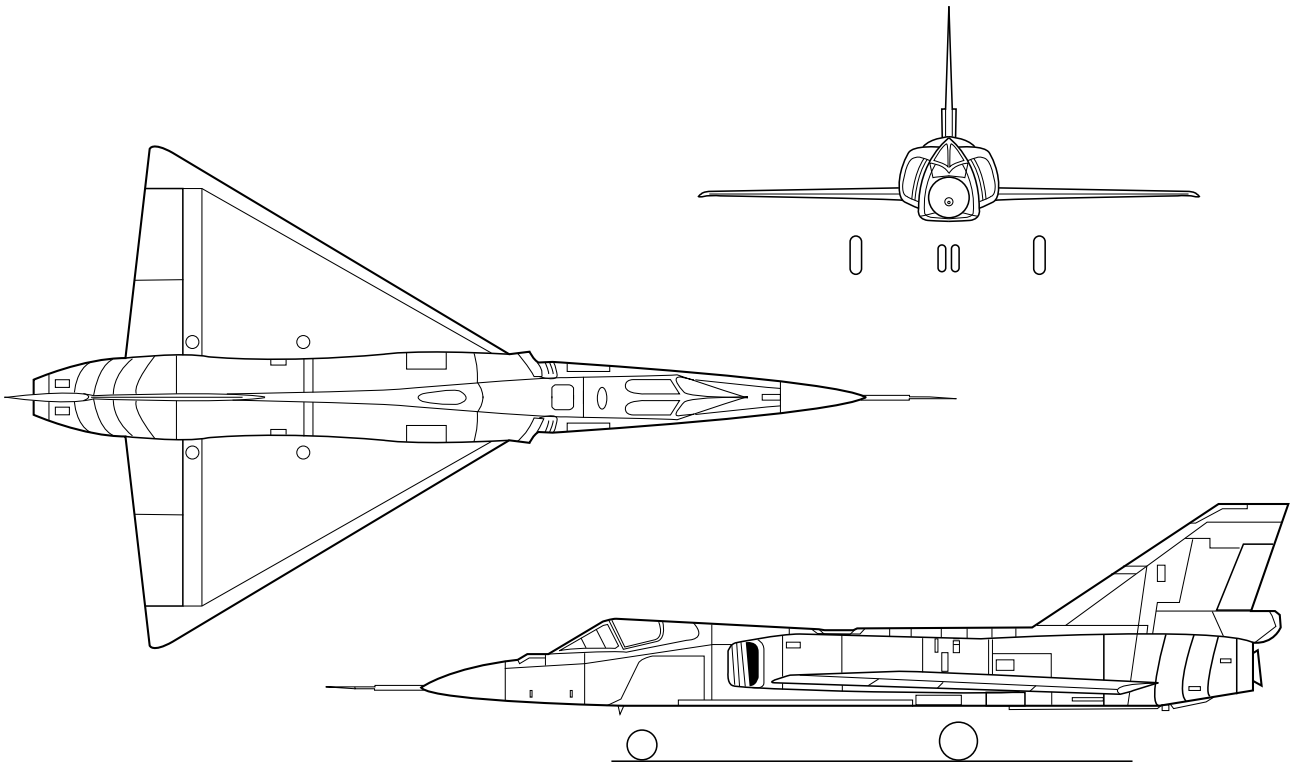


(a) Reference three-view drawing. Image by Kaboldy, CC BY-SA 3.0, via Wikimedia Commons.

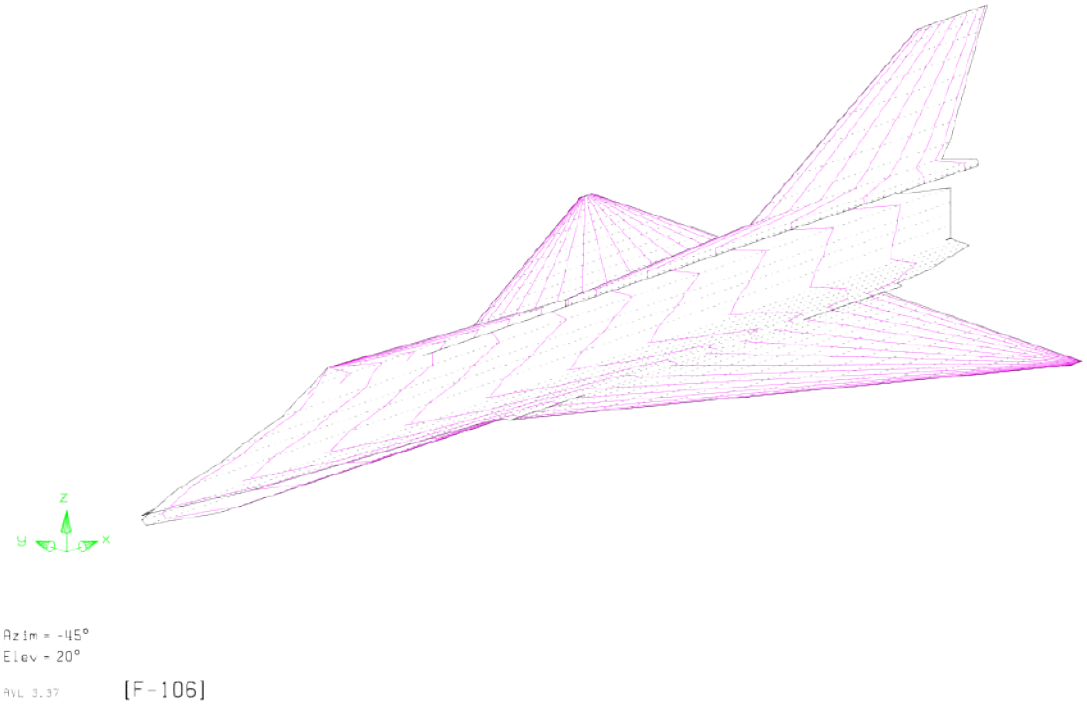


(b) Visualization of the geometry in AVL.

Figure 33: Northrop F-5E Tiger II.

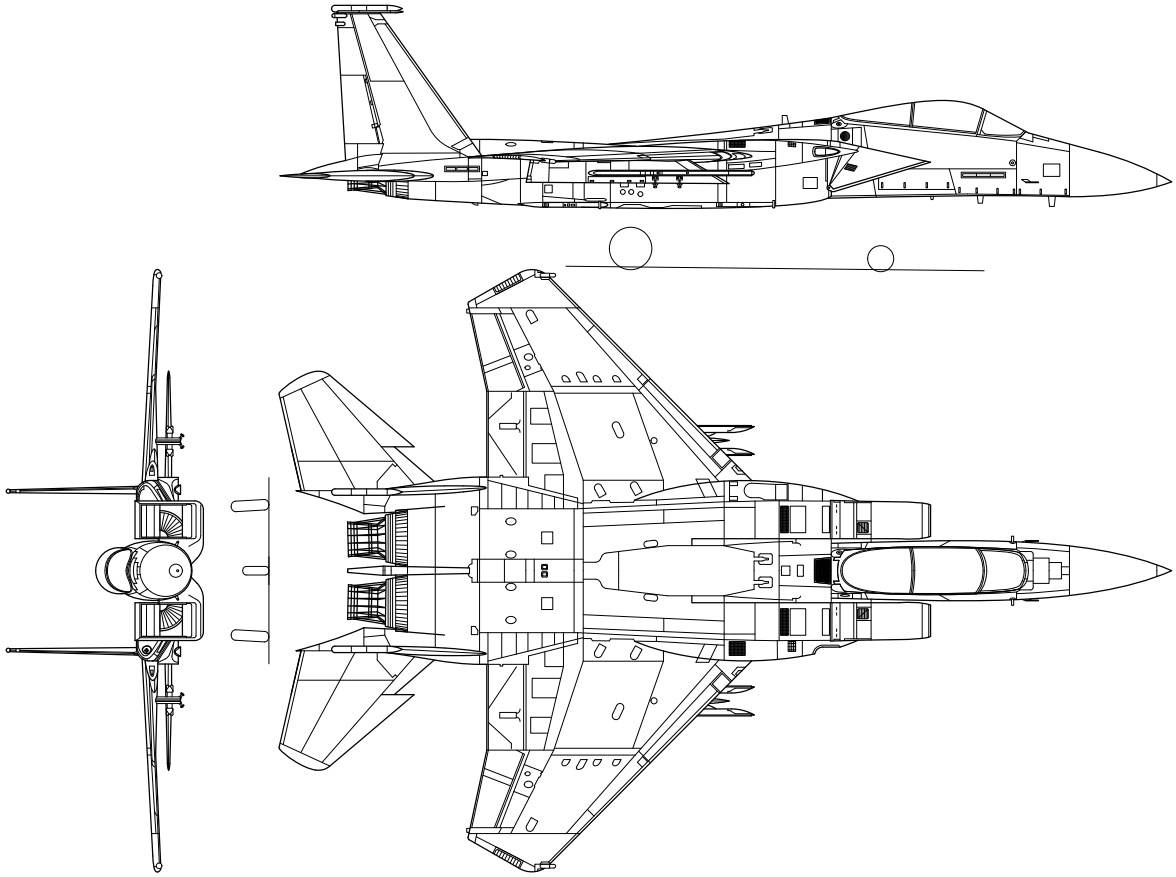


(a) Reference three-view drawing.

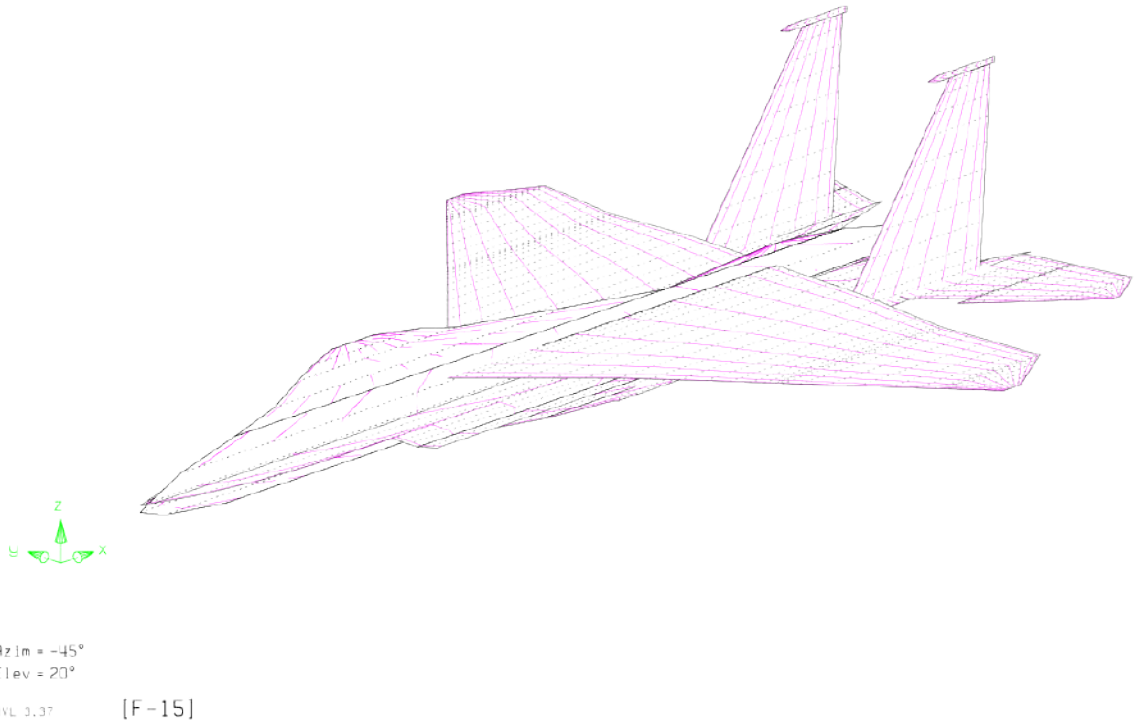


(b) Visualization of the geometry in AVL.

Figure 34: Convair F-106 Delta Dart.

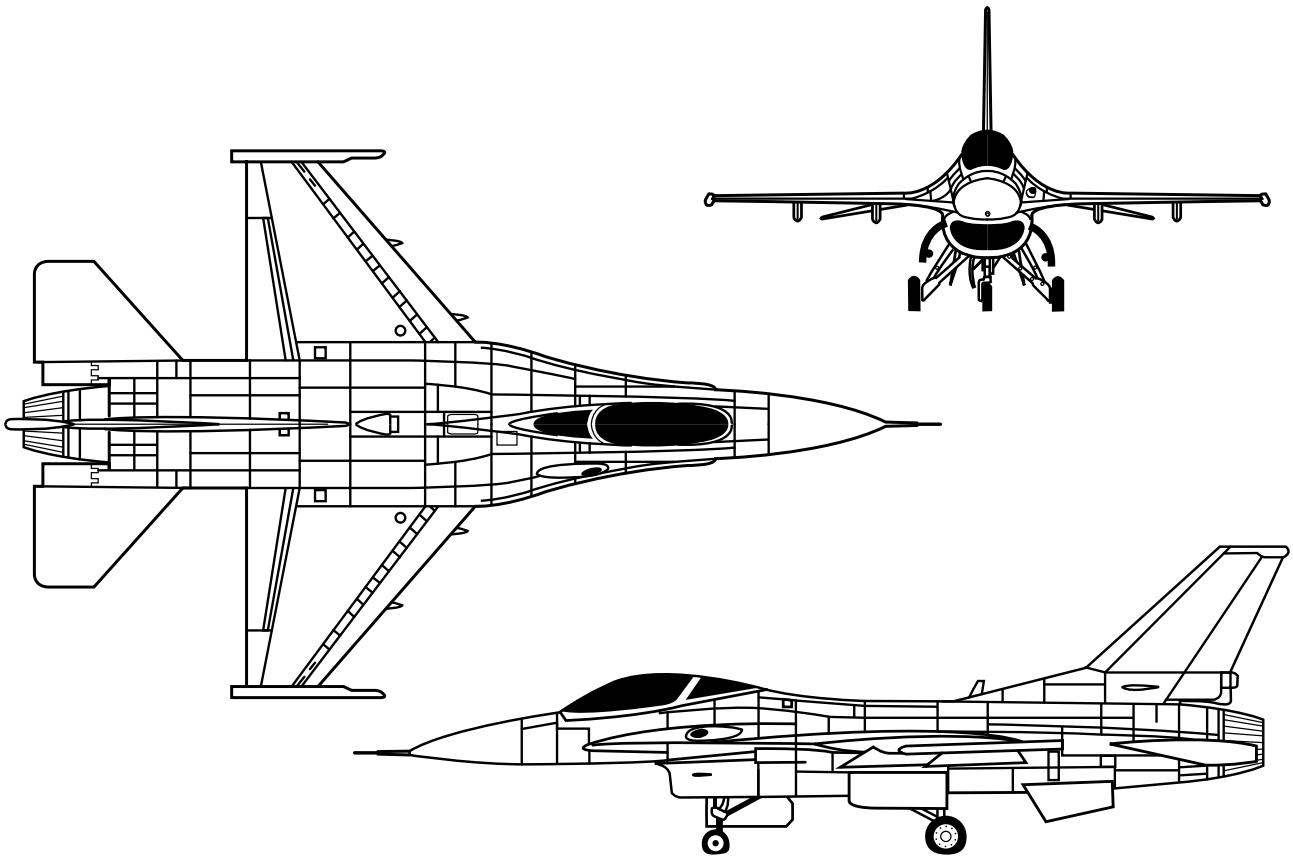


(a) Reference three-view drawing. Image by Kaboldy, CC BY-SA 3.0, via Wikimedia Commons.

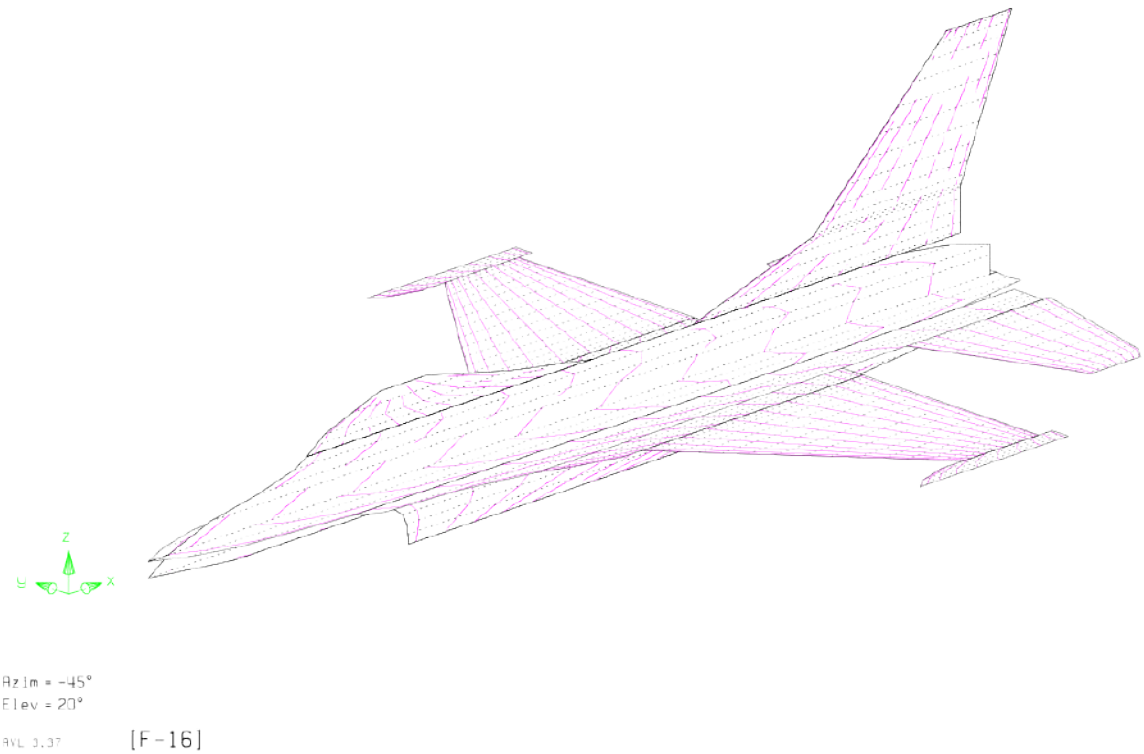


(b) Visualization of the geometry in AVL.

Figure 35: McDonnell Douglas F-15A Eagle.

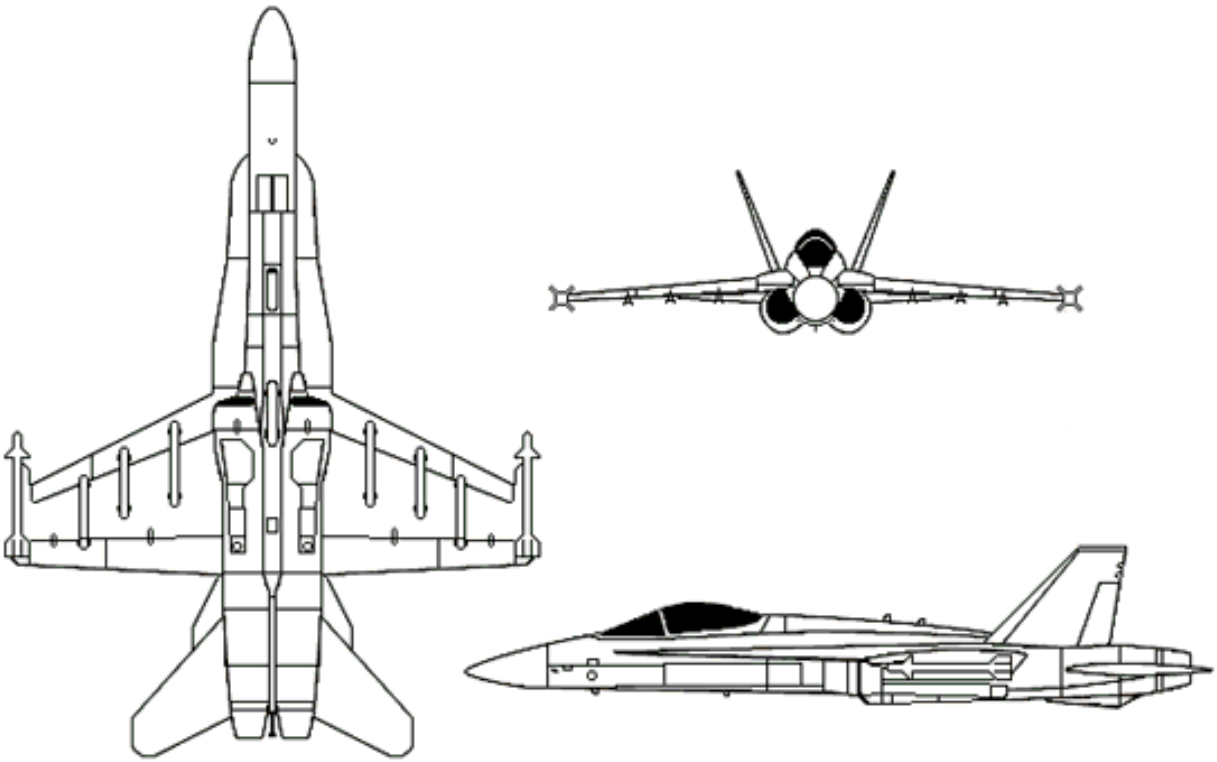


(a) Reference three-view drawing.

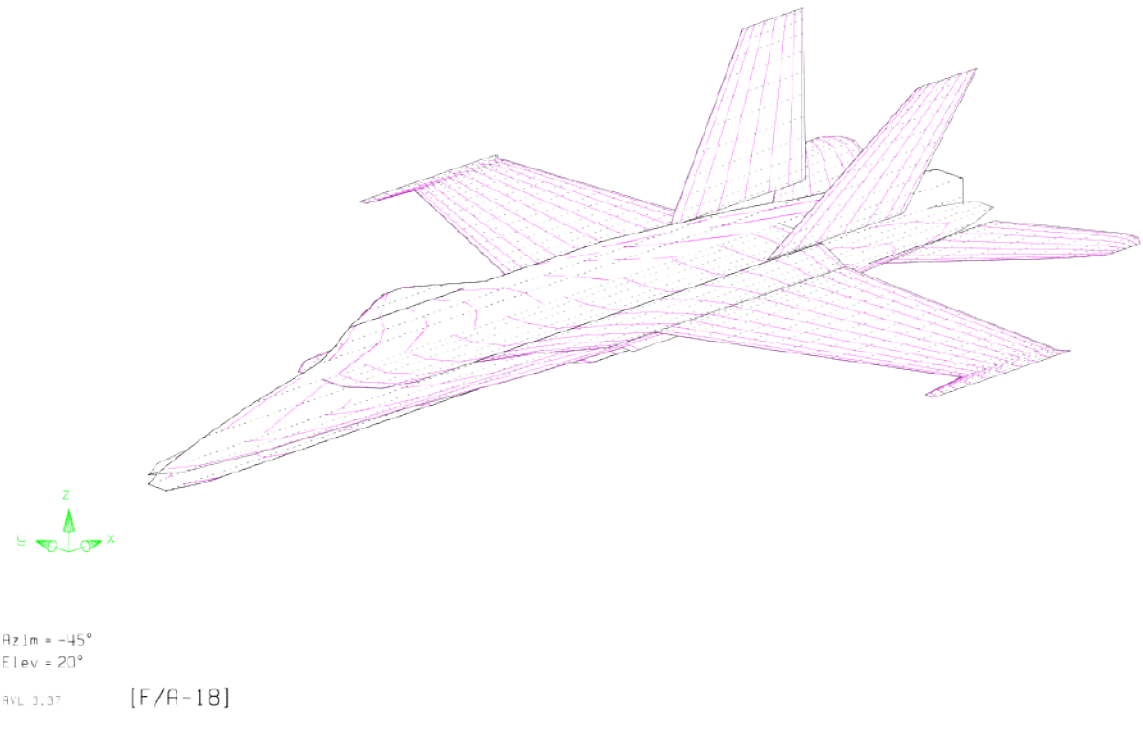


(b) Visualization of the geometry in AVL.

Figure 36: General Dynamics F-16A Fighting Falcon.

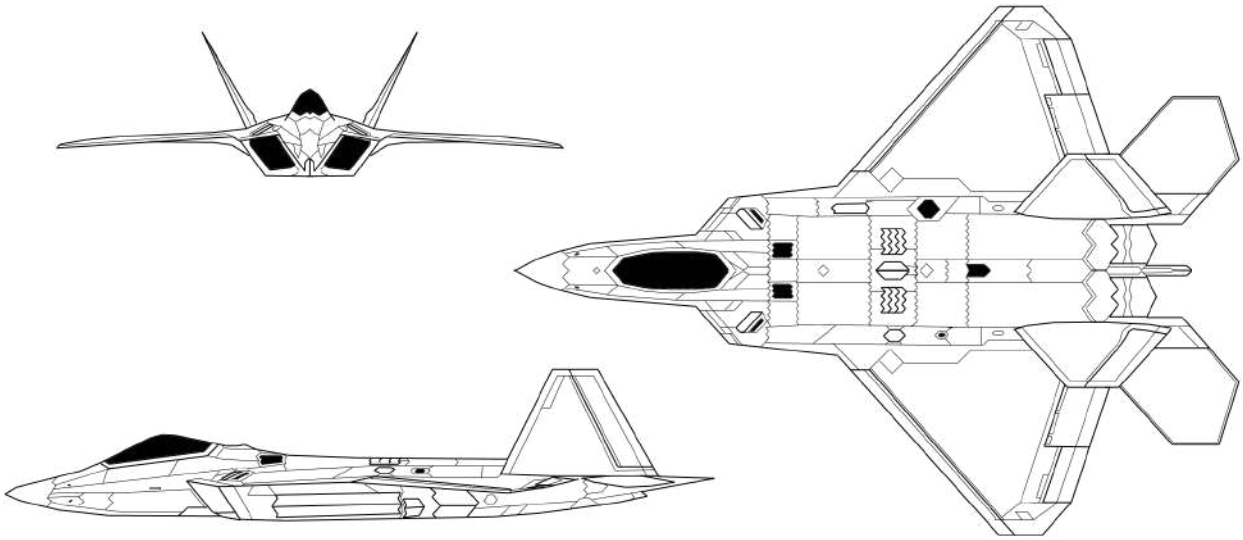


(a) Reference three-view drawing.

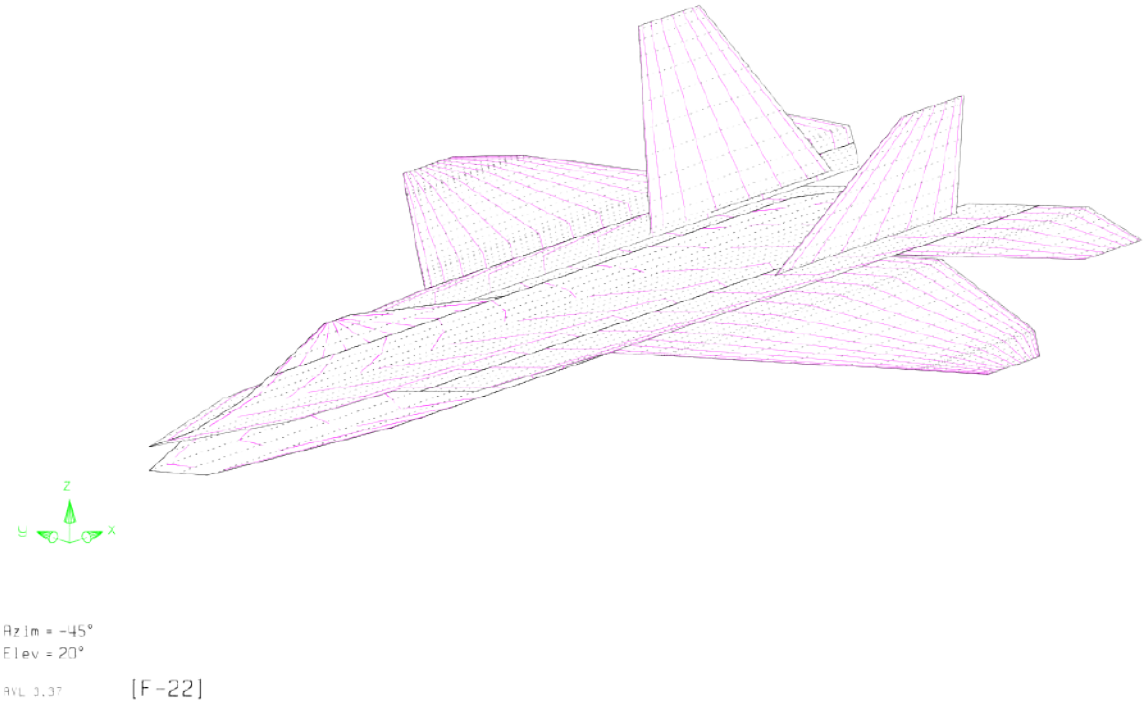


(b) Visualization of the geometry in AVL.

Figure 37: McDonnell Douglas F/A-18C Hornet.

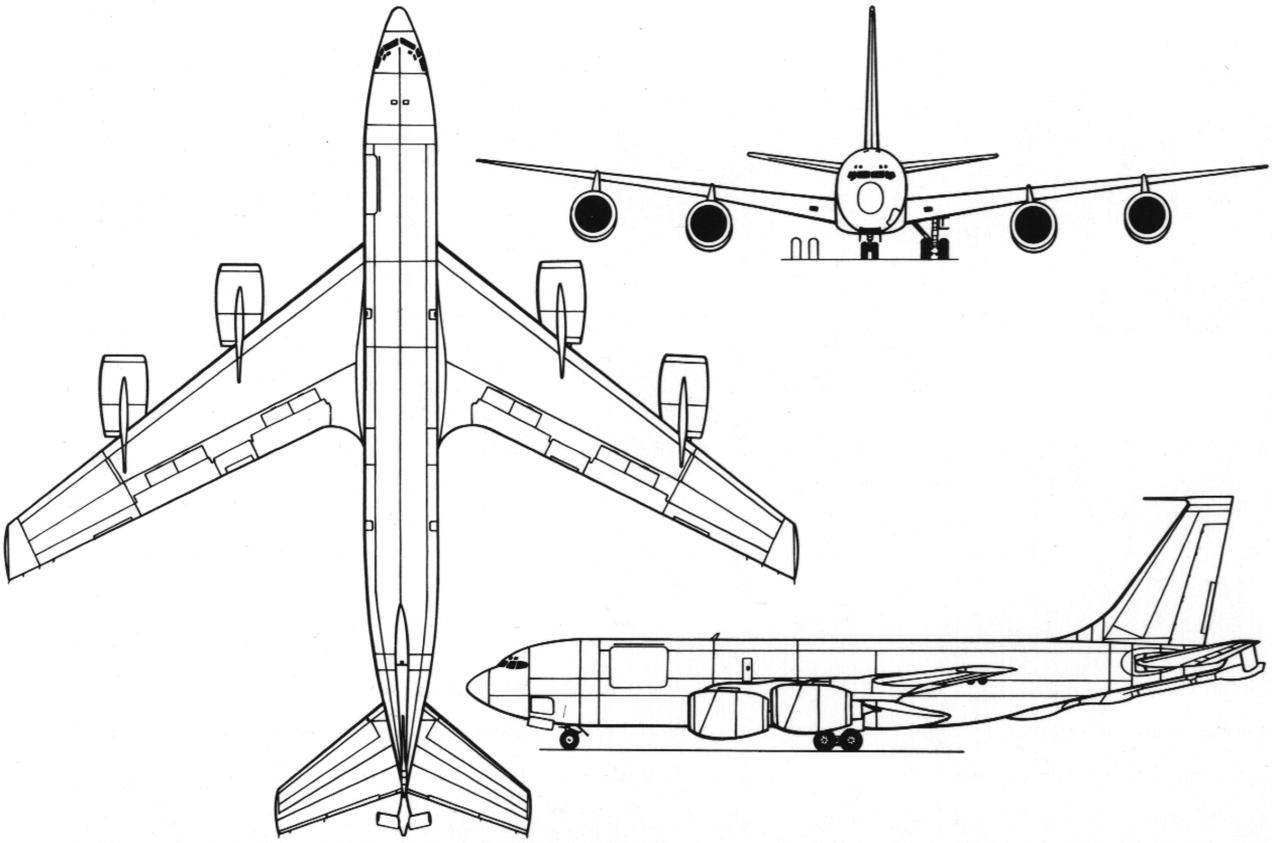


(a) Reference three-view drawing. Image by ZakuTalk, CC BY-SA 2.0, via Wikimedia Commons.

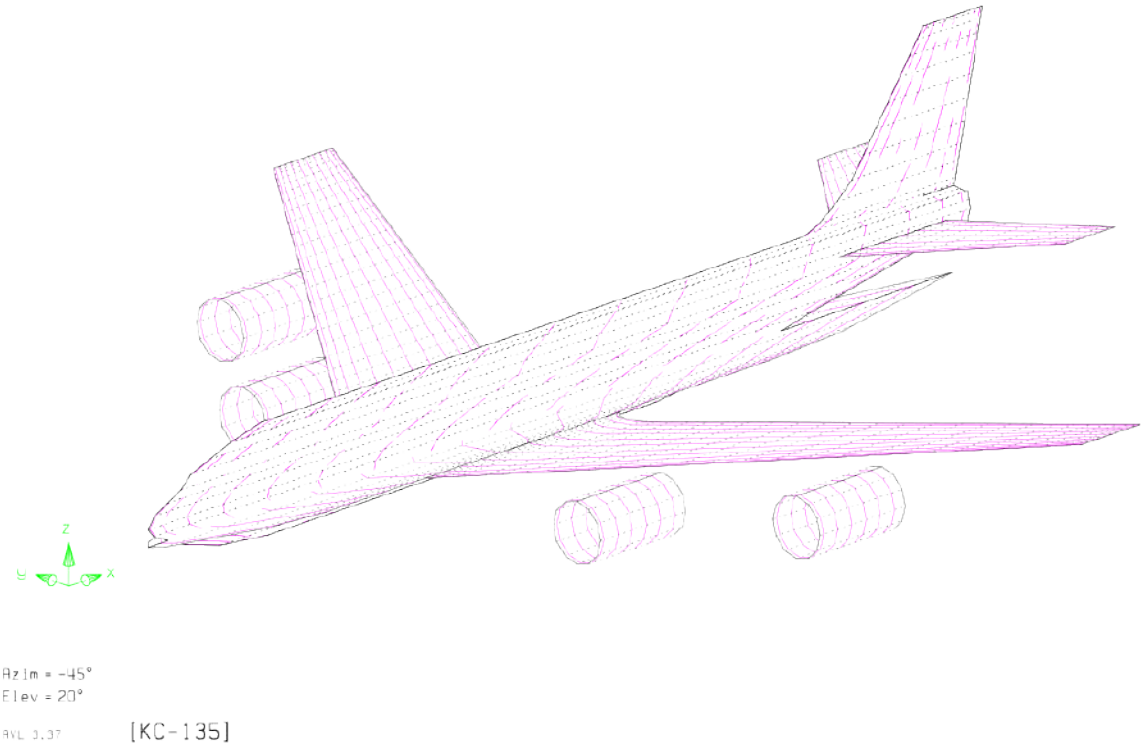


(b) Visualization of the geometry in AVL.

Figure 38: Lockheed Martin/Boeing F-22A Raptor.



(a) Reference three-view drawing.



(b) Visualization of the geometry in AVL.

Figure 39: Boeing KC-135R Stratotanker.

B. MATLAB codes

```
function WingOptimization()
%
% Authors: Federico Poleni, Giacomo Visconti, and Marco Zappoli
%
% ------------------------------------------------------------------
% GENERAL DESCRIPTION
% ------------------------------------------------------------------
% This function automates the aerodynamic optimization
% of a wing by combining MATLAB's genetic algorithm (GA)
% or particle swarm optimization (PSO) with AVL.
% Works for both single aircraft and formation flights.
%
% ------------------------------------------------------------------
% REQUIREMENTS
% ------------------------------------------------------------------
% MATLAB R2020+ with Global Optimization Toolbox.
% AVL installed and accessible as cfg.avlExe.
% Custom functions in MATLAB path:
%   - ACWingModification.m
%   - TwoACFormation.m
%   - GetCoeffAVL_fe.m
%
% ------------------------------------------------------------------
% USE
% ------------------------ ----------------------------------------
% 1. Place each case in a folder named <caseName> with <caseName>.avl.
% 2. Edit USER CONFIG at the top of this script.
% 3. Run: WingOptimization
%
% ------------------------------------------------------------------
% NOTES
% -------------- -------------------------------------------------
% - Only sections 2..N can move; the root is fixed.
% - The last SECTION (Yle) defines the semi-opening.
% - The SURFACE [Wing] must be present in the .avl file.
% ------------------------------------------------------------------

%% ========================= USER CONFIG ==========================
% Number and type of spacing between sections
cfg.N = 5;
cfg.spacing = 'cosine';  % 'equal', 'cosine', or 'halfcosine'

% Relative position between aircraft normalized by span
cfg.Dx = 2.0;
cfg.Dy = 1.0;
cfg.Dz = -0.5;

% Movement allowed for free sections (i=2..N)
cfg.dNegSweep = 40; % maximum value of negative sweep
cfg.dPosSweep = -20; % maximum value of positive sweep
cfg.dDihedral = 3; % maximum value of dihedral
cfg.dxBoxPos = [];
```

```
cfg.dxBoxNeg = [];
cfg.dzBox = [];
cfg.chBoxFrac = 0.1; % chord variation (0.1=10%)

% Fitness
cfg.objective = 'maxLD_at_alpha'; % type of optimization
cfg.alpha_deg = []; % angle of attack for evaluation

% Penalty coefficients
cfg.smoothW_x = 2.0; % penalty for non-continuous curvatures in X
cfg.smoothW_z = 2.0; % penalty for non-continuous curvatures in Z
cfg.smoothW_c = 1.0; % penalty for non-continuous curvatures in chord
cfg.delta_curv = 0.005; % tolerance in non-continuous curvatures
cfg.sweepSlopeW = 2.5; % penalty for abrupt variations in sweep
cfg.dihedralSlopeW = 2.5; % penalty for abrupt variations in dihedral
cfg.lineW_x = 10.0; % penalty for wings with triple or more sweep
cfg.lineW_z = 10.0; % penalty for wings with triple or more dihedral
cfg.refW_x = 5.0; % penalty for X values very far from the original
cfg.refW_z = 4.0; % penalty for Z values very far from the original
cfg.refW_c = 2.5; % penalty for chord values very far from the original
cfg.refTol_x = 0.01; % tolerance on variations of the original in X
cfg.refTol_z = 0.01; % tolerance on variations of the original in Z
cfg.refTol_c = 0.02; % tolerance on variations of the original in chord
cfg.secAreaW = 1.5; % penalty for high variations of adjacent chords
cfg.areaW = 8.0; % penalty for high variations in wing area
cfg.areaTol = 0.01; % tolerance for variations in wing area
cfg.jumpZ_W = 6.0; % penalty for very high dihedrals
cfg.jumpZ_thr = tan(deg2rad(1)); % tolerance for variations in dihedral
cfg.taperW = 30.0; % penalty on inverse taper ratios

% AVL / IO
cfg.avlExe = 'avl';
cfg.keepTemps = false; % keep candidate .avl files
cfg.Formation = false; % optimization considering formation flying
cfg.AllFolders = false; % optimization performed on all folders
cfg.SpecFolder = 'F-18'; % specifies which folder to consider

% Airfoils and Twist
cfg.defaultAirfoil = ''; % used if AFILE not found in reference
cfg.useRefAirfoil = true; % resample AFILE per section if present
cfg.useRefTwist = true; % keep reference twist
cfg.newTwist = false; % section twist variable of the optimization

% Optimizer
cfg.optimizer = 'pso'; % 'ga', 'pso', 'gamulti', or 'gamultisolo'
cfg.popSize = 5*3*(cfg.N-1); % initial population 5*nvars
cfg.useParallel = true;
rng('shuffle');

%% ================== FIND REFERENCE FOLDERS ==================
rootDir = pwd; folders = dir(rootDir); caseDirs = {};
for k = 1 : numel(folders)
    if folders(k).isdir && ~startsWith(folders(k).name, '.')
```

```
        cand = fullfile(rootDir, folders(k).name);
        refAVL = fullfile(cand, [folders(k).name, '.avl']);
        if exist(refAVL,'file'), caseDirs{end+1} = cand; end
    end
end
if isempty(caseDirs)
    warning('No folder with <folder>/<folder>.avl found.');
    L = dir(fullfile(rootDir, '*.avl'));
    if ~isempty(L), caseDirs = {rootDir}; else, return; end
end
% If the user did not request all folders, filter by cfg.SpecFolder
if ~cfg.AllFolders
    newcaseDirs = {};
    for icase = 1 : numel(caseDirs)
        caseDir = caseDirs{icase}; [~, caseName] = fileparts(caseDir);
        if contains(cfg.SpecFolder, caseName)
            newcaseDirs{end+1} = caseDirs{icase};
        end
    end
    caseDirs = newcaseDirs;
end

%% ========================== MAIN LOOP ==========================
for icase = 1 : numel(caseDirs)
    caseDir = caseDirs{icase}; [~, caseName] = fileparts(caseDir);
    fprintf('\n===== CASE: %s =====\n', caseName);

    % Ensure per-case output directory exists
    outDir = fullfile(caseDir, 'wing_opt_outputs', cfg.optimizer);
    if ~exist(outDir,'dir'), mkdir(outDir); end

    % Determine reference AVL to use
    refFile = fullfile(caseDir, [caseName, '.avl']);
    if ~exist(refFile,'file')
        L = dir(fullfile(caseDir,'*.avl'));
        if isempty(L)
            warning('No .avl in %s. Skipping.', caseDir); continue;
        end
        refFile = fullfile(caseDir, L(1).name);
        [~,caseName] = fileparts(refFile);
    end

    % Identification of the angle of attack
    if cfg.Formation
        alpha_formation = readAlpha(refFile, true); cfg.alpha_deg = 0;
    else
        alpha_formation = []; cfg.alpha_deg = readAlpha(refFile);
    end

    % Storage of the reference aerodynamic coefficient
    data_ref = load(fullfile(caseName, 'formation_angle_output', ...
        'Formation_Alpha_history.mat'));
    dot_ref = sprintf('%s_Formation_Alpha_history', ...
```

```
    strrep(caseName, '-', '_'));
data_ref = data_ref.(dot_ref);
coeff_ref = data_ref(end).coeff;
data_solo = load(fullfile(caseName, sprintf('%s.mat', caseName)));
dot_solo = sprintf('%s_Coeff', strrep(caseName, '-', '_'));
coeff_solo = data_solo.(dot_solo);

% Parse reference SURFACE [Wing]
ref = parseAVL_Wing(refFile);
if isempty(ref.y)
    warning('No SURFACE "[Wing]" found in %s.', refFile);
    continue;
end

% Resample reference distributions at stations
b2 = ref.y(end); span_total = 2*b2;
y_half = stationY(span_total, cfg.N, cfg.spacing, ref.y(1));
xle_ref = interp1(ref.y, ref.x, y_half, 'pchip', 'extrap');
xle_ref(1) = ref.x(1);
zle_ref = interp1(ref.y, ref.z, y_half, 'pchip', 'extrap');
zle_ref(1) = ref.z(1);
c_ref   = interp1(ref.y, ref.c, y_half, 'pchip', 'extrap');
c_ref(1) = ref.c(1);
if cfg.useRefTwist && ~isempty(ref.ainc)
    ainc_ref = interp1(ref.y, ref.ainc, y_half, 'linear','extrap');
else
    ainc_ref = zeros(size(y_half));
end
ainc_ref(1) = ref.ainc(1);
claf_ref = interp1(ref.y, ref.claf, y_half, 'pchip', 'extrap');
claf_ref(1) = ref.claf(1);
if cfg.useRefAirfoil && ~isempty(ref.airfoil)
    airfoil_ref = resampleAirfoils(ref.y, ref.airfoil, ...
        y_half, caseDir);
else
    airfoil_ref = repmat({cfg.defaultAirfoil}, 1, numel(y_half));
end

% Bounds for variables
lb = []; ub = [];
for i = 2 : cfg.N
    cfg.dxBoxNeg = (y_half(i) - y_half(i-1)) * tand(cfg.dPosSweep);
    cfg.dxBoxPos = (y_half(i) - y_half(i-1)) * tand(cfg.dNegSweep);
    cfg.dzBox = (y_half(i) - y_half(i-1)) * tand(cfg.dDihedral);
    chFrac = (cfg.chBoxFrac+0.1*(i-2));
    lb = [lb, cfg.dxBoxNeg, -cfg.dzBox, (1-chFrac)];
    ub = [ub, cfg.dxBoxPos,  cfg.dzBox, (1+chFrac)];
end
if cfg.newTwist
    lb = [lb, -5]; ub = [ub, 5];
end
nvars = numel(lb);
```

```
% Setup a DataQueue so worker processes can send results serially.
logFile = fullfile(outDir, 'eval_history.mat');
logVar  = matlab.lang.makeValidName(sprintf('%s_%s_history', ...
    caseName, cfg.optimizer));
q = parallel.pool.DataQueue;
afterEach(q, @(entry) appendHistory(logFile, logVar, entry));

% Objective function
objFun = @(x) evalCandidate_case(x, caseName, caseDir, outDir,
   y_half, xle_ref, zle_ref, c_ref, ainc_ref, airfoil_ref, claf_ref
   , cfg, alpha_formation, coeff_ref, coeff_solo, q);

% ======================= OPTIMIZE =======================
switch lower(cfg.optimizer)
    case 'ga'
        opts = optimoptions('ga', 'PopulationSize', cfg.popSize, '
           MaxGenerations', 50, 'Display', 'iter', 'UseParallel',
           cfg.useParallel, 'MutationFcn','mutationadaptfeasible');
        [xbest,fbest] = ga(objFun,nvars,[],[],[],[],lb,ub,[],opts);
    case 'pso'
        opts = optimoptions('particleswarm', 'SwarmSize', cfg.
           popSize, 'MaxIterations', 75, 'Display', 'iter', '
           UseParallel', cfg.useParallel);
        [xbest,fbest] = particleswarm(objFun, nvars, lb, ub, opts);
    case 'gamulti'
        opts = optimoptions('gamultiobj', 'PopulationSize', cfg.
           popSize, 'MaxGenerations', 60, 'Display', 'iter', '
           UseParallel', cfg.useParallel, 'MutationFcn', '
           mutationadaptfeasible');
        [xbest,fbest] = gamultiobj(objFun, nvars, [], [], [], [],
           lb, ub, [], opts);
    case 'gamultisolo'
        opts = optimoptions('gamultiobj', 'PopulationSize', cfg.
           popSize, 'MaxGenerations', 50, 'Display', 'iter', '
           UseParallel', cfg.useParallel, 'MutationFcn', '
           mutationadaptfeasible');
        [xbest,fbest] = gamultiobj(objFun, nvars, [], [], [], [],
           lb, ub, [], opts);
    otherwise
        error('cfg.optimizer name not found');
end

% Decode best and export
if contains(lower(cfg.optimizer), 'gamulti')
    [~, bestMeta, geo] = objFun(xbest(1,:));
else
    [~, bestMeta, geo] = objFun(xbest);
end
save(fullfile(outDir, sprintf('%s_results.mat',caseName)), ...
    'cfg', 'xbest', 'fbest', 'bestMeta', 'geo', 'ref');
plotPlanform(geo.yle, geo.xle, geo.zle, geo.c, ...
    fullfile(outDir, [caseName '_planform.png']));
if ~cfg.Formation && ~strcmp(cfg.optimizer, 'gamultisolo')
```

```
        fprintf('\n[CASE %s] Best f = %.6g | CL=%.5f, CD=%.5f\n',
           caseName, fbest, bestMeta.CL, bestMeta.CD);
    elseif ~cfg.Formation && strcmp(cfg.optimizer, 'gamultisolo')
        fprintf('\n[CASE %s] Best f0 = %.6g | Best f0 = %.6g | CL=%.5f,
            CD=%.5f\n', caseName, fbest(1, 1), fbest(1, 2), bestMeta.CL
           , bestMeta.CD);
    else
        fprintf('\n[CASE %s] Best f_lead = %.6g | Best f_trail = %.6g |
            CL(lead)=%.5f, CD(lead)=%.5f| CL(trail)=%.5f, CD(trail)=%.5
           f\n', caseName, fbest(1), fbest(2), bestMeta.Lead.CL,
           bestMeta.Lead.CD, bestMeta.Trail.CL, bestMeta.Trail.CD);
    end
end
end

%% ========================== EVALUATOR ==========================
function [f, meta, geo] = evalCandidate_case(x, caseName, caseDir,
   outDir, y_half, xle_ref, zle_ref, c_ref, ainc_ref, airfoil_ref,
   claf_ref, cfg, alpha_formation, coeff_ref, coeff_solo, q)

% Decode variables
evalCount = randi(1e12);
N = numel(y_half);
DX = zeros(1,N);
DZ = zeros(1,N);
SC = ones(1,N);
k = 1;
for i = 2 : N
    DX(i) = x(k); DZ(i) = x(k+1); SC(i) = x(k+2); k = k+3;
end
Xle = xle_ref;
Yle = y_half;
Zle = zle_ref;
Chord = max(1e-6, c_ref .* SC);
for i = 2 : N
    Xle(i) = Xle(i-1) + DX(i); Zle(i) = Zle(i-1) + DZ(i);
end

% Compose parameter and airfoil cell for ACWingModification
if cfg.newTwist
    twist_root = 0; twist_tip = x(end);
    Ainc = interp1([Yle(1), Yle(end)], [twist_root, twist_tip], ...
        Yle, 'linear', 'extrap');
else
    Ainc = ainc_ref;
end
parameter = [Xle; Yle; Zle; Chord; Ainc];
airfoil = airfoil_ref;

dy = diff(Yle); meanDy = mean(dy); span = Yle(end) - Yle(1);
C0 = mean(c_ref); nz = @(v) max(1e-12, v);

D2x = d2_nonuniform(Yle, Xle);
```

```
D2z = d2_nonuniform(Yle, Zle);
D2c = d2_nonuniform(Yle, Chord);

% local slopes
sweepSlope = diff(Xle) ./ nz(dy);
dihedSlope = diff(Zle) ./ nz(dy);

% Single-line residuals
px = polyfit(Yle, Xle, 1);
r1x = mean(abs(Xle - polyval(px, Yle)) / nz(C0));
pz = polyfit(Yle, Zle, 1);
r1z = mean(abs(Zle - polyval(pz, Yle)) / nz(C0));

% Two-line residuals with kink cost
[r2x_abs, kbest_x] = piecewiseLinRes(Yle, Xle, [], true, cfg.refTol_x);
[r2z_abs, kbest_z] = piecewiseLinRes(Yle, Zle, [], true, cfg.refTol_z);
r2x = mean(r2x_abs / nz(C0));
r2z = mean(r2z_abs / nz(C0));
if ~isnan(kbest_x)
    leftSlope = polyfit(Yle(1:kbest_x), Xle(1:kbest_x), 1);
    rightSlope = polyfit(Yle(kbest_x+1:end), Xle(kbest_x+1:end), 1);
    slopeJump = abs(leftSlope(1)-rightSlope(1));
else
    slopeJump = 0;
end
if ~isnan(kbest_z)
    leftSlope = polyfit(Yle(1:kbest_z), Zle(1:kbest_z), 1);
    rightSlope = polyfit(Yle(kbest_z+1:end), Zle(kbest_z+1:end), 1);
    slopeJump_Z = abs(leftSlope(1)-rightSlope(1));
else
    slopeJump_Z = 0;
end

% Deviation from reference
rx_ref = (Xle - xle_ref) / nz(C0);
rz_ref = (Zle - zle_ref) / nz(C0);
rc_ref = (Chord - c_ref) / nz(C0);
secVar = trapz(Yle, abs(Chord - c_ref)) / nz(C0*span);
S_now = planformArea(Yle, Chord);
S_ref = planformArea(Yle, c_ref);
relArea = abs(S_now - S_ref) / max(1e-9, S_ref);

% 1) Curvature penalties
curvX_n = (D2x .* (meanDy^2)) ./ nz(C0);
curvZ_n = (D2z .* (meanDy^2)) ./ nz(C0);
curvC_n = (D2c .* (meanDy^2)) ./ nz(C0);
% Energy penalties punishes many moderate curvatures
pen_curvX_energy = cfg.smoothW_x * sum(curvX_n.^2);
pen_curvZ_energy = cfg.smoothW_z * sum(curvZ_n.^2);
pen_curvC_energy = cfg.smoothW_c * sum(curvC_n.^2);
% Peak penalties punishes single large curvature spikes
peakMultiplier = 150;
```

```
pen_curvX_peak = cfg.smoothW_x * peakMultiplier * mean(huber(curvX_n,
    cfg.delta_curv));
pen_curvZ_peak = cfg.smoothW_z * peakMultiplier * mean(huber(curvZ_n,
    cfg.delta_curv));
pen_curvC_peak = cfg.smoothW_c * peakMultiplier * mean(huber(curvC_n,
    cfg.delta_curv));
% Assemble final smoothness penalties
pen_x = pen_curvX_energy + pen_curvX_peak;
pen_z = pen_curvZ_energy + pen_curvZ_peak;
pen_c = pen_curvC_energy + pen_curvC_peak;

% 2) Slope-jump penalties: penalize abrupt changes
% Use a hinge so small changes are allowed, large jumps are costly.
slope_jump_thresh = 0.5;
sweepSlopeJump = diff(sweepSlope);
dihedSlopeJump = diff(dihedSlope);
hinge = @(r,thr) max(0, abs(r) - thr);
pen_sweepSlope = cfg.sweepSlopeW * sum(hinge(sweepSlopeJump * meanDy,
    slope_jump_thresh).^2);
pen_dihedralSlope = cfg.dihedralSlopeW * sum(hinge(dihedSlopeJump *
    meanDy, slope_jump_thresh).^2);

% 3) Triple or more sweep and dihedral penalties
kinkCost = 0.02;
pen_line_x = cfg.lineW_x*(min(r1x,r2x+kinkCost) + 100*slopeJump^2);
pen_line_z = cfg.lineW_z*(min(r1z,r2z+kinkCost) + 100*slopeJump_Z^2);

% 4) Variation from reference penalties
pen_ref_x = cfg.refW_x * mean(huber(rx_ref, cfg.refTol_x));
pen_ref_z = cfg.refW_z * mean(huber(rz_ref, cfg.refTol_z));
pen_ref_c = cfg.refW_c * mean(huber(rc_ref, cfg.refTol_c));

% 5) Sectional chord variation and Area penalties
pen_secArea = cfg.secAreaW * secVar^2;
pen_area = cfg.areaW * max(0, relArea - cfg.areaTol)^2;

% 6) Z-jump control
slopeZ = abs(dihedSlope);
excess = max(0, slopeZ - cfg.jumpZ_thr);
pen_jump = cfg.jumpZ_W * mean(excess.^2);

% 7) Hard penalty on excessive curvature
Rmin = 0.5;
curvX_local = abs(D2x);
curvZ_local = abs(D2z);
curvX_excess = max(0, curvX_local - 1/Rmin);
curvZ_excess = max(0, curvZ_local - 1/Rmin);
pen_curv_hard = 1e4 * (sum(curvX_excess.^2) + sum(curvZ_excess.^2));

% 8) Inverse taper penalty
taperExcess = max(0,(Chord(2:end)-Chord(1:end-1)) ./ Chord(1:end-1));
pen_taper = cfg.taperW * sum(max(0, taperExcess).^2);
```

```
% Generation and run of the AVL file to obtain aero coefficient
refAVL = fullfile(caseDir, [caseName, '.avl']);
try
    ACWingModification(refAVL, evalCount, parameter, airfoil, claf_ref,
        outDir);
catch ME
    error('ACWingModification failed: %s', ME.message);
end
if ~cfg.Formation
    runFile = fullfile(caseName, 'wing_opt_outputs', cfg.optimizer,
       sprintf('%s_v%d.avl', caseName, evalCount));
    forceFile = fullfile(caseName, 'wing_opt_outputs', cfg.optimizer,
       sprintf('%s_v%d_force', caseName, evalCount));
    coeffFile = fullfile(caseName, 'wing_opt_outputs', cfg.optimizer,
       sprintf('%s_v%d.mat', caseName, evalCount));
else
    ACWM_outFile = fullfile(caseName, 'wing_opt_outputs', cfg.optimizer
       , sprintf('%s_v%d.avl', caseName, evalCount));
    try
        TwoACFormation(ACWM_outFile, cfg.Dx, cfg.Dy, cfg.Dz, [],
           alpha_formation(1), alpha_formation(2));
    catch ME
        error('TwoACFormation failed: %s', ME.message);
    end
    runFile = fullfile(caseName, 'wing_opt_outputs', cfg.optimizer,
       sprintf('%s_v%d_TwoACFormation.avl', caseName, evalCount));
    forceFile = fullfile(caseName, 'wing_opt_outputs', cfg.optimizer,
       sprintf('%s_v%d_TwoACFormation_force', caseName, evalCount));
    coeffFile = fullfile(caseName, 'wing_opt_outputs', cfg.optimizer,
       sprintf('%s_v%d_TwoACFormation.mat', caseName, evalCount));
end
res = GetCoeffAVL_fe(runFile, cfg.alpha_deg);

% Evaluation of the fitness
switch cfg.objective
    case 'maxLD_at_alpha'
        if ~cfg.Formation
            if strcmp(cfg.optimizer, 'gamultisolo')
               if res.CD<=(coeff_solo.CD*20/100) || res.CL>=2*
                  coeff_solo.CL || res.CL<=0
                   f0 = 1e6; f1 = 1e6;
               else
                   f0 = -10*res.CL; f1 = 100*res.CD;
               end
            else
                if res.CD<=(coeff_solo.CD*20/100) || res.CL>=2*
                   coeff_solo.CL || res.CL<=0
                    f0 = 1e6;
                else
                    f0 = -res.CL/res.CD;
                end
            end
        elseif strcmp(cfg.optimizer, 'gamulti')
```

```
            if res.Lead.CD<=(coeff_solo.CD*20/100) || res.Trail.CD<=(
               coeff_solo.CD*20/100) || res.Lead.CL>=2*coeff_solo.CL ||
                res.Trail.CL>=2*coeff_solo.CL || res.Lead.CL<=0 || res.
               Trail.CL<=0
                f0 = 1e6; f1 = 1e6;
            else
                f0 = -res.Lead.CL/res.Lead.CD;
                f1 = - res.Trail.CL/res.Trail.CD;
            end
        end
    otherwise
        error('Unknown fitness');
end

% Building of the function outputs
if ~cfg.Formation && ~strcmp(cfg.optimizer, 'gamultisolo')
    f = f0 + pen_x + pen_z + pen_c + pen_area + pen_jump +
       pen_sweepSlope + pen_dihedralSlope + pen_line_x + pen_line_z +
       pen_ref_x + pen_ref_z + pen_ref_c + pen_secArea + pen_curv_hard
       + pen_taper;
else
    f = [f0, f1] + (pen_x + pen_z + pen_c + pen_area + pen_jump +
       pen_sweepSlope + pen_dihedralSlope + pen_line_x + pen_line_z +
       pen_ref_x + pen_ref_z + pen_ref_c + pen_secArea + pen_curv_hard
       + pen_taper)/2;
end
meta = res;
meta.f0 = f0;
if cfg.Formation || strcmp(cfg.optimizer, 'gamultisolo')
    meta.f1 = f1;
end
meta.pen = struct('x',pen_x,'z',pen_z,'c',pen_c,'area',pen_area,'jumpZ'
   ,pen_jump, 'sweepSlope',pen_sweepSlope,'dihedralSlope',
   pen_dihedralSlope, 'lineX',pen_line_x,'lineZ',pen_line_z, 'refX',
   pen_ref_x,'refZ',pen_ref_z,'refC',pen_ref_c, 'secArea',pen_secArea,'
   relArea',relArea, 'hardCurvature', pen_curv_hard, 'TaperRatio',
   pen_taper);
if cfg.newTwist
    geo = struct('xle',Xle,'yle',Yle,'zle',Zle,'c',Chord,'ainc',Ainc);
else
    geo = struct('xle', Xle, 'yle', Yle, 'zle', Zle, 'c', Chord);
end
if ~cfg.keepTemps
    safeDelete(runFile);
    safeDelete(forceFile);
    safeDelete(coeffFile);
    if cfg.Formation
        safeDelete(ACWM_outFile);
    end
end

%---- Logging (append to MAT file) ----
logEntry = struct('f', f, 'f0', f0, 'coeff', res, 'xle', Xle, ...
```

```
    'yle', Yle, 'zle', Zle, 'c', Chord);
if ~isempty(q)
    send(q, logEntry);
end

    function H = huber(r, delta) % Huber loss penalty
        a = abs(r);
        H = zeros(size(r));
        mask = (a <= delta);
        H(mask)  = 0.5 * r(mask).^2;
        H(~mask) = delta * (a(~mask) - 0.5*delta);
    end

    function d2 = d2_nonuniform(y, f)
        y = y(:); f = f(:); N = numel(y);
        d2 = zeros(N-2,1);
        for j = 2:N-1
            hm = y(j) - y(j-1);
            hp = y(j+1) - y(j);
            d2(j-1) = 2 * ((f(j+1) - f(j))/max(hp,1e-12) - ...
                (f(j) - f(j-1))/max(hm,1e-12))/max(hm+hp,1e-12);
        end
    end

    function [r_out,kbest] = piecewiseLinRes(y,x,j,useHuber,huberTol)
        x = x(:);
        y = y(:);
        n = numel(y);
        if nargin < 3, j = []; end
        if nargin < 4 || isempty(useHuber), useHuber = false; end
        if nargin < 5 || isempty(huberTol)
            huberTol = 0.02 * max(1e-9, mean(abs(x)));
        end
        if n < 3
            r_out = zeros(n,1); kbest = NaN; return;
        end

        candJ = 2:(n-1);
        if ~isempty(j) && isfinite(j) && j >= 2 && j <= n-1
            if ~ismember(j, candJ), candJ = [candJ, j]; end
        end

        % Search best breakpoint
        bestCost = inf; bestR = zeros(n,1); kbest = candJ(1);
        for jj = candJ
            if jj >= 2
                if jj >= 2
                    r1 = safeLineResidual(y(1:jj), x(1:jj));
                else
                    r1 = zeros(jj,1);
                end
            else
                r1 = zeros(jj,1);
```

```
            end
            nRight = n - jj;
            if nRight >= 2
                r2 = safeLineResidual(y(jj+1:end), x(jj+1:end));
            else
                r2 = zeros(nRight,1);
            end
            r = [r1; r2];

            % Convert to nonnegative measure
            if useHuber
                rr = huber(r, huberTol); cost = mean(rr);
            else
                rr = abs(r); cost = mean(rr);
            end
            if cost < bestCost
                bestCost = cost; bestR = rr; kbest = jj;
            end
        end
        r_out = bestR;
    end

    function r = safeLineResidual(yseg, xseg)
        % Residuals of best affine fit x ~ a*y + b on the segment.
        m = numel(yseg);
        if m >= 2
            p = polyfit(yseg, xseg, 1); r = xseg - polyval(p, yseg);
        else
            r = zeros(m,1);
        end
    end
end

%% ======================== EXTRA FUNCTIONS ========================
function ref = parseAVL_Wing(avlPath)
% Extract SECTION data from SURFACE [Wing] in an AVL file.
ref = struct('x',[],'y',[],'z',[],'c',[],'ainc',[], ...
    'airfoil',{{}},'claf',[]);

if ~exist(avlPath,'file')
    warning('Missing %s', avlPath); return;
end
fid = fopen(avlPath,'r');
if fid<0, warning('Cannot open %s',avlPath); return; end
C = onCleanup(@() fclose(fid));
lines = textscan(fid,'%s','Delimiter','\n','Whitespace','');
lines = lines{1};

isWing = false;
for i = 1 : numel(lines)
    L = strtrim(lines{i});
    if startsWith(upper(L),'SURFACE')
        j = i + 1;
```

```
        while j <= numel(lines) && strlength(strtrim(lines{j})) == 0
            j = j + 1;
        end
        if j <= numel(lines) && strcmpi(strtrim(lines{j}),'[Wing]')
            isWing = true; i = j; continue;
        else
            isWing = false; continue;
        end
    end
    if isWing && contains(upper(L),'!ENDSURFACE'), break; end
    if isWing && startsWith(upper(L),'SECTION')
        while i <= numel(lines) && ~contains(lower(strtrim(lines{i})),
           '!beginsection')
            i = i + 1;
        end
        if i > numel(lines), break; end
        i = i + 2;
        if i > numel(lines), break; end
        t = sscanf(lines{i},'%f');
        if numel(t) < 5
            warning('SECTION structure not correct in %s', avlPath);
            continue;
        end
        x = t(1); y = t(2); z = t(3); c = t(4); a = t(5);
        af = ''; % default airfoil
        k = i + 1;
        while k <= numel(lines) && ~(contains(lower(strtrim(lines{k})),
           '!endsection') || contains(upper(strtrim(lines{k})),'CLAF'))
            line_k = strtrim(lines{k});
            if startsWith(upper(line_k),'AFILE') || ~isempty(regexp(
               line_k,'^\d{4,5}$','once'))
                af = [strtrim(lines{k+1})];
            end
            k = k + 1;
        end
        claf = 1;
        while k <= numel(lines) && ~contains(lower(strtrim(lines{k})),
           '!endsection')
            line_k = strtrim(lines{k});
            if startsWith(upper(line_k),'CLAF') || ~isempty(regexp(
               line_k,'^\d{4,5}$','once'))
                claf = str2double(strtrim(lines{k+1}));
            end
            k = k + 1;
        end
        i = k;

        % Save
        ref.x(end+1) = x; ref.y(end+1) = y; ref.z(end+1) = z;
        ref.c(end+1) = c; ref.ainc(end+1) = a;
        ref.airfoil{end+1} = af;  ref.claf(end+1) = claf;
    end
end
```

```
% Ensure row vectors
fld = {'x','y','z','c','ainc'};
for k = 1 : numel(fld)
    ref.(fld{k}) = ref.(fld{k})(:).';
end
end

function af_out = resampleAirfoils(y_ref, af_ref, y_target, caseDir)
% Interpolated airfoil from the one of af_ref
af_out = repmat({''}, 1, numel(y_target));
if isempty(af_ref{1}), return;
else
    coords_ref = cell(1, 2);
    coords_ref{1} = readAirfoil(af_ref{1});
    coords_ref{end} = readAirfoil(af_ref{end});
    outFolder = fullfile(caseDir, 'airfoils');
    [~, caseName] = fileparts(caseDir);
    if ~exist(outFolder, 'dir'), mkdir(outFolder); end

    Npts = 300;
    x_common = linspace(0, 1, Npts)';
    for i = 1 : numel(y_target)
        if (coords_ref{1}(:,2) - coords_ref{end}(:,2)) == 0
            af_out{i} = af_ref{1};
        else
            yt = y_target(i);
            % Interpolation factor
            t = (yt - y_ref(1)) / (y_ref(end) - y_ref(1));
            t = max(min(t,1),0);
            % Interpolate root airfoil
            [xu1, yu1, xl1, yl1] = splitAirfoil(coords_ref{1});
            yu1i = interp1Unique(xu1, yu1, x_common);
            yl1i = interp1Unique(xl1, yl1, x_common);
            % Interpolate tip airfoil
            [xu2, yu2, xl2, yl2] = splitAirfoil(coords_ref{end});
            yu2i = interp1Unique(xu2, yu2, x_common);
            yl2i = interp1Unique(xl2, yl2, x_common);
            % Blend upper and lower surfaces
            yu_interp = (1-t)*yu1i + t*yu2i;
            yl_interp = (1-t)*yl1i + t*yl2i;
            % Assemble in Selig format
            X = [flipud(x_common); x_common(2:end)];
            Y = [flipud(yu_interp); yl_interp(2:end)];
            % Save to file
            savename = fullfile(caseName, 'airfoils', sprintf('
               airfoil_section_%d', i));
            filename = fullfile(caseName, 'airfoils', sprintf('
               airfoil_section_%d.dat', i));
            fid = fopen(filename, 'w');
            fprintf(fid, 'airfoil_section_%d\n', i);
            fprintf(fid, '%.6f %.6f\n', [X Y]');
            fclose(fid);
            if isfile(savename)
```

```
                safeDelete(savename);
            end

            % Generation and run of the XFOIL file
            fileID = fopen('XFOILInput.txt','w');
            fprintf(fileID, 'load %s\n', filename);
            fprintf(fileID, 'ppar\n');
            fprintf(fileID, 'n\n');
            fprintf(fileID, '%d\n\n\n', Npts);
            fprintf(fileID, 'save\n');
            fprintf(fileID, '%s\n', savename);
            fprintf(fileID, 'quit\n');
            fclose(fileID);
            status = system("xfoil < XFOILInput.txt");
            if status ~= 0
                error('XFOIL non ha eseguito correttamente.');
            end
            af_out{i} = savename;
            safeDelete(filename);
        end
    end
end
end

function coords = readAirfoil(filename)
data = readmatrix(filename, 'FileType', 'text');
coords = data(~any(isnan(data),2), :);
end

function [xu, yu, xl, yl] = splitAirfoil(coords)
[~, idxLE] = min(coords(:,1));
xu = coords(1:idxLE, 1); yu = coords(1:idxLE, 2);
xl = coords(idxLE:end, 1); yl = coords(idxLE:end, 2);
end

function yi = interp1Unique(x, y, xq)
[xu, ia] = unique(x, 'stable');
yu = y(ia); yi = interp1(xu, yu, xq, 'linear', 'extrap');
end

function y_half = stationY(span_total, N, spacing, root)
b2 = span_total/2;
switch lower(spacing)
    case 'equal'
        y_half = linspace(root, b2, N);
    case 'cosine'
        y_half = root + 0.5 * (b2-root) * (1-cos(linspace(0,pi,N)));
    case 'halfcosine'
        y_half = root + (b2-root) * (- cos(linspace(pi/2,pi,N)));
    otherwise
        error('Spacing type not found');
end
end
```

```
function S = planformArea(y, c) % Half-wing area by trapezoidal rule
S_half = sum(0.5*(c(1:end-1)+c(2:end)).*diff(y)); S = 2*S_half;
end

function plotPlanform(y, xle, zle, c, outPng)
% Quick visualization: top and front views
figure('Color','w','Position',[100 100 1000 360]);
subplot(1,2,1);
xx = [xle, fliplr(xle + c)]; yy = [y, fliplr(y)];
plot(xx, yy, '-'); hold on; plot(xx, -yy, '-'); axis equal; grid on;
ylim([-ceil(y(end)), ceil(y(end))]);
xlabel('x [m]'); ylabel('y [m]'); title('Planform (top)');
subplot(1,2,2);
plot(y, zle, '-o'); hold on; plot(-y, zle, '-o'); axis equal; grid on;
xlabel('y [m]'); ylabel('z [m]'); title('Dihedral (front)');
if nargin>4 && ~isempty(outPng)
    exportgraphics(gcf, outPng, 'Resolution', 150);
end
end

function safeDelete(p)
try
    if exist(p,'file'), delete(p); end
catch
end
end

function appendHistory(logFile, logVar, entry)
% Appends a single evaluation 'entry' to the .mat history file.
if isfile(logFile) && any(strcmp(who('-file', logFile), logVar))
    S = load(logFile, logVar); H = S.(logVar); H(end+1) = entry;
else
    H = entry;
end
T = struct(); T.(logVar) = H;
if isfile(logFile)
    save(logFile, '-struct', 'T', '-append');
else
    save(logFile, '-struct', 'T');
end
end
```

Function that generates the modified AVL geometry files from a baseline AVL file by editing only the wing surface named `[Wing]`.

```
function ACWingModification(refwing_avl, count, parameter, airfoil,
   claf, optimization_folder, deltaS)
%
% Authors: Federico Poleni, Giacomo Visconti, and Marco Zappoli
%
% -------------------------------------------------------------------
% GENERAL DESCRIPTION
% -------------------------------------------------------------------
% ACWingModification Generate a modified AVL wing geometry file.
%
% Inputs:
%  refwing_avl - string: path to the reference AVL file to read
%  count - integer: index identifying this modified version
%  parameter - 4xN or 5xN matrix: per-section geometry parameters
%              rows: [Xle; Yle; Zle; Chord; (optional) Ainc]
%  airfoil - 1xN cell array of strings: airfoil identifier per section
%  claf - 1xN numeric vector: per-section CLAF values
%  optimization_folder- string (optional): folder to write output into
%  deltaS - scalar (optional): shift applied to Sref (if present)
%
% Output:
%   A new AVL file is written with filename: <FileName>_v<count><ext>
%   Only the SURFACE named [Wing] is modified
% -------------------------------------------------------------------

% -------- INITIALIZE FLAGS AND EXTRACT PER-SECTION ARRAYS --------
% aincVar: boolean flag to indicate whether Ainc row is present
aincVar = false;
Xle = parameter(1, :);
Yle = parameter(2, :);
Zle = parameter(3, :);
Chord = parameter(4, :);
% If the parameter matrix has a fifth row, treat it as Ainc
if size(parameter, 1) == 5
    aincVar = true;
    Ainc = parameter(5, :);
end

% -------- PREPARE INPUT/OUTPUT FILE NAMES AND OPEN FILES --------
[~, FileName, ext] = fileparts(refwing_avl);
oldgeometry = fopen(refwing_avl, 'r');
% If an optimization folder is specified, build the new geometry
% base path there, otherwise write the new file in the current
% working directory with same base name
if nargin > 4
    newgeometryFile = fullfile(optimization_folder, FileName);
else
    newgeometryFile = FileName;
end
newgeometry = fopen(newgeometryFile+"_v"+num2str(count)+ext, 'w');
```

```
% ---------------- SETUP PARSING STATE VARIABLES ----------------
fgetl(oldgeometry);
line = "[" + FileName + "_wing_version_" + num2str(count) + "]";
prev_line = ""; prev_prev_line = ""; index = 0;

% ------- MAIN PARSING LOOP: PROCESS UNTIL "!endgeometry" -------
% Loop reads the reference file line-by-line and writes to the new
% file, applying modifications only within the Wing SURFACE blocks.
while line ~= "!endgeometry"
    % If the previous line was Sref Cref Bref, optionally adjust Sref
    if prev_line == "#Sref Cref Bref"
       if nargin == 7
        ref_geom = str2double(split(line));
        ref_geom(1) = ref_geom(1) + deltaS;
        line = sprintf('%.3f %.3f %.3f', ref_geom);
       end
    % Detect the beginning of the Wing surface block: the pattern is:
    %   SURFACE
    %   [Wing]
    elseif prev_line == "[Wing]" && prev_prev_line == "SURFACE"
        while line ~= "!endsurface"
            if line == "SECTION", index = index+1; end
            if prev_line == "#Xle Yle Zle Chord Ainc Nspanwise Sspace"
                if aincVar
                    line = sprintf('%.3f %.3f %.3f %.3f %.3f 0 0', Xle(
                       index), Yle(index), Zle(index), Chord(index),
                       Ainc(index));
                else
                    line = sprintf('%.3f %.3f %.3f %.3f 0.0 0 0', Xle(
                       index), Yle(index), Zle(index), Chord(index));
                end
            % If the previous line was AFILE, replace the line with the
                user-provided airfoil
            elseif strcmpi(prev_line, "AFILE")
               line = sprintf("%s", airfoil{index});
            % If the previous line was CLAF, replace the line with the
               new CLAF numeric value
            elseif strcmpi(prev_line, "CLAF")
                line = sprintf('%.4f', claf(index));
            end
            % Write the current line to the new geometry file
            fprintf(newgeometry, "%s\n", line);
            prev_line = line;
            line = fgetl(oldgeometry);

            % If it reached the end of the current surface but
            % still have more section definitions to append,
            % append new SECTION blocks for the remaining sections.
            if line == "!endsurface" && length(Xle)>index
                for j = index+1 : length(Xle)
                    index = j;
                    fprintf(newgeometry, "#--------------------" + ...
                        "----------------------------------------\n");
```

```
                    fprintf(newgeometry, "SECTION\n");
                    fprintf(newgeometry, "!beginsection\n");
                    fprintf(newgeometry, "#Xle Yle Zle Chord Ainc
                       Nspanwise Sspace\n");
                    if aincVar
                        fprintf(newgeometry, '%.3f %.3f %.3f %.3f %.3f
                           0 0\n', Xle(index), Yle(index), Zle(index),
                           Chord(index), Ainc(index));
                    else
                        fprintf(newgeometry, '%.3f %.3f %.3f %.3f 0.0 0
                            0\n', Xle(index), Yle(index), Zle(index),
                           Chord(index));
                    end
                    if ~isempty(airfoil{index})
                        fprintf(newgeometry, "AFILE\n");
                        fprintf(newgeometry, "%s\n", airfoil{index});
                    end
                    if claf(index)~=1
                        fprintf(newgeometry, "CLAF\n");
                        fprintf(newgeometry, "%.4f\n", claf(index));
                    end
                    fprintf(newgeometry, "!endsection\n");
                end
            % If the section ended but there are no more parameter
            % sections to add, skip until "!endsurface" is encountered.
            elseif prev_line == "!endsection" && line ~= "!endsurface"
               && length(Xle) == index
                while line ~= "!endsurface"
                    prev_line = line;
                    line = fgetl(oldgeometry);
                end
            end
        end
    end

    % Outside the nested SURFACE handling, write the current line.
    fprintf(newgeometry, "%s\n", line);
    prev_prev_line = prev_line; prev_line = line;
    line = fgetl(oldgeometry);
end
% Write the final "!endgeometry" line and close both files cleanly
fprintf(newgeometry, "%s\n", line);
fclose(oldgeometry);
fclose(newgeometry);
```

Function that runs AVL for a given `.avl` geometry at a specified angle of attack, extracts aerodynamic coefficient outputs written by AVL, aggregates them into a MATLAB struct, saves the result, and returns the struct.

```
function res = GetCoeffAVL_fe(filename_avl, alpha)
%
% Authors: Federico Poleni, Giacomo Visconti, and Marco Zappoli
%
% ------------------------------------------------------------------
% GENERAL DESCRIPTION
% ------------------------------------------------------------------
%
% GetCoeffAVL_fe Run AVL and extract aerodynamic coefficients
% from its "fe" output.
%
% res = GetCoeffAVL_fe(filename_avl)
% res = GetCoeffAVL_fe(filename_avl, alpha)
%
% INPUTS
%  filename_avl - string: path to an AVL geometry file (.avl)
%  alpha        - scalar (optional): overall angle of attack
%                 to use in AVL (deg). If not provided, the function
%                 will attempt to read it with external function.
%
% OUTPUT
%  res - struct containing aerodynamic coefficients.
%        For single-aircraft cases fields correspond to aggregated
%        coefficients (CL, CY, CD, CDi/CDv, etc).
%        For formation cases (filename contains "TwoACFormation")
%        the struct has subfields 'Lead' and 'Trail'.
%
% NOTE
%   This function requires 'avl.exe' available in the system PATH.
%   It creates temporary input/batch files and a temporary AVL output
%   file named "<basename>_force". It expects the AVL force output to
%   contain blocks labelled by surfaces and surfaces.
% ------------------------------------------------------------------

filename_avl = string(filename_avl);
[folderPath, baseFile, ext] = fileparts(filename_avl);
folderPath = string(folderPath); baseFile = string(baseFile);
ext = string(ext);
if ~strcmp(ext, '.avl')
    error('filename_avl must have extension .avl');
end

% If 'alpha' not provided, attempt to read it with external function
if nargin == 1, alpha = readAlpha(filename_avl); end

% Prepare and remove any existing force output file (cleanup)
forceFileFull = fullfile(folderPath, baseFile + "_force");
if isfile(forceFileFull)
    delete(forceFileFull);
end
```

```
% Create a unique temporary input filename for AVL to avoid collisions
count = randi(1e12);
inputFile = sprintf('AVLInput_%d.txt', count);

% Write AVL input command file: the file contains commands to
% load the AVL geometry, enter OPER mode, set alpha, execute,
% request "fe" (force) output and to write the force output
fileID = fopen(inputFile,'w');
fprintf(fileID, 'load %s%s\n', fullfile(folderPath, baseFile), ext);
fprintf(fileID, 'oper\n');
fprintf(fileID, 'a\n');
fprintf(fileID, 'a %g\n', alpha);
fprintf(fileID, 'x\n');
fprintf(fileID, 'fe\n');
fprintf(fileID, '%s\n\n', char(strrep(forceFileFull, '\', '/')));
fprintf(fileID, 'quit\n\n');
fclose(fileID);

%% ======= Run AVL using a temporary batch file (Windows) =======
% A temporary .bat is written that pipes the input file to avl.exe.
batFile = sprintf('run_avl_%d.bat', count);
fid = fopen(batFile,'w');
fprintf(fid,'avl.exe < "%s"\n', inputFile);
fclose(fid);
status = system(sprintf('start /MIN /WAIT cmd /c "%s"', batFile));
if status ~= 0
    error('AVL failed running.');
end
safeDelete(batFile);
safeDelete(inputFile);

%% === Parse the AVL force output and build coefficient structs ===
pairs = { 'CL',  'Cl',  ' CLsurf  = %f  Clsurf  = %f';
          'CY',  'Cm',  ' CYsurf  = %f  Cmsurf  = %f';
          'CD',  'Cn',  ' CDsurf  = %f  Cnsurf  = %f';
          'CDi', 'CDv', ' CDisurf  = %f  CDvsurf  = %f'};

makeCoeffStruct = @() cell2struct(num2cell(zeros(1,2*size(pairs,1))),
   reshape(pairs(:,1:2)',[],1), 2);
% If the filename indicates a formation, create separate structs
if contains(filename_avl, 'TwoACFormation', 'IgnoreCase', true)
    Coeff_lead = makeCoeffStruct();
    Coeff_trail = makeCoeffStruct();
else
    Coeff = makeCoeffStruct();
end

forceFile = fullfile(folderPath, baseFile + "_force");
if ~isfile(forceFile)
    error('Force output file "%s" not found.', forceFile);
end
```

```
fileID = fopen(forceFile, 'r');
line = fgetl(fileID);
% Read line-by-line and search for aerodynamic surfaces.
while ~feof(fileID)
    if contains(line, 'Surface #')
        % Determine whether this surface block belongs to leader or
           trail
        if contains(line, '(lead)'), ind = 1;
        elseif contains(line, '(trail)'), ind = 2;
        else, ind = 0;
        end
        % Skip 3 lines that appear between "Surface #" and the
           numerical coefficient lines in AVL output.
        for i = 1 : 3, line = fgetl(fileID); end

        % Read the four lines that contain the coefficients
        line1 = line; nums1 = sscanf(line1, pairs{1,3});
        line2 = fgetl(fileID); nums2 = sscanf(line2, pairs{2,3});
        line3 = fgetl(fileID); nums3 = sscanf(line3, pairs{3,3});
        line4 = fgetl(fileID); nums4 = sscanf(line4, pairs{4,3});

        % Ensure parsing yielded 2 numbers for each expected line
        if numel(nums1) == 2 && numel(nums2) == 2 && numel(nums3) == 2
           && numel(nums4) == 2
            nums_vec = [nums1'; nums2'; nums3'; nums4'];
            for k = 1:size(pairs,1)
                nums = nums_vec(k, :);
                if ind == 1
                    Coeff_lead.(pairs{k,1}) = Coeff_lead.(pairs{k,1}) +
                        nums(1);
                    Coeff_lead.(pairs{k,2}) = Coeff_lead.(pairs{k,2}) +
                        nums(2);
                elseif ind == 2
                    Coeff_trail.(pairs{k,1}) = Coeff_trail.(pairs{k,1})
                        + nums(1);
                    Coeff_trail.(pairs{k,2}) = Coeff_trail.(pairs{k,2})
                        + nums(2);
                else
                    Coeff.(pairs{k,1}) = Coeff.(pairs{k,1}) + nums(1);
                    Coeff.(pairs{k,2}) = Coeff.(pairs{k,2}) + nums(2);
                end
            end
        end
    else
        line = fgetl(fileID);
    end
end
fclose(fileID);

% If a two-aircraft formation was detected, package the two per-
   aircraft structs into Coeff
if contains(filename_avl, 'TwoACFormation', 'IgnoreCase', true)
    Coeff = cell2struct({Coeff_lead,Coeff_trail},{'Lead','Trail'},2);
```

```
end

try
    baseNameSafe = matlab.lang.makeValidName(baseFile);
    savePath = fullfile(folderPath, baseFile + '.mat');
    S.(baseNameSafe + "_Coeff") = Coeff;
    save(savePath, '-struct', 'S');
catch
end

res = Coeff;

function safeDelete(p)
try
    if exist(p,'file'), delete(p); end
catch
end
end

end
```

Function that builds a combined AVL geometry file representing a two-aircraft formation.

```
function TwoACFormation(lead_avl, Dx, Dy, Dz, trail_avl, AOA_lead,
    AOA_trail)
%
% Authors: Federico Poleni, Giacomo Visconti, and Marco Zappoli
%
% ------------------------------------------------------------------
% GENERAL DESCRIPTION
% ------------------------------------------------------------------
% TwoACFormation Create a two-aircraft formation AVL file.
%
% Purpose:
%  Construct a single AVL geometry file that contains two aircraft
%  (lead and trail) in formation. The trail aircraft geometry is
%  placed relative to the lead via non-dimensional offsets Dx, Dy, Dz
%  (normalized by the mean wing span). If only one .avl file is
%  provided, the same geometry is used for both vehicles.
%
% Inputs:
%  lead_avl   - path to the lead aircraft .avl file
%  Dx, Dy, Dz - normalized offsets between homologous points
%  trail_avl  - (optional) path to the trail aircraft .avl file;
%               if omitted, trail_avl is set equal to lead_avl
%  AOA_lead   - (optional) AOA to force for the lead aircraft
%  AOA_trail  - (optional) AOA to force for the trail aircraft
%
% Output:
%  A new .avl file is written in the lead aircraft folder with a name
%  indicating the two-aircraft formation.
%
% Notes / assumptions:
%  - If AOA_lead/AOA_trail are provided, the function replaces section
%    ANGLE values for non-vertical/non-cockpit surfaces accordingly.
%  - The code preserves the full lead geometry and appends a modified
%    trail geometry, inserting a TRANSLATE block after the 'ANGLE'.
%  - If Dx == 2.0, Dy == 1.0 e Dz == -0.50, then it will obtain the
%    reference situation of "F/A-18 AIRCRAFT PERFORMANCE BENEFITS
%    MEASURED DURING THE AUTONOMOUS FORMATION FLIGHT PROJECT"
%    (AIAA 2002-4491), M. Jake Vachon, Ronald J. Ray, Kevin R. Walsh
%    and Kimberly Ennix.

% If trail_avl is missing or empty, use the lead geometry only.
if nargin < 5 || isempty(trail_avl)
    trail_avl = lead_avl;
elseif ~isfile(trail_avl)
    error('Trail .avl file not found: %s', trail_avl);
end
% If AOA_lead is not provided, diffAngle=false and the original
% ANGLE lines are kept.
if nargin < 6, diffAngle = false;
else, diffAngle = true;
end
```

```
[leadFolder, leadName] = fileparts(lead_avl);
[~, trailName] = fileparts(trail_avl);
% Define the output filename: if lead and trail use the same file
% name, use a concise naming convention.
if strcmp(lead_avl, trail_avl)
    outFile = fullfile(leadFolder, leadName + "_TwoACFormation.avl");
else
    outFile = fullfile(leadFolder, leadName + "_" + trailName + "
       _Formation.avl");
end

leadID = fopen(lead_avl, 'r');
if leadID == -1
    error('Could not open lead .avl file: %s', lead_avl);
end
formationID = fopen(outFile, 'w');
if formationID == -1
    fclose(leadID);
    error('Could not create formation .avl file: %s', outFile);
end

% Initialize parser state variables used while copying the lead file.
prev_line = ""; bref_lead = []; max_Lcomp = 0;
fgetl(leadID);
if strcmp(trail_avl, lead_avl)
    line = "[" + leadName + "_two-aircraft_formation]";
else
    line = "[" + leadName + "_and_" + trailName + "_in_formation]";
end
% Copy lead geometry to the formation file while capturing metadata
while line ~= "!endgeometry"
    if strcmp(prev_line, "#Sref Cref Bref")
        ref_geom = str2double(split(line)); bref_lead = ref_geom(end);
        % Encountering a SURFACE, record the surface name as the
        % 'lastSurface' so that next 'ANGLE' lines can be linked with
           it.
    elseif strcmp(prev_line, "SURFACE")
        % Append " (lead)]" to surface headers so the output file
        % clearly labels lead surfaces.
        lastSurface = line; line = erase(line, "]") + " (lead)]";
    elseif strcmp(prev_line, "COMPONENT")
        Lcomp = str2double(line);
        if ~isnan(Lcomp), max_Lcomp = max(Lcomp, max_Lcomp); end
    elseif diffAngle && strcmp(prev_line, "ANGLE") && ~(contains(upper(
       lastSurface), 'VERTICAL') || contains(upper(lastSurface), '
       COCKPIT'))
        line = sprintf("%.5f", AOA_lead);
    end
    fprintf(formationID, "%s\n", line);
    prev_line = line; line = fgetl(leadID);
end
fclose(leadID);
```

```
trailID = fopen(trail_avl, 'r');
if trailID == -1
    fclose(formationID);
    error('Could not open trail .avl file: %s', trail_avl);
end

% Reset state variables used for reading the trail file.
prev_line = ""; bref_trail = [];
line = fgetl(trailID);
while line ~= "!begingeometry"
    if strcmp(prev_line, "#Sref Cref Bref")
        ref_geom = str2double(split(line)); bref_trail = ref_geom(end);
    end
    prev_line = line; line = fgetl(trailID);
end

if isempty(bref_lead) || isempty(bref_trail)
    fclose(trailID); fclose(formationID);
    error('Could not determine Bref for lead or trail aircraft.');
end

% Compute the average wingspan and convert normalized offsets
% Dx,Dy,Dz into absolute distances.
bavg = (bref_lead + bref_trail)/2;
DX = bavg * Dx; DY = bavg * Dy; DZ = bavg * Dz;

% Append trail geometry to formation file, modifying indices,
% YDUPLICATE and ANGLE
prev_line = line;
while ~feof(trailID)
    line = fgetl(trailID);
    % Mark SURFACE names as belonging to the trail aircraft.
    if strcmp(prev_line, "SURFACE")
        lastSurface = line; line = erase(line, "]") + " (trail)]";
    elseif strcmp(prev_line, "YDUPLICATE")
        line = sprintf("%.3f", DY);
    elseif strcmp(prev_line, "COMPONENT")
        line = sprintf("%d", str2double(line) + max_Lcomp);
    elseif diffAngle && strcmp(prev_line, "ANGLE") && ~(contains(upper(
       lastSurface), 'VERTICAL') || contains(upper(lastSurface), '
       COCKPIT'))
        line = sprintf("%.4f", AOA_trail);
    end
    fprintf(formationID, "%s\n", line);
    % Immediately after an ANGLE block insert a translation block
    % to place the trail relative to the lead.
    if strcmp(prev_line, "ANGLE")
        fprintf(formationID,"#\nTRANSLATE\n%.3f %.3f %.3f\n",DX,DY,DZ);
    end
    prev_line = line;
end
fclose(trailID); fclose(formationID);
```

Helper function that reads a previously saved text file containing the angle(s) of attack for a single aircraft or a two-aircraft formation.

```
function alpha = readAlpha(avlFile, formazione)
%
% Authors: Federico Poleni, Giacomo Visconti, and Marco Zappoli
%
% ------------------------------------------------------------------
% GENERAL DESCRIPTION
% ------------------------------------------------------------------
% readAlpha Read stored angle-of-attack values.
%
% alpha = readAlpha(avlFile)
% alpha = readAlpha(avlFile, formazione)
%
% INPUTS
%  avlFile    - path to the .avl (or base) file used to derive naming
%  formazione - logical (optional): if true, read formation alpha file
%               that contains 'Alpha_lead' and 'Alpha_trail'
%
% OUTPUT
%  alpha - scalar when formazione==false: the single aircraft alpha
%          1x2 vector when formazione==true: [alpha_lead, alpha_trail]
%
% NOTES
%  - If the file is missing, the function returns alpha = 0.0.
% ------------------------------------------------------------------

% Default for formation flag.
if nargin < 2, formazione = false; end

avlFile = string(avlFile);
[folderPath, baseFile, ~] = fileparts(avlFile);
folderPath = string(folderPath);
baseFile = string(baseFile);

% Construct the expected alpha filename depending on whether
% formation results are requested.
if formazione
    alphaFile = fullfile(folderPath, 'formation_angle_output', sprintf(
       '%s_formation_alpha',baseFile));
else
    alphaFile = fullfile(folderPath, sprintf('%s_alpha',baseFile));
end
% If the alpha file does not exist, return 0.0 and print info.
if ~isfile(alphaFile)
    alpha = 0.0;
    fprintf('File "%s" non trovato.\n', alphaFile);
    fprintf('Impostato alpha = %.1f\n', alpha);
    return;
end

% Read the alpha file contents as text lines
fid = fopen(alphaFile,'r');
```

```
lines = textscan(fid,'%s','Delimiter','\n','Whitespace','');
fclose(fid);
lines = lines{1};

if formazione
    alphaLines = lines(contains(lines,'Alpha'));
    % Extract 'Alpha_lead = <value>'
    leadtokens = regexp(alphaLines, 'Alpha_lead\s*=\s*([-\d.]+)', '
       tokens');
    leadtokens = leadtokens(~cellfun(@isempty,leadtokens));
    if ~isempty(leadtokens)
        alpha_lead = str2double(leadtokens{1}{1});
    else
        warning('Alpha_lead not found. Setting to NaN.');
        alpha_lead = NaN;
    end
    % Extract 'Alpha_trail = <value>'
    trailtokens = regexp(alphaLines, 'Alpha_trail\s*=\s*([-\d.]+)', '
       tokens');
    trailtokens = trailtokens(~cellfun(@isempty,trailtokens));
    if ~isempty(trailtokens)
        alpha_trail = str2double(trailtokens{1}{1});
    else
        warning('Alpha_trail not found. Setting to NaN.');
        alpha_trail = NaN;
    end
    % Return a two-element vector: [lead, trail]
    alpha = [alpha_lead, alpha_trail];
else
    lines = lines(contains(lines, 'Alpha ='));
    tokens = regexp(lines{1}, 'Alpha\s*=\s*([-\d.]+)', 'tokens');
    alpha = str2double(tokens{1}{1});
end
```

Function that automates a numerical search for a pair of angles-of-attack (leader, trail) that produce a desired trim condition in two-aircraft formation.

```
function FormationAlpha()
%
% Authors: Federico Poleni, Giacomo Visconti, and Marco Zappoli
%
% ------------------------------------------------------------------
% GENERAL DESCRIPTION
% ------------------------------------------------------------------
% FormationAlpha Find formation angles-of-attack that match a target CL
%
% This top-level driver:
% - locates candidate case folders that contain <folder>/<folder>.avl
% - for each selected case, reads a reference alpha and reference CL
% - runs a parallel particle-swarm optimization to find AOA_lead and
%   AOA_trail that minimize a formation fitness based on CL differences
% - writes an evaluation history to
%   formation_angle_output/Formation_Alpha_history.mat.
% ------------------------------------------------------------------

%% ========================= USER CONFIG ==========================
cfg.avlExe       = 'avl';
cfg.keepTemps    = false; % keep candidate .avl files
cfg.AllFolders   = false; % optimization performed on all folders
cfg.SpecFolder   = 'F-22'; % specifies which folder to consider
cfg.Dx = 2.0;
cfg.Dy = 1.0;
cfg.Dz = -0.5;

%% ================== FIND REFERENCE FOLDERS ==================
rootDir = pwd; folders = dir(rootDir); caseDirs = {};
for k = 1 : numel(folders)
    if folders(k).isdir && ~startsWith(folders(k).name, '.')
        cand = fullfile(rootDir, folders(k).name);
        refAVL = fullfile(cand, [folders(k).name, '.avl']);
        if exist(refAVL,'file'), caseDirs{end+1} = cand; end
    end
end
if isempty(caseDirs)
    warning('No folder with <folder>/<folder>.avl found.');
    L = dir(fullfile(rootDir, '*.avl'));
    if ~isempty(L), caseDirs = {rootDir}; else, return; end
end
% If the user did not request all folders, filter by cfg.SpecFolder
if ~cfg.AllFolders
    newcaseDirs = {};
    for icase = 1 : numel(caseDirs)
        caseDir = caseDirs{icase}; [~, caseName] = fileparts(caseDir);
        if contains(cfg.SpecFolder, caseName)
            newcaseDirs{end+1} = caseDirs{icase};
        end
    end
    caseDirs = newcaseDirs;
```

```
end

%% ========================== MAIN LOOP ==========================
for icase = 1 : numel(caseDirs)
    caseDir = caseDirs{icase}; [~, caseName] = fileparts(caseDir);
    fprintf('\n===== CASE: %s =====\n', caseName);

    % Ensure per-case output directory exists:
    outDir = fullfile(caseDir, 'formation_angle_output');
    if ~exist(outDir,'dir'), mkdir(outDir); end

    % Determine reference AVL to use
    refFile = fullfile(caseDir, [caseName, '.avl']);
    if ~exist(refFile,'file')
        L = dir(fullfile(caseDir,'*.avl'));
        if isempty(L)
            warning('No .avl in %s. Skipping.', caseDir); continue;
        end
        refFile = fullfile(caseDir, L(1).name);
        [~,caseName] = fileparts(refFile);
    end

    % Read stored alpha for the reference geometry
    cfg.alpha_deg = readAlpha(refFile);

    % Copy the reference .avl into the output folder;
    % the optimization operates on copies.
    copyfile(refFile, outDir);
    newrefFile = fullfile(caseName, 'formation_angle_output', sprintf('
       %s.avl', caseName));
    newforceFile = fullfile(caseName, 'formation_angle_output', sprintf
       ('%s_force', caseName));
    ref = GetCoeffAVL_fe(newrefFile, cfg.alpha_deg); CL_ref = ref.CL;
    safeDelete(newrefFile); safeDelete(newforceFile);

    % Define optimizer bounds around the reference alpha
    lb = [cfg.alpha_deg-1.5, cfg.alpha_deg-1.5];
    ub = [cfg.alpha_deg+1.5, cfg.alpha_deg+1.5];
    nvars = numel(lb);

    % Setup a DataQueue so worker processes can send results serially.
    logFile = fullfile(outDir, 'Formation_Alpha_history.mat');
    logVar  = matlab.lang.makeValidName(sprintf('%
       s_Formation_Alpha_history', caseName));
    q = parallel.pool.DataQueue;
    afterEach(q, @(entry) appendHistory(logFile, logVar, entry));

    % Objective function
    objFun= @(AOA) optimizationObjective(AOA,caseName,outDir,CL_ref,q);

    % Particle-swarm options
    opts = optimoptions('particleswarm', 'SwarmSize', 20, '
       MaxIterations', 50, 'Display', 'iter', 'UseParallel', true, '
```

```
        ObjectiveLimit', 1e-4);
    [AOAbest,fbest,~,~] = particleswarm(objFun, nvars, lb, ub, opts);
    [~, AOA_optimal] = objFun(AOAbest);
    fprintf('\n[CASE %s] Best f = %.6g | AOA_lead=%.4f, AOA_trail=%.5f\
        n', caseName, fbest, AOA_optimal(1), AOA_optimal(2));
end

%% ========================== EVALUATOR ==========================
    function [fitness_formation, AOA_case] = optimizationObjective(AOA,
        caseName, outDir, CL_target, q)

        evalCount = randi(1e12);

        % Unpack the candidate AOA pair and prepare the return vector.
        AOA_lead = AOA(1); AOA_trail = AOA(2);
        AOA_case = [AOA_lead, AOA_trail];

        try
            copyfile(refFile, fullfile(outDir, sprintf('%s_%d.avl',
                caseName, evalCount)));
            copiedFile = fullfile(caseName, 'formation_angle_output',
                sprintf('%s_%d.avl', caseName, evalCount));
            TwoACFormation(copiedFile, cfg.Dx, cfg.Dy, cfg.Dz, [],
                AOA_lead, AOA_trail);
            runFile = fullfile(caseName, 'formation_angle_output',
                sprintf('%s_%d_TwoACFormation.avl', caseName, evalCount)
                );
            forceFile = fullfile(caseName, 'formation_angle_output',
                sprintf('%s_%d_TwoACFormation_force', caseName,
                evalCount));
            coeffFile = fullfile(caseName, 'formation_angle_output',
                sprintf('%s_%d_TwoACFormation.mat', caseName, evalCount)
                );
            res = GetCoeffAVL_fe(runFile, 0);

            % Remove temporary files unless the request to keep them
            if ~cfg.keepTemps
                try
                    if exist(copiedFile, 'file')
                        delete(copiedFile);
                    end
                    if exist(runFile, 'file')
                        delete(runFile);
                    end
                    if exist(forceFile, 'file')
                        delete(forceFile);
                    end
                    if exist(coeffFile, 'file')
                        delete(coeffFile);
                    end
                catch
                end
            end
```

```
            % Extract lift coefficients for lead and trail
            CL_lead = res.Lead.CL; CL_trail = res.Trail.CL;

            % Fitness is the sum of absolute deviations of
            % each aircraft CL from the target CL.
            fitness_formation = abs(CL_lead - CL_target) + abs(CL_trail
                - CL_target);
            if isnan(fitness_formation) || fitness_formation > 5
                fitness_formation = NaN;
            end

            % Package an entry for the DataQueue
            parRes.Alpha_lead = AOA_lead;
            parRes.Alpha_trail = AOA_trail;
            parRes.coeff = res;
            parRes.fitness = fitness_formation;
        catch ME
            parRes.Alpha_lead = AOA_lead;
            parRes.Alpha_trail = AOA_trail;
            parRes.coeff = [];
            parRes.fitness = NaN;
        end
        if ~isempty(q), send(q, parRes); end
    end

    function safeDelete(p)
        try
            if exist(p,'file'), delete(p); end
        catch
        end
    end

    function appendHistory(logFile, logVar, entry)
        % Appends a single evaluation 'entry' to the .mat history file.
        if isfile(logFile) && any(strcmp(who('-file', logFile), logVar)
            )
            S = load(logFile, logVar);
            H = S.(logVar); H(end+1) = entry;
        else
            H = entry;
        end
        T = struct(); T.(logVar) = H;
        if isfile(logFile)
            save(logFile, '-struct', 'T', '-append');
        else
            save(logFile, '-struct', 'T');
        end
    end
end
```

Function that performs a parametric evaluation of formation positions using a Latin-hypercube of candidate offsets.

```
function FormationEval()
%
% Authors: Federico Poleni, Giacomo Visconti, and Marco Zappoli
%
% ------------------------------------------------------------------
% GENERAL DESCRIPTION
% ------------------------------------------------------------------
% FormationEval Evaluate formation performance across multiple
% relative positions.
%
% Purpose:
%  For each aircraft case folder in the current working directory
%  that contains a same-named .avl file (<folder>/<folder>.avl),
%  run a position sweep using a predefined Latin-hypercube of offsets
%  For each position, particle-swarm search is performed over AOA_lead
%  and AOA_trail to find angles that match a baseline lift coefficient.
% ------------------------------------------------------------------

%% ========================= USER CONFIG ==========================
cfg.avlExe       = 'avl';
cfg.keepTemps    = false; % keep candidate .avl files
cfg.AllFolders   = false; % optimization performed on all folders
cfg.SpecFolder   = 'F-22'; % specifies which folder to consider

%% ======================== LATIN HYPERCUBE ========================
% Predefined set of formation offsets (Dx,Dy,Dz) to evaluate.
X = [3, 4, 3, 4, 2, 4, 2, 3];
Y = [0.5, 1.5, 1.5, 1, 0.5, 0.5, 1.5, 1];
Z = [0, 0.5, 0, 0.5, -0.5, 0.5, -0.5, 0];
Npts = length(X);

%% ================== FIND REFERENCE FOLDERS ==================
rootDir = pwd; folders = dir(rootDir); caseDirs = {};
for k = 1 : numel(folders)
    if folders(k).isdir && ~startsWith(folders(k).name, '.')
        cand = fullfile(rootDir, folders(k).name);
        refAVL = fullfile(cand, [folders(k).name, '.avl']);
        if exist(refAVL,'file'), caseDirs{end+1} = cand; end
    end
end
if isempty(caseDirs)
    L = dir(fullfile(rootDir, '*.avl'));
    if ~isempty(L), caseDirs = {rootDir}; else, return; end
end
% If the user did not request all folders, filter by cfg.SpecFolder
if ~cfg.AllFolders
    newcaseDirs = {};
    for icase = 1 : numel(caseDirs)
        caseDir = caseDirs{icase};
        [~, caseName] = fileparts(caseDir);
        if contains(cfg.SpecFolder, caseName)
```

```
            newcaseDirs{end+1} = caseDirs{icase};
        end
    end
    caseDirs = newcaseDirs;
end

%% ========================= MAIN LOOP ==========================
for icase = 1 : numel(caseDirs)
    caseDir = caseDirs{icase}; [~, caseName] = fileparts(caseDir);
    fprintf('\n===== CASE: %s =====\n', caseName);

    % Ensure per-case output directory exists
    outDir = fullfile(caseDir, 'formation_position_output');
    if ~exist(outDir,'dir'), mkdir(outDir); end

    % Determine reference AVL to use
    refFile = fullfile(caseDir, [caseName, '.avl']);
    if ~exist(refFile,'file')
        L = dir(fullfile(caseDir,'*.avl'));
        if isempty(L)
            warning('No .avl in %s. Skipping.', caseDir); continue;
        end
        refFile = fullfile(caseDir, L(1).name);
        [~,caseName] = fileparts(refFile);
    end

    % Read reference alpha previously saved for the geometry
    alpha_ref = readAlpha(refFile);

    % Copy the reference file into the output directory and run AVL
    copyfile(refFile, outDir);
    newrefFile = fullfile(caseName, 'formation_position_output',
       sprintf('%s.avl', caseName));
    newforceFile = fullfile(caseName, 'formation_position_output',
       sprintf('%s_force', caseName));
    ref = GetCoeffAVL_fe(newrefFile, alpha_ref); CL_ref = ref.CL;
    safeDelete(newrefFile); safeDelete(newforceFile);

    % Define optimizer bounds around the reference alpha
    lb = [alpha_ref-2, alpha_ref-2]; ub = [alpha_ref+2, alpha_ref+2];
    nvars = numel(lb);

    % Retrieve optimal angles previously found by the FormationAlpha
    data = load(fullfile(caseName, 'formation_angle_output', '
       Formation_Alpha_history.mat'));
    dot = sprintf('%s_Formation_Alpha_history', strrep(caseName, '-', '
       _'));
    data = data.(dot);
    alpha_lead = data(end).Alpha_lead;
    alpha_trail = data(end).Alpha_trail;
    coeffFinal = data(end).coeff; fitness = data(end).fitness;

    % Initialize a results container and store the baseline entry
```

```
    caseResults = struct('Dx',{},'Dy',{},'Dz',{},'alpha_lead',{}, ...
        'alpha_trail',{},'coeff',{},'fitness',{});
    resEntry = struct();
    resEntry.Dx = 2; resEntry.Dy = 1; resEntry.Dz = -0.5;
    resEntry.alpha_lead  = alpha_lead;
    resEntry.alpha_trail = alpha_trail;
    resEntry.coeff = coeffFinal; resEntry.fitness = fitness;
    caseResults(end+1) = resEntry;

    % Loop through the Npts candidate positions
    for j = 1 : Npts
        Dx = X(j); Dy = Y(j); Dz = Z(j);
        fprintf('\n===== POSITION: DX = %.1f , DY = %.1f , DZ = %.1f
           =====\n', Dx, Dy, Dz);

        % Local objective function
        objFun = @(AOA) optimizationObjective(AOA, caseName, outDir,
           CL_ref, Dx, Dy, Dz, refFile);

        % Particleswarm options
        opts = optimoptions('particleswarm', 'SwarmSize', 20, '
           MaxIterations', 50, 'UseParallel', true, 'Display', 'iter',
           'ObjectiveLimit', 4e-4);
        [AOAbest, ~] = particleswarm(objFun, nvars, lb, ub, opts);
        [~, ~, parRes] = objFun(AOAbest);
        alpha_lead = parRes.Alpha_lead;
        alpha_trail = parRes.Alpha_trail;
        coeffFinal = parRes.coeff; fitness = parRes.fitness;

        % Store per-position results into the caseResults array
        resEntry = struct();
        resEntry.Dx = Dx; resEntry.Dy = Dy; resEntry.Dz = Dz;
        resEntry.alpha_lead = alpha_lead;
        resEntry.alpha_trail = alpha_trail;
        resEntry.coeff = coeffFinal; resEntry.fitness = fitness;
        caseResults(end+1) = resEntry;
    end
    % Save all per-case position results into formation_position_output
    formation_position_values = caseResults;
    save(fullfile(outDir, 'formation_position_values.mat'), '
       formation_position_values');
end

%% ========================== EVALUATOR ==========================
    function [fitness_formation, AOA_case, parRes] =
       optimizationObjective(AOA, caseName, outDir, CL_target, Dx, Dy,
       Dz, refFile)

        evalCount = randi(1e12);
        % Unpack the candidate AOA pair and prepare the return vector.
        AOA_lead = AOA(1); AOA_trail = AOA(2);
        AOA_case = [AOA_lead, AOA_trail];
```

```
        copyfile(refFile, fullfile(outDir, sprintf('%s_%d.avl',
            caseName, evalCount)));
        copiedFile = fullfile(caseName, 'formation_position_output',
            sprintf('%s_%d.avl', caseName, evalCount));
        try
            TwoACFormation(copiedFile, Dx, Dy, Dz, [], AOA_lead,
                AOA_trail);
            runFile = fullfile(caseName, 'formation_position_output',
                sprintf('%s_%d_TwoACFormation.avl', caseName, evalCount)
                );
            forceFile = fullfile(caseName, 'formation_position_output',
                sprintf('%s_%d_TwoACFormation_force', caseName,
                evalCount));
            coeffFile = fullfile(caseName, 'formation_position_output',
                sprintf('%s_%d_TwoACFormation.mat', caseName, evalCount
                ));
            res = GetCoeffAVL_fe(runFile, 0);

            % Extract lift coefficients for lead and trail
            CL_lead = res.Lead.CL; CL_trail = res.Trail.CL;

            % Fitness is the sum of absolute deviations of
            % each aircraft CL from the target CL.
            fitness_formation = abs(CL_lead - CL_target) + abs(CL_trail
                - CL_target);
            if isnan(fitness_formation) || fitness_formation > 5
                fitness_formation = NaN;
            end

            % Package the evaluation record for storage
            parRes.Alpha_lead = AOA_lead;
            parRes.Alpha_trail = AOA_trail;
            parRes.coeff = res; parRes.fitness = fitness_formation;
        catch ME
            fitness_formation = NaN;
            parRes.Alpha_lead = AOA_lead;
            parRes.Alpha_trail = AOA_trail;
            parRes.coeff = []; parRes.fitness = NaN;
        end
        % Remove temporary files unless the request to keep them
        if ~cfg.keepTemps
            safeDelete(copiedFile); safeDelete(runFile);
            safeDelete(forceFile); safeDelete(coeffFile);
        end
    end

    function safeDelete(p)
        try
            if exist(p, 'file'), delete(p); end
        catch
        end
    end
end
```